

Università degli Studi di Napoli
Federico II

A.A. 2026-2027

# Tecnologie Web

# Indice


0 Note sul corso 3

I DNS 4

1 Lezione 1 DNS: Domain Name System 5
- I nomi 5
- Domini, zone e delega 5
  - Dominio vs Zona 5
- Tipi di record 5
  - Principali tipi di record DNS 6
- Gli attori 6
- Query ricorsive vs iterative 6
- Interazioni DNS: query e risposte 6
- Risposte DNS 7
  - Risposte NOERROR 7
- Esempio di risoluzione: cold-cache 7
  - La cache nella risoluzione DNS 7
- Pratica 7

II HTTP 9

2 Lezione 2 HTTP: The Hypertext Transfer Protocol 10
- Il Contesto: Web Technologies 10
  - Ipertesti 10
- HTTP: Hypertext Transfer Protocol 10
  - Caratteristiche e Ruolo 10
  - URL: Uniform Resource Locator 10
  - Intestazioni delle Richieste 10
- Risposte HTTP 10
  - Struttura di una Risposta 10
  - Codici di Stato HTTP 11
  - L'Intestazione Host 11
- Caratteristica Statelessness 11
- Server HTTP 11
  - Definizione e Ruoli 11
  - Modalità di Elaborazione delle Richieste 12
  - Server HTTP Statici 12
- Risoluzione DNS per Siti Web Locali 12

3 Lezione 3 HTTP Rappresentazioni, Efficienza e Sicurezza 13
- Content Negotiation 13
  - Principi fondamentali 13
  - Pesi e fallback 13
  - Header di negoziazione comuni 13
- Caching 14
  - Benefici 14
  - Problematiche critiche della cache 14
  - Livelli, cache private e condivise 14
  - Il dilemma della consistenza 14
  - Tre decisioni fondamentali 14
  - L'header Cache-Control 14
  - Freschezza 15
  - Validatori: ETag e Last-Modified 15
  - Caching e Content Negotiation 16
- HTTPS e Sicurezza della Comunicazione 17
  - HTTPS e TLS 17
  - Proprietà e limiti 17
- HTTP/2: Multiplexing e Efficienza 17
  - Limitazioni di HTTP/1.1 con richieste parallele 17
  - Multiplexing in HTTP/2 17
- HTTP/3 e QUIC 18
  - Caratteristiche di QUIC 18

III HTML 19

4 Lezione 4 HTML: Hypertext Markup Language 20
- Web e HTML 20
  - Struttura del documento 20
  - Commenti 20
- Elementi fondamentali 20
  - Titoli e paragrafi 20
  - Semantica a livello di testo 20
  - Ancore e URL 21
  - Tabelle 21
  - Liste 21
  - Riferimenti di carattere 21
  - Immagini 22
- Form 22
  - Controlli di input 22
  - Invio (submission) 22
- Organizzazione del contenuto 23
- Errori HTML e recupero dei browser 24
- DevTools 24

# IV CSS 25

## 5 Lezione 05 CSS: Cascading Style Sheets 26

Ruolo del CSS nel web 26
Stili di default del browser (user agent styles) 26
Includere stili in una pagina 26
Selettori 26
Selettori semplici 27
Selettori composti (compound) 27
Combinatori 27
Pseudo-classi 28
Pseudo-elementi 28
La cascata (the cascade) 28
Origine e importanza 29
Layers 29
Specificità 30
Posizione e ordine di apparizione 30
Ereditarietà (inheritance) 31

## 6 Lezione 06 CSS: Layouts, Responsive Design 32

Unità di misura (sizing units) 32
Panoramica 32
Unità assolute 32
Percentuali 32
Unità relative al font 32
Unità relative al viewport 32
Il box model 33
Layout 33
Flow layout (normal flow) 33
La proprietà display 33
Floats 34
Positioning 34
Layout moderni: Flexbox e Grid 34
Flexbox: flex container 34
Flexbox: flex items 35
Flexbox: gestione dello spazio libero 35
Grid 35
Media queries 36
Tipi di output 36
Media features 36
Responsive design 36
Fixed-width layouts 37
Liquid (fluid) layouts 37
Siti separati (mobile) 37
Adaptive layouts 37
Responsive layouts 37
Viewport meta tag 38

## 7 Lezione 07: Modern, Adaptive CSS 39

Pseudo-classi funzionali 39
`:is()` (matches-any) 39
`:where()` 39
`:not()` 39
`:has()` 39
Specificità delle pseudo-classi funzionali 39
Custom properties (variabili CSS) 40
Ereditarietà 40
Fallback 40
At-rule `@property` 40
Valori calcolati 40
`calc()` 41
`min()` e `max()` 41
`clamp()` 41
Unità viewport su dispositivi mobili 41
Pattern di layout moderni 41
`gap` 41
Shorthand `flex` 41
Container queries 41
Candidati immagine responsive 42
`srcset` 42
`<picture>` e art direction 42
Media query e preferenze utente 42
Query `@supports` 43
Nesting nativo del CSS 43
Selettore di nesting: `&` 43

# V JavaScript 44

## 8 Lezione 8 JavaScript 45

Il linguaggio 45
Includere JavaScript in una pagina 45
Strict mode 45
Sintassi di base 45
Operatori 45
Controllo di flusso 46
Variabili e scope 46
Ciclo di vita 46
Dichiarazione 46
Hoisting 47
`var` e dichiarazione implicita (pre-ECMAScript 6) 47
Scope 47
Funzioni 47
Scope interno e visibilità 47
Function expressions 48
Funzioni annidate e closure 48
Oggetti 48
Proprietà 48
Riferimenti 48
Cloning 48
Metodi 49
`this` 49
Costruttori 49
Optional chaining 49
Configurazione delle proprietà 49
Getter e setter 50

## 9 Lezione 9 JavaScript 51

Prototipi ed ereditarietà 51
Il prototype 51
Metodi: due alternative 52

Strutture dati 52
Array 52
Destructuring 52
Iterabili e Symbol.iterator 53
Map 53
Set 54
Classi 54
La sintassi class 54
Ereditarietà: extends 54
Gestione degli errori 55
Errori e try/catch/finally 55
instanceof 55
throw 56
Moduli 56
Modularità 56
export e import 56

10 Lezione 11 --- JavaScript in a Browser Environment 58
L'ambiente browser 58
Host environment 58
window 58
DOM 58
BOM 58
Il DOM 59
Struttura ad albero 59
Tipi di nodo 59
Trovare elementi 59
Navigare nel DOM 60
Navigazione solo elementi 60
Gerarchia delle classi dei nodi 60
Proprietà dei nodi 61
Manipolare il DOM 62
Creare nuovi elementi 62
Metodi di inserimento 62
Interagire con l'utente 63
Eventi 63
Definizione e classificazione 63
Handler 63
`this` negli handler 64
L'oggetto event 64
Event bubbling 64
Event delegation 64
Fermare il bubbling 65
Generare eventi (dispatching) 65
Capturing 65
Ordine di esecuzione degli script 66

11 Lezione 11 JavaScript asincronismo e richieste di rete 67
Storage dei dati nel browser 67
Cookie 67
localStorage 67
sessionStorage 67
IndexedDB 67
Sincronismo e asincronismo 67
Codice sincrono 68
Codice asincrono 68
Callback 68
Limiti delle callback 68
Promise 68
Produttore e consumatore 68
Creazione di una Promise 69
Ciclo di vita 69
then 69
catch 69
finally 70
Chaining 70
Metodi statici dell'API Promise 70
async e await 71
async 71
await 71
Richieste di rete 71
XMLHttpRequest 71
fetch 72
Lavorare con la risposta 72
Corpo della risposta 72

VI Server Side 73

12 Lezione 12 Web statico e dinamico 74
Richiami su Promise e Fetch 74
Promise 74
Fetch API 74
Il web statico 74
App web statiche 75
Limitazioni del web statico 75
Programmazione server-side 75
Generazione on-the-fly 75
CGI 75
Definizione 75
Flusso di una richiesta 76
Input e output del programma 76
Primo script CGI 76
Server-side scripting 77
Limiti dell'approccio CGI 77
Linguaggi di scripting server-side 77
PHP 77

13 Lezione 13 Node.js 79
JavaScript in ambiente server 79
Gestione multi-threaded delle richieste 79
La filosofia di Node.js 79
Il single-threaded event loop 80
Installazione 80
Node.js vs browser 80
npm 80
Il file package.json 81
Installare le dipendenze 81
Distribuire un pacchetto 81
Una prima applicazione web 81
Debugging in VS Code 82
Auto Attach 82
Azioni di debug 82
Live reloading 82

14 Lezione 14 83

15 Lezione 15 Web app con Node.js 84
Il caso di studio 84
Il server HTTP in Node.js puro 84
Routing 84
Percorsi 84
Metodi HTTP 85
File statici 85
Oltre il caso semplice 85
Template engine 86
Pug 86
Include 86
Ereditarietà dei template 87
Locali 87
Condizionali 88
Iterazioni 88
Altri template engine 88
Parsing del body delle richieste 88
Lettura del body 88
Parsing della stringa 89
Anatomia di una web application tipica 89

16 Lezione 16 Sessioni in Node.js 91
HTTP e session tracking 91
Sessioni basate su cookie 91
Gestione della richiesta di login 91
Verifica dell'autenticazione 91
Propagazione del contesto di autenticazione 92
Limiti dell'approccio con cookie 92
Sessioni lato server 92
Esempio di autenticazione con sessioni 93
Implementazione da zero in Node.js 93
Metodi di session tracking 94

17 Lezione 17 Web Frameworks: Express 95
Dalla programmazione server-side ai framework 95
Framework e librerie 95
Libreria software 95
Framework 95
Inversion of Control 95
Componenti dei web framework 96
Framework opinionati 96
Express 96
Prima applicazione 96
Routing 97
Metodi della risposta 98
Rendering dei template 98
Ciclo richiesta-risposta 98
Middleware 99
Gestione degli errori 99
Error handler personalizzati 99
Middleware utili 100
express.static 100
express.urlencoded 100
express.json 100
express-session 100
cookie-parser 100
Flash message 100

18 Lezione 18 Web app con Express 102
Object-Relational Mapping 102
Sequelize 102
Modelli 102
Sincronizzazione dei modelli 103
Creazione, lettura, modifica, cancellazione 103
Associazioni 103
Pro e contro dell'ORM 103
Organizzazione di un'app Express 104
Configurazione dell'applicazione 104
To-do List con Express 104
Configurazione dei middleware 104
Rotte e gestione degli errori 104
Autenticazione con sessioni 105

19 Lezione 19 REST API 106
Dal server-centric all'architettura a due componenti 106
API e REST 106
Fondamenti 106
Risorse nidificate 107
Archetipi di risorse 107
Query string: filtering e paginazione 107
Implementazione di una REST API 107
Sicurezza della API: token e JWT 108
Regole di design REST 109
OpenAPI (Approfondimento) 109

20 Lezione 20 TypeScript 111
JavaScript e i tipi 111
Transpiler 111
Compilazione e downleveling 111
Dichiarare i tipi 112
`any` e inferenza dei tipi 112
Tipi strutturali 112
Array e object types 112
Union types 112
Type alias e interface 113
Altri strumenti di tipizzazione 113
Generics 113
Altri tipi da conoscere 114
Livelli di strictness 114

21 Lezione 21 Frontend Tooling 115
Perché serve del tooling 115
CSS pre e post processori 115
SASS 115
Variabili e nesting 115
Mixin 116
Moduli e partial 116
Liste, mappe e control flow 116
Framework CSS 116
Uso e personalizzazione 117
Bundling e minification 117
Minification 117
Vite 117
La fase di build 118

## 22 Lezione 22 Single Page Applications 119

App multi-page 119
Perché le SPA 119
Client-side routing 119
Anatomia di una SPA minimale 120
Gestione della navigazione 120
Componenti 120
Recupero dei dati: AJAX e XMLHttpRequest 121
Le sfide delle SPA e i framework 121

## 23 Lezione 23 Frontend Frameworks Angular 123

Frontend framework 123
Angular 123
CLI 123
Componenti 123
Standalone components 124
Stato ed eventi 124
Control flow nei template 124
Comunicazione tra componenti 124
Da genitore a figlio: `@Input()` 124
Property binding 125
Da figlio a genitore: `@Output()` 125
Routing 125
Setup 125
Navigazione senza full-reload 126
Link attivi 126

## 24 Lezione 24 Angular (parte II) 127

Form 127
Template-driven 127
Reactive 127
Submission 127
Services e Dependency Injection 127
Proteggere le rotte 128
HttpClient 128
Observables 129
Interceptor 129
Change detection 129
Signals 130
Computed ed effects 130

## 25 Lezione 25 Angular To-do List SPA 131

Rotte e protezione 131
Login con reactive form 131
`AuthService` con signal 132
Servizio REST e intercettore 132
Pagina dei to-do e componente figlio 132

## 26 Lezione 26 Web App Security 134

Sicurezza delle web app 134
La catena dello stack moderno 134
Cross-Site Scripting (XSS) 134
XSS in Angular 135
Cross-Site Request Forgery (CSRF) 135
SQL injection 136
Session hijacking 137
Validazione degli input 137
Il modello di sicurezza dei browser 137
CORS 138

## 27 Lezione 27 Web Testing 139

Verifica del software 139
Livelli di testing 139
Unit testing con Angular 139
Test double e Spy 140
Code coverage 140
End-to-End testing 140
Manuale vs automatizzato 141
Locator strategies 141
Flakiness 141
Isolamento dei test E2E 141
Playwright 142
Isolamento e fixture 142
Locating, actions, asserting 142
Capture and Replay 142

Capitolo 0

# Note sul corso

I seguenti appounti sono stati presi durante le lezioni del corso di Tecnologie Web.
**Nota:**

# Parte I

# DNS

Capitolo 1

# Lezione 1 DNS: Domain Name System

**Definition**

**DNS**

**Database distribuito e gerarchico** di Internet, talvolta descritto in modo improprio come la «rubrica telefonica» della rete. Memorizza e fornisce le informazioni associate ai nomi.

## I nomi

Un nome DNS è una sequenza gerarchica di etichette, lette da destra verso sinistra.

- **Etichetta (label)**: componente elementare del nome; convenzionalmente le etichette sono stampate separate da punti.
- La **gerarchia** si legge da destra verso sinistra: la specificità del nome aumenta da destra verso sinistra.
- Le etichette sono **case insensitive**: non distinguono maiuscole e minuscole.
- **Root**: l'etichetta vuota dopo il punto finale del nome, che rappresenta la radice del namespace.

Nel nome `informatica.dieti.unina.it.` la gerarchia si legge: root → `it` → `unina` → `dieti` → `informatica`.

I nomi DNS sono talvolta chiamati informalmente *hostname*, ma in realtà:

- Un nome può non corrispondere a un host, bensì a un servizio, a un punto di delega, a una policy di instradamento della posta o semplicemente a un nodo del database DNS distribuito.
- Lo stesso host può avere nomi DNS diversi.

## Domini, zone e delega

**Definition**

**Dominio**

**Parte dello spazio dei nomi (namespace)**: un nodo dell'albero e il sottoalbero che vi è radicato.

- **Delega**: suddivisione di un dominio in unità più piccole e gestibili, tramite l'assegnazione dell'autorità su di esse.
- **Zona**: unità più piccola risultante dalla delega.
- Ogni zona è servita da uno o più **name server autoritativi**, che memorizzano e forniscono le informazioni DNS della zona.
- Possono esistere più name server autoritativi per la stessa zona, per ridondanza.
- L'autorità di un name server termina in corrispondenza delle zone figlie delegate.
- Un name server può essere autoritativo per molte zone.

### Dominio vs Zona

Dominio e zona sono concetti diversi, ed è la delega a renderli tali.

- Il **dominio** è un concetto **strutturale**: un sottoalbero del namespace.
- La **zona** è un concetto **amministrativo**: la parte del namespace per la quale un'autorità pubblica dati.

Se `unina.it.` delega `dieti.unina.it.` al DIETI, `dieti.unina.it.` smette di far parte della zona `unina.it.` perché l'autorità è stata delegata, ma continua a far parte del dominio `unina.it.` perché resta nel sottoalbero radicato in `unina.it.`.

> *I domini descrivono l'albero. Le zone descrivono la responsabilità amministrativa su parti dell'albero.*

## Tipi di record

Il DNS memorizza le informazioni come **Resource Record** (RR), descritti dai seguenti campi.

| Owner name | TTL | Class | Type | Value |
|---|---|---|---|---|
| `informatica.dieti.unina.it.` | 21600 | `IN` | `A` | `143.225.97.81` |

- **Owner name**: il nome DNS descritto dal record.
- **TTL** (*time-to-live*): per quanto tempo, in secondi, la risposta può essere conservata in cache.
- **Class**: normalmente `IN`, ovvero Internet.
- **Type**: il tipo di informazione contenuta nel record.
- **Value**: il dato associato al nome.

**Principali tipi di record DNS**

| Tipo | Scopo |
|---|---|
| `A` | Da nome a indirizzo IPv4 |
| `AAAA` | Da nome a indirizzo IPv6 |
| `CNAME` | Alias verso un nome canonico |
| `NS` | Server autoritativi di una zona |
| `SOA` | Identifica una zona |
| `MX` | Server di posta per un dominio |

## Gli attori

Nel passaggio da un nome DNS a un indirizzo IP sono coinvolti diversi attori.

- **Client**: chiede un record di un certo tipo per un dato nome a un'interfaccia locale di risoluzione.
- **Stub resolver**: interfaccia locale di risoluzione gestita a livello di sistema operativo, che riceve la richiesta del client e la delega al resolver ricorsivo.
- **Resolver ricorsivo**: tipicamente gestito da ISP o provider pubblici (Google `8.8.8.8`, Cloudflare `1.1.1.1`); naviga la **gerarchia autoritativa** cercando il record, il che può richiedere il dialogo con più server DNS.
- Il resolver ricorsivo restituisce il record oppure un errore.

## Query ricorsive vs iterative

- **Query ricorsiva**: chiede al server di trovare la risposta definitiva; il server interpellato è responsabile di ottenere il risultato e svolge tutto il lavoro.
- **Query iterativa**: chiede al server la migliore informazione che ha; ogni server può restituire una risposta o un referral, e il resolver segue i referral decidendo quale sia la query successiva.
- La query ricorsiva è usata dal client verso il resolver ricorsivo; la query iterativa è usata dal resolver ricorsivo verso i server DNS.

## Interazioni DNS: query e risposte

Un'interazione DNS consiste normalmente di due messaggi: una **question** e una **response**.

- **Question**: la domanda, es. `informatica.dieti.unina.it. IN A`.
- **Response**: la risposta, articolata in sezioni.

Le sezioni della response sono:

- **Header**: stato e flag.
- **Question**: la domanda, riportata dalla query (echo).
- **Answer**: i record che rispondono alla domanda.
- **Authority**: dati autoritativi o referral.
- **Additional**: record di supporto utili.

Le question contengono tre campi principali: **NAME** (il nome richiesto), **CLASS** (normalmente `IN` per Internet, talvolta omessa) e **TYPE** (il tipo di record richiesto).

# Risposte DNS

Gli header delle risposte DNS iniziano con un **codice di stato**. I principali codici sono:

| Stato | Significato |
|---|---|
| `NOERROR` | Il server ha elaborato la query con successo |
| `NXDOMAIN` | Il nome richiesto non esiste |
| `SERVFAIL` | Il server non è riuscito a completare la query |
| `REFUSED` | Il server ha rifiutato di eseguire l'operazione richiesta |
| `FORMERR` | Il messaggio di query era malformato |

### Risposte NOERROR

`NOERROR` significa che il server ha capito ed elaborato la query, ma non implica che il record sia stato trovato. Gli scenari possibili sono:

- **Risposta positiva**: il blocco Answer contiene il record richiesto.
- **Referral**: il blocco Authority contiene record `NS` di una zona che potrebbe sapere di più.
- **NODATA**: il nome esiste ma non esiste alcun record del tipo richiesto; il blocco Answer è vuoto.

# Esempio di risoluzione: cold-cache

**Example**

Nella risoluzione di un record `A` per `squids.unina.it.` con nessun dato in cache, il processo attraversa tutta la gerarchia.

1. Il client chiede allo stub resolver.
2. Lo stub resolver invia una query DNS al resolver ricorsivo.
3. Il resolver ricorsivo interroga la root e riceve un referral verso `.it`.
4. Il resolver ricorsivo interroga `.it` e riceve un referral verso `unina.it`.
5. Il resolver ricorsivo interroga `unina.it` e riceve il record `A`.
6. Il resolver ricorsivo risponde allo stub resolver.

### La cache nella risoluzione DNS

- Nel mondo reale, browser, stub resolver e resolver ricorsivi memorizzano i dati in una cache per efficienza.
- Se un record è già in cache e non è scaduto, non serve alcuna navigazione della gerarchia.
- **Risoluzione cold-cache**: lo scenario peggiore, in cui nessun dato in cache è disponibile durante la risoluzione.
- Il **TTL** determina per quanto tempo un record in cache può essere usato.

# Pratica

**Lab**

- Con **dig**, analizziamo query e risposte DNS; l'opzione **+trace** mostra la risoluzione gerarchica completa, senza cache, dai root server fino ai server autoritativi:

```bash
$ dig @8.8.8.8 squids.unina.it. +trace
```

- Ogni livello rimanda ai name server del livello successivo: il DNS di Google elenca i root server, il root server selezionato rimanda ai name server del TLD `.it`, questi a quelli di `unina.it`, e infine il name server autoritativo (`dscna2.unina.it`) restituisce il record A:

```Bash
.                   87203   IN      NS      h.root-servers.net.
it.                 172800  IN      NS      m.dns.it.
unina.it.           3600    IN      NS      dscna2.unina.it.
squids.unina.it.    1800    IN      A       143.225.131.206
```

- Se il nome **non esiste** (es. `webtechnologies.unina.it.`), la risposta ha status **NXDOMAIN** e il browser mostra «Server Not Found»:

```Bash
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN
;webtechnologies.unina.it.        IN      A
```

- Se invece il nome **esiste ma non ha record A**, la risposta è **NODATA** (status NOERROR, nessuna sezione ANSWER), è il caso di `docenti.unina.it` rispetto a `www.docenti.unina.it`, che sono **nomi diversi**: solo il primo ha un record A, quindi il secondo restituisce «Server Not Found»:

```Bash
$ dig www.docenti.unina.it A     → NOERROR, A 143.225.212.48
$ dig docenti.unina.it A         → NOERROR, nessuna ANSWER SECTION
```

> *Nota: il DNS è usato anche per ottenere l'IP (record A) del NS selezionato, se la risposta intermedia non lo contiene già nella sezione additional!*

# Parte II

# HTTP

Capitolo 2

# Lezione 2 HTTP: The Hypertext Transfer Protocol

## Il Contesto: Web Technologies

**Internet** è una rete globale di computer interconnessi che condividono informazioni tramite i protocolli Internet; le sue origini risalgono ad **ARPANET** (1969). La **World Wide Web** (WWW, o semplicemente "il Web") non coincide con Internet: ne è un sottoinsieme. Fu inventata da Sir Tim Berners-Lee all'inizio degli anni '90 come sistema di **documenti ipertestuali** interconnessi tramite **hyperlink**, e i suoi componenti fondamentali sono **HTTP** e **HTML**.

### Ipertesti

I documenti tradizionali sono semplici sequenze di caratteri. Un **ipertesto**, invece, è un documento che contiene anche collegamenti ad altri contenuti, come altri ipertesti, documenti o media.

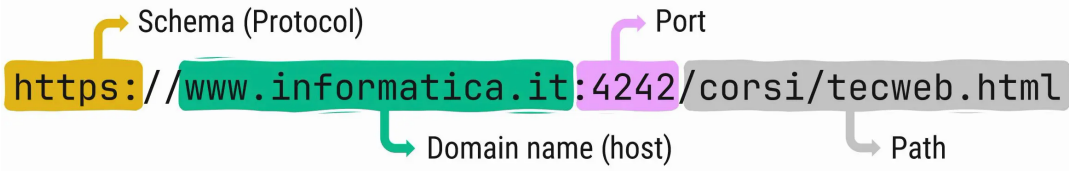
## HTTP: Hypertext Transfer Protocol

### Caratteristiche e Ruolo

**HTTP** è un **protocollo applicativo** costruito sopra **TCP/IP** e rappresenta la fondazione e la spina dorsale della World Wide Web. Sviluppato originariamente per la trasmissione di ipertesti, è oggi utilizzato anche per la trasmissione di altre tipologie di risorse. Nel modello HTTP, il **client invia una richiesta** al server per interagire con una risorsa, e il **server risponde** alla richiesta. Le risorse sono identificate da **URL (Uniform Resource Locator)**.

### URL: Uniform Resource Locator

Un URL completo ha la forma:




### Intestazioni delle Richieste

Le intestazioni HTTP sono meccanismi per passare informazioni aggiuntive nelle richieste e nelle risposte. Un'intestazione è composta da un nome (case-insensitive) seguito da un due-punti e dal valore: `HEADER_NAME: value`.

## Risposte HTTP

### Struttura di una Risposta

Una risposta HTTP è composta da:

- **Linea di Stato**: contiene la versione del protocollo, il codice di stato, e un messaggio descrittivo.
- **Intestazioni (Headers)**: metadati che descrivono la risposta.
- **Linea Vuota**: separa le intestazioni dal corpo della risposta.
- **Corpo della Risposta (facoltativo)**: contiene la risorsa richiesta o ulteriori informazioni.

**Example**

```HTTP
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 153
```

### Codici di Stato HTTP

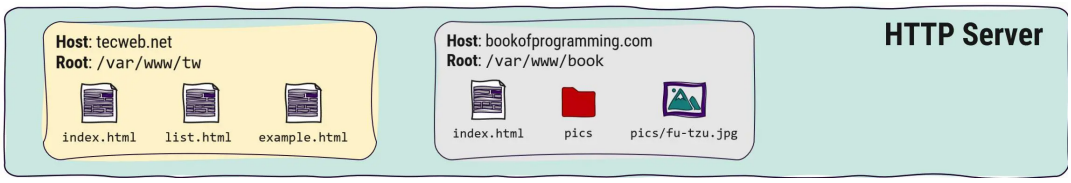
I codici di stato indicano se una richiesta è stata completata con successo. Sono organizzati in cinque classi:

- **1xx (Informazionale)**: Lo scambio richiesta-risposta è in corso.
- **2xx (Successo)**: La richiesta è stata elaborata con successo.
- **3xx (Reindirizzamento)**: Il client deve intraprendere un'azione aggiuntiva, ad esempio un reindirizzamento.
- **4xx (Errore della Richiesta)**: La richiesta non può essere elaborata come presentata (errore nel client).
- **5xx (Errore del Server)**: Il server, o un componente intermedio come un database, ha fallito.

### L'Intestazione Host

Una questione che sorge naturalmente è: perché le richieste HTTP includono un'intestazione **Host** quando il nome di dominio è già stato risolto in indirizzo IP dal DNS?

In realtà **più nomi di host possono risolvere allo stesso IP**, ad esempio `www`, `informatica`, `biblioteca` ed `erasmus.dieti.unina.it` puntano tutti a 143.225.97.81. Poiché un server HTTP può essere configurato per gestire più **virtual host**, ciascuno con la propria document root, è proprio l'header **Host** a permettergli di selezionare il virtual host corretto, poiché il server non conosce automaticamente quale dominio il client ha digitato nel browser; conosce solo l'indirizzo IP verso cui la connessione è stata stabilita.




## Caratteristica Statelessness

**HTTP è un protocollo stateless**, il che significa che ogni richiesta è indipendente dalle precedenti e il server non mantiene informazioni sui progressi delle richieste precedenti di un client.

Questa semplicità è un vantaggio architetturale, ma per implementare conversazioni "statefull" (con memória del contesto) è necessario ricorrere a meccanismi specifici, come i **cookie**.

## Server HTTP

### Definizione e Ruoli

Un **server HTTP** è un software in esecuzione su una macchina host che:

- è in ascolto su una porta specifica (ad esempio 80 per HTTP, 443 per HTTPS);
- riceve richieste HTTP;
- interpreta il metodo, la destinazione e le intestazioni della richiesta;
- produce una risposta HTTP.

Il termine "server" può riferirsi a più entità:

- il **ruolo** del server come partecipante che risponde alle richieste;
- il **software** server che ascolta le richieste HTTP e risponde;
- il **processo** come istanza esecutiva del software;
- la **macchina** su cui il software è in esecuzione.

### Modalità di Elaborazione delle Richieste

Quando un server gestisce una richiesta, può costruire la risposta in modi diversi.

- Può recuperare un file dal proprio file system (servendo **file statici**) e includerne il contenuto nel corpo della risposta.
- Può **eseguire codice per generare una risposta dinamicamente**, il che può coinvolgere interrogazioni di database e interazioni con altri sistemi, con complessità arbitraria.

Tra i server HTTP statici più noti figurano **nginx** e **Apache httpd**.

### Server HTTP Statici

I server statici mappano le risorse richieste a file nel filesystem, secondo una configurazione decisa dal server. Tipicamente si servono i file a partire da una directory specifica, la **document root**: se questa è `/var/www/`, la richiesta `GET /webtech/hello.txt` restituisce `/var/www/webtech/hello.txt`. La root e molte altre impostazioni si definiscono nei file di configurazione dedicati del server.

## Risoluzione DNS per Siti Web Locali

Il suffisso `.local` non è un **Top-Level Domain (TLD)** che può essere acquistato pubblicamente. Per fare in modo che i nomi di dominio vengano risolti all'indirizzo IP corretto (127.0.0.1), è necessario ricorrere a un meccanismo locale. Nei sistemi operativi moderni, è possibile utilizzare il **file hosts** per ottenere questo risultato.

I **risolutori DNS stub** (le componenti locali del sistema di risoluzione dei nomi) consultano questo file prima di inviare la richiesta a un risolutore ricorsivo remoto. Aggiungendo al file hosts una voce come `127.0.0.1 book-of-programming.local`, le richieste DNS per questo dominio verranno automaticamente risolte a 127.0.0.1 (localhost).

# Lezione 3 HTTP Rappresentazioni, Efficienza e Sicurezza

## Content Negotiation

### Principi fondamentali

La **content negotiation** consente a un client di esprimere preferenze rispetto alle rappresentazioni di una risorsa. Il client comunica queste preferenze attraverso specifici header della richiesta, permettendo al server di scegliere una rappresentazione adatta tra quelle disponibili.

### Pesi e fallback

Le preferenze possono essere assegnate con un livello di priorità tramite il parametro `q` (quality factor).

**Example**

HTTP

```http
GET /sections/wisdom.txt HTTP/1.1
Host: book-of-programming.local
Accept: text/plain
Accept-Language: en, it;q=0.9, fr;q=0.8
Accept-Encoding: gzip, deflate, br
Accept-Charset: utf-8, iso-8859-1;q=0.7, *;q=0.7
```

Una possibile risposta è:

HTTP

```http
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Language: it
Content-Encoding: br
Content-Charset: utf-8

Contenuto in italiano.
```

Il server non è obbligato a disporre di tutte le rappresentazioni richieste; se nessuna rappresentazione è accettabile per il client, il server può rispondere con uno status **406 Not Acceptable**. In pratica, tuttavia, i server spesso scelgono un fallback predefinito, ad esempio servendo la versione in una lingua di default se quella richiesta non è disponibile.

### Header di negoziazione comuni

I principali header di negoziazione includono `Accept` (tipo di contenuto), `Accept-Language` (lingua), `Accept-Encoding` (compressione) e `Accept-Charset` (set di caratteri). Ogni header comunica al server quale forma della risorsa il client preferisce.

#### Content Encoding

L'header `Accept-Encoding` della richiesta indica gli algoritmi di compressione lossless che il client è in grado di decodificare. La compressione ha un impatto significativo sulla quantità di dati trasmessi, soprattutto per file di testo di grandi dimensioni. L'header **identity** indica l'assenza di compressione, invece, gli algoritmi più utilizzati sono:

- **gzip** (basato su Lempel-Ziv coding – LZ77),
- **br** (Brotli),
- **deflate** (che impiega la struttura zlib con l'algoritmo deflate).

**Impatto della compressione** Il beneficio della compressione aumenta con la dimensione del file, va però ricordato che comprimere e decomprimere introduce a sua volta un overhead.

# Caching

La cache immagazzina una risposta affinché possa essere riutilizzata per richieste successive.

### Benefici

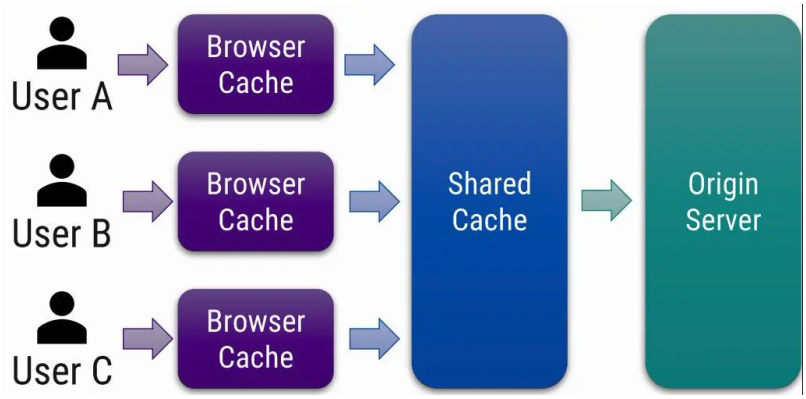
- **Latenza ridotta**: la risposta può essere disponibile più vicina al client.
- **Meno traffico di rete**: la stessa rappresentazione non viene ritrasmessa.
- **Meno lavoro per il server**: un numero inferiore di richieste raggiunge l'origine.
- **Scalabilità**: le cache condivise possono servire molti client.
- **Resilienza**: le cache possono servire risposte memorizzate quando l'origine è temporaneamente non disponibile.

### Problematiche critiche della cache

Il caching non è banale, perché non tutte le risposte vanno memorizzate I metodi **safe** (GET, HEAD) sono sempre cacheabili, mentre PUT e DELETE non lo sono; POST e PATCH sono teoricamente cacheabili se la risposta include header specifici, ma in pratica lo sono raramente.

### Livelli, cache private e condivise

Le risposte possono essere memorizzate in più punti lungo il percorso della richiesta, quindi una risposta può raggiungere il browser senza che il server di origine la elabori. Una cache **privata** serve un singolo utente, come la cache del browser; una cache **condivisa** può riusare la stessa risposta per più utenti, il che è appropriato per i contenuti pubblici ma non per i dati personali, che non devono mai finire per caso a un altro utente. Si noti che **se una risposta può essere memorizzata e chi può riusarla sono due decisioni distinte**.




### Il dilemma della consistenza

La caching introduce un **problema di consistenza**: le risposte memorizzate rappresentano lo stato della risorsa in un momento precedente. La cache deve bilanciare due obiettivi conflittuali: riutilizzare le risposte il più possibile per migliorare le prestazioni, ed evitare di servire risposte inaccettabilmente vecchie o inadatte. Questa tensione è al cuore della progettazione delle strategie di caching.

### Tre decisioni fondamentali

Per ogni risposta, la cache decide tre cose: se può **memorizzarla**, per quanto tempo può **riusarla senza validazione** e, quando il riuso diretto non è permesso, come sapere se la propria copia è ancora attuale mediante **validazione**. L'esito dipende dalla risposta specifica ed è controllato dai suoi **metadati**.

### L'header Cache-Control

L'header **Cache-Control** trasporta direttive che controllano il comportamento di caching. Le direttive sono separate da virgole; alcune ammettono un valore, altre no, e i tempi sono espressi in secondi.

| Direttiva | Significato |
|---|---|
| max-age=300 | La risposta è fresca per 300 secondi. |
| s-max-age=300 | Durata di freschezza per le cache condivise. |
| public | La risposta può essere memorizzata in cache pubbliche. |
| private | La risposta è destinata a una cache privata. |
| no-cache | La risposta può essere memorizzata, ma va validata prima di ogni riuso. |
| no-store | La risposta non va memorizzata in nessuna cache. |
| must-revalidate | Scaduta la freschezza, la risposta va validata prima del riuso. |
| immutable | La rappresentazione non cambierà durante la durata di freschezza. |

### Freschezza

```HTTP
HTTP/1.1 200 OK
Date: Thu, 13 Aug 2026 08:00:00 GMT
Cache-Control: public, max-age=300
```

Mentre una risposta è **fresca** può essere riusata direttamente senza validazione; non appena diventa **stale**, richiede in genere la validazione. La freschezza si calcola dai metadati della risposta, ad esempio `Date` più `max-age`, quindi una risposta può essere memorizzata in cache quando è già scaduta. In assenza di politiche esplicite le cache possono ricorrere a euristiche; per questo i server dovrebbero sempre fornire istruzioni di caching esplicite.

### Validatori: ETag e Last-Modified

Un server può fornire **validatori** nelle sue risposte per supportare una validazione efficiente.

- L'**ETag** (Entity Tag) è un identificatore opaco scelto dal server che cambia quando la rappresentazione cambia. È il validatore più preciso e affidabile.
- **Last-Modified** indica quando il server ritiene che la rappresentazione sia stata modificata per l'ultima volta; è meno preciso di ETag ma comunque utile.

```HTTP
ETag: "article-v7"
Last-Modified: Thu, 13 Aug 2026 08:15:00 GMT
```

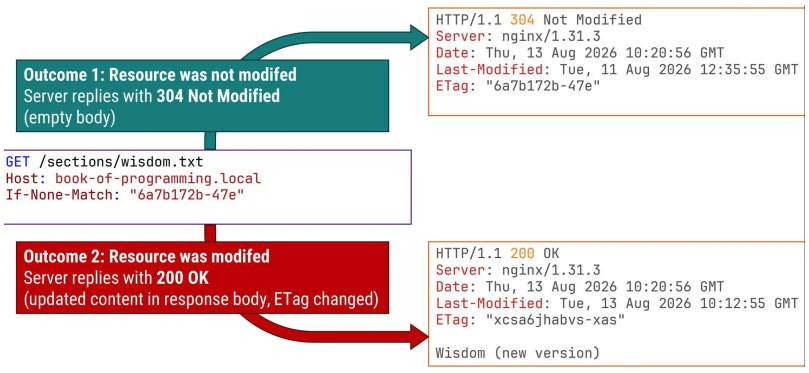
**Definition**

**Identificatore opaco**

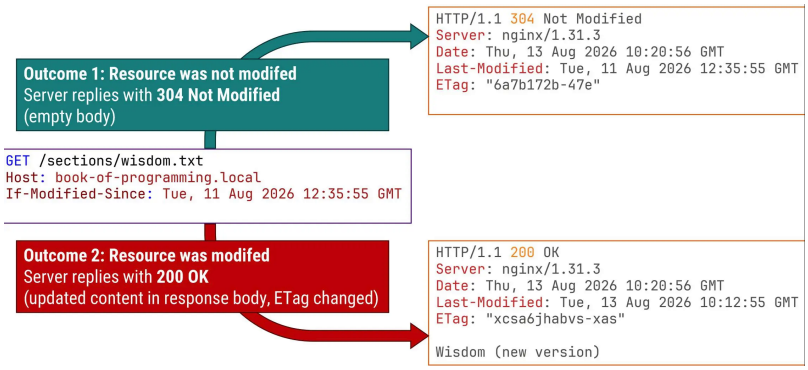
Il client non ha bisogno di conoscerne il significato interno né come è stata generata

### Validazione tramite conditional requests

La validazione avviene tramite **conditional requests**, ovvero richieste normali che includono header di validazione aggiuntivi. Quando una risposta è diventata *stale*, ma aveva un ETag o un Last-Modified, la cache può inviare una richiesta condizionale. Se la cache include l'header `If-None-Match: "article-v7"` e il server conferma che l'ETag è ancora valido, il server risponde con **304 Not Modified** (corpo vuoto), e la cache riusa il corpo memorizzato. Se la risposta è stata modificata, il server risponde con **200 OK** e il nuovo contenuto.



Analogamente, `If-Modified-Since` consente di validare basandosi sulla data di modifica.




### Caching e Content Negotiation

La **content negotiation** può creare varianti della stessa risposta. Un server può fornire dati compressi e non compressi per lo stesso URL. La cache potrebbe necessitare di archiviare risposte separate per ogni variante. Lo stesso accade per altre caratteristiche negoziate come `Accept-Language` e `Accept`. Quando il client invia una richiesta con `Accept-Language: it` non si può servire una risposta memorizzata con `Content-Language: en`.

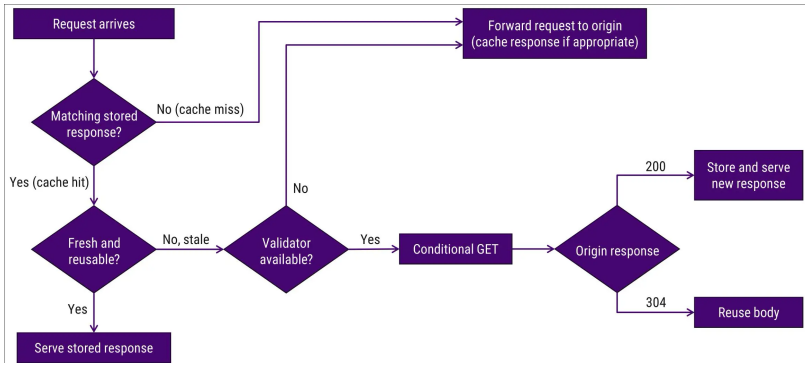
### L'header Vary

Per risolvere questo problema, i server devono utilizzare l'header `Vary` per indicare quali header di richiesta influenzano la risposta.

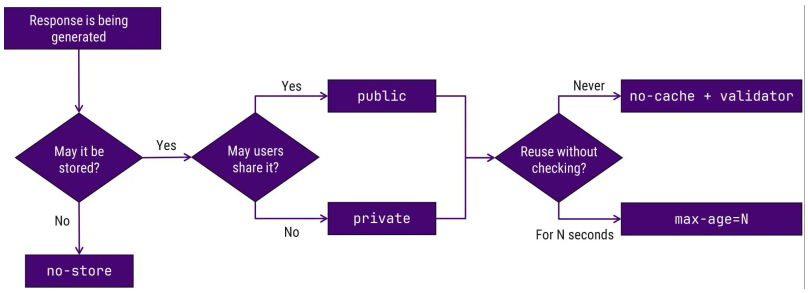
Ciò significa che una risposta precedente può essere riutilizzata per una richiesta successiva solo se la richiesta originale e quella attuale condividono lo stesso URL e metodo, e la richiesta attuale corrisponde all'header Vary della risposta vecchia.

### Flusso decisionale della cache

- Cache view:




- Server view:




# HTTPS e Sicurezza della Comunicazione

Con HTTP un attaccante in grado di osservare o alterare la comunicazione può **leggere** richieste e risposte, rubando credenziali o identificatori di sessione, e può **modificare** le risposte, reindirizzando l'utente o impersonando il server legittimo. **HTTPS** affronta queste minacce a livello di comunicazione.

### HTTPS e TLS

HTTPS ha la **stessa semantica** di HTTP ma usa tipicamente la porta 443 invece della 80: è in sostanza HTTP su un canale **criptato**, la cui sicurezza è garantita da **TLS/SSL**. Concettualmente il funzinamento è questo:

1. Il client richiede `https://squids.unina.it`.
2. Il server presenta un **certificato** contenente gli hostname validi e una chiave pubblica.
3. Il client verifica catena di fiducia, validità temporale e hostname del certificato.
4. Il server dimostra di controllare la corrispondente chiave privata.
5. Viene stabilito un canale cifrato e protetto da integrità.

Con TLS 1.3, dopo il *ServerHello* l'handshake e tutto lo scambio HTTP successivo sono cifrati: un intercettatore vede ancora indirizzi IP, direzioni, tempi e dimensioni dei pacchetti, ma non può leggere metodo, URL, header né corpi delle risposte.

### Proprietà e limiti

HTTPS garantisce tre proprietà fondamentali:

- **riservatezza** (chi ascolta il canale non può leggere le richieste),
- **integrità** (nessuno può alterare richieste e risposte) ,
- **autenticazione del server** (il client verifica l'identità associata all'hostname).

Non bisogna però fidarsi troppo del "lucchetto verde": HTTPS garantisce **solo** la cifratura della comunicazione. Non assicura che il sito non sia malevolo, che le informazioni pubblicate siano corrette, che il server non sia compromesso, né che i dati siano al sicuro prima della trasmissione o dopo il loro arrivo al server.

## HTTP/2: Multiplexing e Efficienza

### Limitazioni di HTTP/1.1 con richieste parallele

Le moderne pagine web richiedono dozzine o centinaia di richieste per essere renderizzate in un browser. HTTP/1.1 può riusare una connessione TCP persistente, ma le risposte non possono essere liberamente interleaved: devono arrivare nell'ordine delle richieste. Ciò significa che una risposta lenta può ritardare tutte le risposte successive. I browser aperti tipicamente molteplici connessioni TCP verso lo stesso server per mitigare questo problema, il che comporta overhead di stabilimento e mantenimento di più connessioni, setup di trasporto e TLS ripetuti, competizione tra connessioni e un utilizzo inefficiente della capacità di rete.

### Multiplexing in HTTP/2

HTTP/2 mira a risolvere queste limitazioni. I messaggi sono rappresentati utilizzando **frame binari**. Ogni coppia richiesta-risposta è assegnata a uno **stream**. I frame provenienti da stream diversi possono essere interleaved, consentendo una serializzazione efficiente su un'unica connessione TCP. La semantica di più richieste HTTP è preservata: le richieste concettualmente rimangono `GET /index.html`, `GET /logo.png`, `GET /style.css`. L'unica differenza è che non sono trasmesse utilizzando la sintassi testuale di HTTP/1.1.

**Example**

Stream A: A1 A2 A3 Stream B: B1 B2 Stream C: C1 C2
Possible serialization over a single TCP connection A1 B1 B2 C1 A2 C2 A3
I frame A1, B1, B2, C1, A2, C2, A3 provenienti da tre stream (A, B, C) possono essere transmessi in questa sequenza mischiata.

### Head-of-Line Blocking e limitazione di TCP

HTTP/2 multiplessa gli stream a livello HTTP, ma TCP non può vedere questi stream indipendenti. TCP presenta un unico flusso di byte affidabile e ordinato; se un segmento TCP è perso, deve essere recuperato prima di consegnare i seguenti.

Questo collo di bottiglia è detto **Head-of-Line (HoL) Blocking**: un singolo elemento ritardato o bloccato arresta gli elementi successivi dal progredire, anche se questi ultimi sono indipendenti.

**Example**

Supponiamo tre streams: $A = (A1, A2) : B = (B1, B2) : C = (C1, C2)$
TCP Segment
Conceptual HTTP/2 bytes
Result
Segment 1 (S1)
A1 and part of B1
Received
Segment 2 (S2)
Rest of B1 and C1
Lost
Segment 3 (S3)
B2 and A2
Received but buffered (waiting for Segment 2)
HTTP/2 non può ricevere lo stream A sebbene A1 e A2 siano stati ricevuti, poiché S2 non contiene dati di A.

# HTTP/3 e QUIC

HTTP/3 è stato introdotto per mantenere lo stile di multiplexing di HTTP/2, evitando tuttavia il collo di bottiglia del TCP Head-of-Line blocking. HTTP/3 trasporta HTTP su **QUIC** anziché direttamente su TCP.

### Caratteristiche di QUIC

QUIC è costruito su **UDP**, fornisce **stream indipendenti multipli**, garantisce affidabilità e controllo della congestione, integra il handshake TLS 1.3 e è supportato da tutti i principali browser. È stato adottato dal 35-40% dei siti web principali.

# Parte III

# HTML

Capitolo 4

# Lezione 4 HTML: Hypertext Markup Language

## Web e HTML

- Il **web** è un sistema di documenti **ipertesto** interconnessi tramite **hyperlink**, recuperati dai client (browser) tramite HTTP.
- Un **server HTTP** è un software in ascolto su una porta che può servire più host (da qui l'header **Host**) e distribuisce i file a partire dalla **document root** del proprio filesystem.
- Una **Web App** consiste di una o più pagine web, descritte in HTML e visualizzate dal browser.

Definition

**Linguaggio di markup**

Un **linguaggio di markup** arricchisce un documento con annotazioni (**tag**, tra parentesi angolari) che ne controllano struttura, formattazione o relazioni tra le parti; i tag di apertura possono contenere **attributi** chiave-valore con valore opzionale.

```HTML
<tagName attr1="value" attr2> ... <\tagName>
```

- HTML esiste dal 1993 in diverse versioni; il riferimento attuale è l'**HTML Living Standard**.

### Struttura del documento

- Il documento inizia con la dichiarazione `<!DOCTYPE HTML>`: non è un tag, indica al client il tipo di documento atteso.
- Il tag `<html>` rappresenta l'intero documento e contiene un `<head>` e un `<body>`; l'attributo **lang** è raccomandato per l'accessibilità.
- Lo `<head>` contiene i **metadati** (dati sul documento, spesso non mostrati all'utente ma utili a browser e motori di ricerca) e deve contenere un `<title>`.
- Lo `<body>` contiene il contenuto effettivo del documento.

Definition

**Attributi globali**

Alcuni attributi sono **globali**, cioè ammessi su qualunque elemento; altri hanno senso solo per alcuni elementi.

- **id**: identificatore univoco dell'elemento nel documento.
- **lang**: lingua del contenuto dell'elemento.
- **style, class**: usati per lo stile.

### Commenti

I **commenti** sono ignorati dai browser, delimitati da `<!--` e `-->`; servono per note o per nascondere temporaneamente contenuto.

## Elementi fondamentali

### Titoli e paragrafi

- `<h1>`–`<h6>` rappresentano titoli e sottotitoli di livello crescente.
- `<p>` rappresenta un paragrafo, che tipicamente inizia su una nuova riga.

### Semantica a livello di testo

- `<em>` enfatizza il contenuto; `<strong>` indica forte importanza; `<br>` è un elemento *void* che inserisce un'interruzione di riga.

- `<abbr title="...">` definisce acronimi e abbreviazioni.
- `<del>` marca contenuto eliminato dal documento; `<ins>` contenuto inserito.

### Ancore e URL

- I **link** si definiscono con l'ancora `<a>`; l'attributo **href** punta all'URL di destinazione.
- Un URL è composto da **schema** (protocollo), **nome di dominio**, **porta** e **percorso**; può includere una **query string** e un **anchor**.

### URL assoluti e relativi

- Un **URL assoluto** include schema e hostname e contiene tutte le informazioni per raggiungere la risorsa.
- Un **URL relativo** specifica solo un percorso; schema e hostname sono dedotti dal contesto corrente.
- Un URL relativo che inizia con `/` sostituisce l'intero percorso; altrimenti sostituisce solo l'ultimo segmento del percorso.
- I segmenti **dot** indicano directory: `.` è la directory corrente, `..` la directory padre.

| href | Percorso risultante (da `/a/b/c/hello.html`) |
|---|---|
| `./index.html` | `/a/b/c/index.html` |
| `../foo.html` | `/a/b/foo.html` |
| `../../pic.jpg` | `/a/pic.jpg` |

- Le URL relative sono da preferire per link interni alla stessa web app (cambiare l'hostname non richiede modifiche alle pagine); per risorse esterne servono URL assoluti.
- L'attributo **target** specifica dove aprire la risorsa: `_self` (default) nella stessa scheda, `_blank` in una nuova scheda/finestra.

### Anchor e index.html

- Un **anchor** (`#id`) è un "segnalibro" dentro la risorsa: indica al browser di mostrare il contenuto in corrispondenza dell'elemento con quell'`id`.
- Quando un URL punta a una directory, il server risponde tipicamente con il file `index.html` in essa (comportamento configurabile: anche `home.html`, `default.htm`, o generazione automatica di un indice).

### Tabelle

- `<table>` contiene righe `<tr>`; ogni riga contiene celle `<td>` o intestazioni `<th>`.
- `<thead>`, `<tbody>` e `<tfoot>` raggruppano rispettivamente intestazioni di colonna, righe di dati e righe di riepilogo; `<caption>` descrive la tabella nel suo insieme.

### Liste

- **Liste ordinate** `<ol>`: per enumerazioni.
- **Liste non ordinate** `<ul>`: per elenchi puntati; entrambe contengono elementi `<li>`.
- **Liste di descrizione** `<dl>`: coppie di termine `<dt>` e descrizione `<dd>`, spesso usate per glossari.

### Riferimenti di carattere

- Alcuni caratteri (es. `<`, `>`) sono **riservati** in HTML: per visualizzarli si usano i **riferimenti di carattere** (entità) nella forma `&nome;` o `&#numero;`.

| Risultato | Riferimento |
|---|---|
| Spazio non separabile | ` ` |
| < | `<` |
| > | `>` |
| & | `&` |
| " | `"` |
| ' | `'` |
| © | `©` |

### Immagini

- `<img>` è un elemento *void* che incorpora un'immagine: **src** specifica l'URL, **alt** il testo alternativo, **width**/**height** le dimensioni in pixel.
- **`alt`** è un sostituto testuale che deve esprimere significato o scopo dell'immagine: sostituendo mentalmente l'immagine col suo testo, la pagina deve comunicare la stessa informazione. > Non usare descrizioni generiche: lo scopo è far trasmettere dagli screen reader lo stesso significato o funzione a chi non vede le immagini.
- Non usare descrizioni generiche: lo scopo è far trasmettere dagli screen reader lo stesso significato o funzione a chi non vede le immagini.

Info

**Testo alt in base allo scopo**

- Immagine **informativa** → testo alternativo significativo.
- Immagine **decorativa** → `alt=""` (vuoto).
- Immagine che è l'unico contenuto di un link → `alt` che descrive la destinazione o lo scopo del link.

Info

**Dietro le quinte**

I documenti HTML erano finora **autonomi** (self-contained). Con `<img>` il contenuto esterno è indicato solo tramite URL: l'immagine non è inclusa nel documento, quindi il browser, che **parsa dall'alto verso il basso**, deve **recuperare (fetch)** risorse aggiuntive per visualizzarlo.

## Form

- L'elemento **`<form>`** raccoglie input dell'utente, tipicamente inviati a un server; contiene controlli come `<input>`, `<label>`, `<textarea>`, `<select>`.

Definition

**Controlli successful e form data set**

All'invio, il **form data set** è costruito raccogliendo tutti i **controlli successful** del form: coppie nome/valore separate da `&`, dove il nome è l'attributo `name` di un input e il valore è quello assunto al momento dell'invio.

### Condizioni per un controllo successful:

- Deve essere definito dentro un `<form>` e avere un attributo **`name`**.
- Un controllo **`disabled`** non è successful (ignorato nell'invio).
- Con più pulsanti di invio, solo quello attivato è successful.
- Tutte le checkbox selezionate possono essere successful.
- Per i radio button con lo stesso `name`, solo quello selezionato è successful.
- L'algoritmo completo è nell'HTML Living Standard.

### Controlli di input

- Il tipo di `<input>` è definito dall'attributo **type**: `text`, `password`, `number`, `radio` (una scelta tra molte), `checkbox` (zero o più scelte), `submit` (pulsante), oltre a tipi dedicati a date e orari (`date`, `week`, `month`, `time`, `datetime-local`) e altri (color picker, file picker, datalist…).
- `<select>` definisce menu a tendina con `<option>`; l'attributo **multiple** permette la selezione multipla.
- L'attributo **disabled** fa sì che un controllo sia ignorato durante l'invio.

### Label e raggruppamento

- **`<label>`** etichetta un input: l'attributo **`for`** deve valere l'`id` dell'input corrispondente; usarle è una buona pratica di usabilità e accessibilità.
- **`<fieldset>`** raggruppa logicamente i controlli, con **`<legend>`** come titolo/caption del gruppo.

### Invio (submission)

Alla sottomissione si genera una nuova richiesta HTTP verso il **form-handler** indicato dall'attributo: - **action** (default: URL della pagina corrente); - **method** indica il metodo HTTP (default: GET).

HTML

```
<form action="/handler.html" method="GET">
Message: <input type="text" name="msg"><br>
Number: <input type="number" name="num"><br>
```

```
4  <input type="submit">
5  </form>
```

I dati raccolti formano il **form data set**: coppie nome/valore `name1=value1&...&nameN=valueN`, dove il nome è l'attributo **name** di ciascun controllo e il valore il suo contenuto all'atto dell'invio.

**Method**

- Con **GET** i dati sono accodati alla URL del handler come **query string** (`/handler.html?msg=Hello!&num=42`): i nomi sono detti **query parameters**, coppie chiave/valore separate da `&`, che il server può usare prima di restituire la risorsa.
- Con **POST** i dati sono inviati nel corpo della richiesta.

**URL Encoding**

La **URL encoding** sostituisce i caratteri speciali con terne `%XX`, dove XX sono due cifre esadecimali che rappresentano il carattere ASCII (gli spazi diventano `%20` o `+`).




### Validazione nativa

- I browser moderni hanno **validazione integrata**: verificano vincoli sull'input e bloccano l'invio se non soddisfatti.
- Vincoli esprimibili con attributi: **type** (tipo atteso), **required** (valore non vuoto), **minlength/maxlength** (lunghezza del testo), **min/max/step** (numeri e date), **pattern** (espressione regolare da rispettare).

Warning

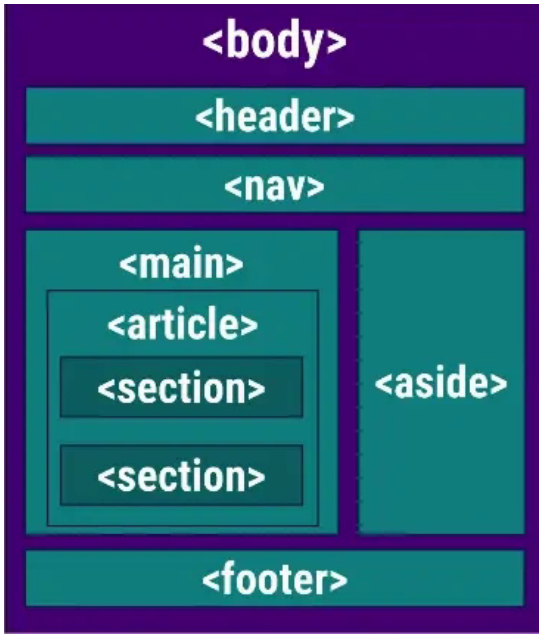
**La validazione HTML non è sicurezza**

La validazione gira nel browser dell'utente, che potrebbe usare un browser senza validazione, modificare l'HTML con le DevTools o inviare richieste HTTP direttamente: un client malintenzionato può inviare qualunque valore. Restano comunque utili per intercettare gli errori presto e migliorare l'esperienza utente.

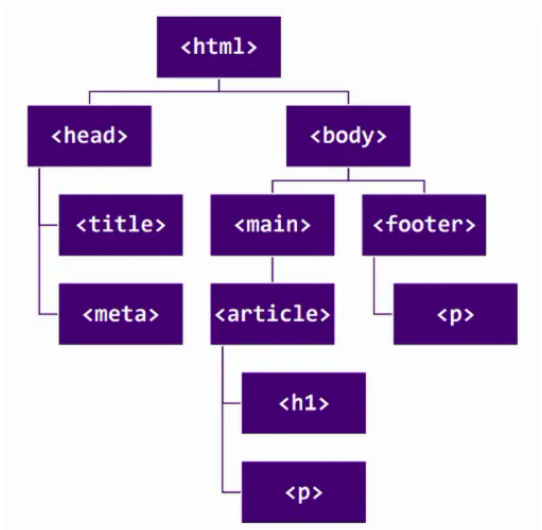
# Organizzazione del contenuto

Il contenuto di una pagina può essere raggruppato con **divisioni** `<div>` (nessuna semantica oltre al raggruppamento) o con **tag semantici**, che descrivono il significato del contenuto a browser, sviluppatori e software.

- `<nav>` contiene link di navigazione;
- `<main>` il contenuto principale;
- `<article>` contenuto indipendente e autonomo;
- `<aside>` contenuto tangenzialmente correlato;
- `<header>`, `<footer>`, `<section>` autoesplicativi.




Un documento HTML è un **albero**: `html` è la radice, con figli `head` e `body`; il nesting di elementi definisce la gerarchia.




## Errori HTML e recupero dei browser

- I browser cercano di visualizzare la pagina anche in presenza di errori (**tag soup**): chiudono automaticamente tag non chiusi, aggiungono markup mancante, correggono entità e problemi strutturali.
- Questo può mascherare errori durante l'apprendimento: una pagina visualizzata correttamente non implica HTML sintatticamente o semanticamente corretto; conviene usare linter (es. HTMLHint) o validatori online (es. validator.w3.org/nu).

## DevTools

**Lab** La scheda "Inspector"/"Analisi pagina" dei DevTools del browser permette di ispezionare l'HTML del documento e modificarlo **localmente**, cioè solo nella copia caricata nel browser, non sul server.

# Parte IV

# CSS

Capitolo 5

# Lezione 05 CSS: Cascading Style Sheets

## Ruolo del CSS nel web

- **HTML** si occupa della struttura e della semantica dei documenti: non dice nulla sull'aspetto (appearance) dei contenuti.
- Un elemento come `<em>` specifica solo che il contenuto va enfatizzato, non come: l'enfasi può essere resa con corsivo, colori o sfondi diversi.

Definition

**CSS**

Linguaggio **dichiarativo e basato su regole** che specifica come i documenti vengono presentati agli utenti.

- Un **stylesheet** è un insieme di **regole**; ogni regola è composta da un **selettore** (quali elementi HTML sono coinvolti) e da un blocco di **dichiarazioni** (coppie proprietà-valore che specificano lo stile da applicare).

```css
selector {
    property: value;
    property: value;
}
```

### Stili di default del browser (user agent styles)

- Il browser applica a ogni pagina dei suoi stili di base, detti **user agent styles**: titoli più grandi e in grassetto, `<p>` a capo riga, `<em>` in corsivo, `<strong>` in grassetto, `<a>` sottolineati e blu, `<ul>` con elenchi puntati, ecc.
- Questi default sono all'incirca uguali tra i vari browser, ma esistono alcune differenze.

## Includere stili in una pagina

- **`<link rel="stylesheet" href="...">`** nel `<head>`: collega un foglio di stile esterno; `rel` specifica la relazione con il documento collegato, `href` l'URL del foglio. Come per `<img>`, il browser fa una richiesta HTTP aggiuntiva per recuperare il foglio prima del rendering.
- **`<style>`** nel `<head>`: definisce regole CSS direttamente dentro il documento HTML; è in genere preferibile usare fogli esterni e `<link>`.
- **Stili inline**: attributo `style` su un singolo elemento; il valore è una sequenza di dichiarazioni separate da `;` e vale solo per l'elemento che lo porta.

```html
<em style="color: fuchsia; font-weight: bold;">inline style</em>
```

## Selettori

Info

**Usi dei selettori oltre allo stile**

I selettori CSS non servono solo per lo stile: con **JavaScript** selezionano gli elementi con cui interagire, nel **testing automatizzato** del web individuano gli elementi su cui agiscono i test, e nello **scraping/crawling** selezionano gli elementi che contengono le informazioni da estrarre.

### Selettori semplici

- **Selettore universale** (wildcard): `*`, corrisponde a qualsiasi elemento.
- **Selettore di tipo**: il nome del tag; corrisponde a tutti gli elementi di quel tipo (es. `a`).
- **Selettore di id**: forma `#ElementId`, corrisponde all'elemento con quell'attributo `id`.
- **Selettore di classe**: forma `.classname`, corrisponde agli elementi con quell'attributo `class` (un elemneto può avere più classi, limportante che siano separati da uno spazio `class=".classname1 classname2 ..."` ).
- **Selettore di attributo**: forma `[attribute]` o `[attribute='value']`, corrisponde agli elementi con quell'attributo (o con quel valore).

```css
[for] { background: yellow; }        /* tutti gli elementi con attributo for */
[type='number'] { background: cyan; } /* tutti gli elementi con type=number */
```

### Operatori di corrispondenza parziale sui valori degli attributi

- `[attribute*='v']`: il valore **contiene** `v`.
- `[attribute^='v']`: il valore **inizia con** `v`.
- `[attribute$='v']`: il valore **termina con** `v`.

```css
[href^='https'] { color: red; }  /* inizia con https */
[href$='.it/']  { color: green; } /* termina con .it/ */
```

### Selettori composti (compound)

- Si **concatenano** selettori senza spazio per ottenere un controllo fine: la selezione è l'**intersezione** dei selettori coinvolti (es. `a[href*='programming'].my-class` corrisponde agli `a` il cui `href` contiene "programming" e che hanno la classe `my-class`).

### Combinatori

- I **combinatori** selezionano elementi in base alla loro posizione nel documento (un documento HTML è un albero); sintassi: `selector1 combinator selector2`.

| Combinatore | Nome | Semantica |
|---|---|---|
| (spazio) | Selettore discendente | `A B` corrisponde a tutti gli elementi che soddisfano `B` contenuti (discendenti) in un elemento `A` |
| `>` | Selettore figlio | `A > B` corrisponde a tutti gli elementi `B` figli **diretti** di `A` |
| `+` | Selettore di fratelli adiacenti | `A + B` corrisponde agli elementi `B` fratelli **immediatamente successivi** a `A` |
| `~` | Selettore generale di fratelli | `A ~ B` corrisponde agli elementi `B` fratelli **successivi** (anche non adiacenti) a `A` |

```css
section em { color: teal; }       /* discendente */
main > em  { color: teal; }        /* figlio diretto */
.master + li { color: red; }       /* fratello adiacente */
.master ~ li.disciple { color: red; } /* fratelli successivi */
```

### Pseudo-classi

**Definition**

**Pseudo-classe**

I elementi HTML possono trovarsi in **stati** diversi (per interazione dell'utente o per relazione con altri elementi); le pseudo-classi, con sintassi che inizia con `:`, permettono di stilizzare gli elementi in base al loro stato.

- **Stati interattivi** (derivanti dall'interazione utente): `:hover` (puntatore del mouse sopra l'elemento), `:active` (elemento attivamente in interazione, es. pulsante premuto), `:focus` (elemento attualmente selezionato/fuocato, es. link o campo input).
- **Stati storici** (memorizzano i link visitati): `:link` (link non ancora visitati), `:visited` (link già visitati). Nota: gli stati storici non sono supportati nel Live Preview di VS Code; vanno verificati in un browser reale.
- **Stati dei form** (specifici dell'interazione con i form): `:disabled`, `:invalid`, `:checked`.
- **Stati di posizione** (relazioni con gli altri elementi tra fratelli, siblings):
  - `:first-child` e `:last-child`: primo/ultimo figlio tra un insieme di fratelli.
  - `:only-child`: elementi senza fratelli.
  - `:first-of-type` e `:last-of-type`: primo/ultimo fratello **considerando solo gli elementi dello stesso tipo**.
  - `:nth-child(n)` e `:nth-of-type(n)`: elemento in ennesima posizione tra i fratelli; accettano anche parole chiave come `even` e `odd`.

**Warning**

**Indicizzazione**

In CSS l'indicizzazione parte da **1**, non da 0.

CSS

```css
em:last-of-type { color: red; }
li:nth-child(even) { color: red; }
li:nth-child(odd)  { color: blue; }
```

### Pseudo-elementi

**Definition**

**Pseudo-elemento**

Pseudo-elemento (sintassi `selector::pseudo-element`) che permette di mirare **parti specifiche del contenuto** di un elemento HTML, senza aggiungere markup HTML aggiuntivo.

- `::first-letter`: la prima lettera del contenuto di un elemento a livello di blocco (block-level).
- `::first-line`: la prima riga del contenuto di un elemento block-level.
- `::selection`: il contenuto attualmente selezionato dall'utente.
- `::before`: crea un elemento che è il **primo figlio** dell'elemento selezionato.
- `::after`: crea un elemento che è l'**ultimo figlio** dell'elemento selezionato.

CSS

```css
p::first-letter { font-weight: bold; }
.narrator::before {
    content: "Narrator >> ";
    font-weight: bold;
}
```

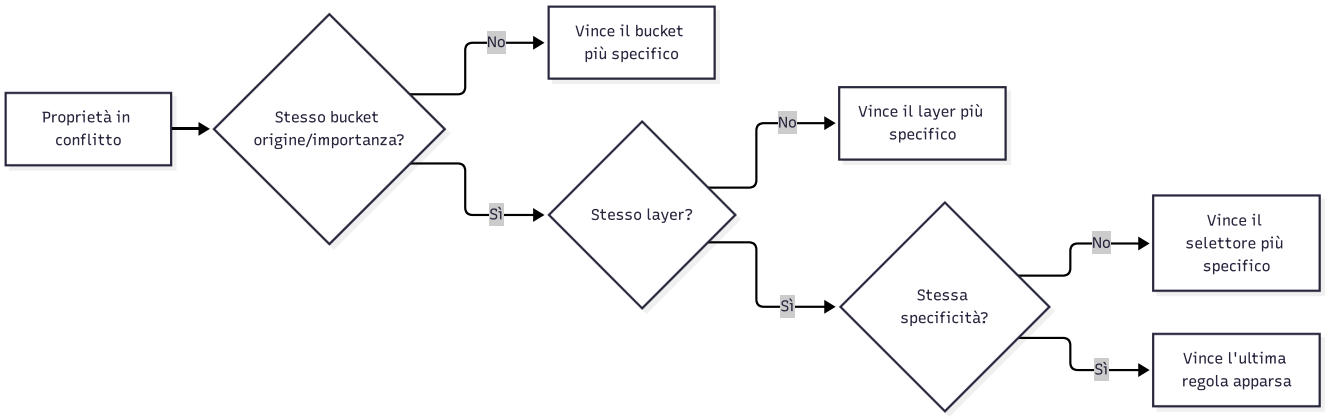
## La cascata (the cascade)

**Definition**

**La cascata**

Algoritmo usato per risolvere i **conflitti** quando due o più regole si applicano allo stesso elemento assegnando valori diversi alla stessa proprietà. Input: un insieme di proprietà conflittuali per un elemento; output: la singola proprietà (con valore) da applicare effettivamente.

- La cascata considera 4 aspetti chiave, in ordine: **origine e importanza**, **layers**, **specificità**, **posizione/ordine di apparizione** della regola.

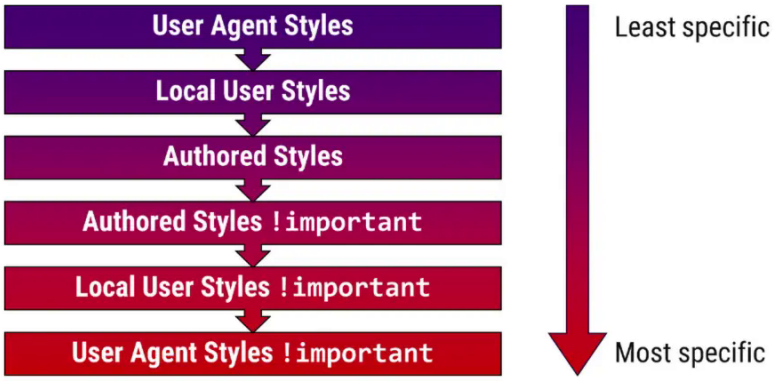



### Origine e importanza

- Il CSS che scriviamo (**authored CSS**) non è l'unico applicato a una pagina: esistono gli **user agent styles** e altri stili (**local user styles**) aggiunti da estensioni del browser o dal sistema operativo, ad esempio per accessibilità (schemi a alto contrasto, font più grandi per ipovedenti).
- L'annotazione `!important`, aggiunta alla fine di una dichiarazione, conferisce più importanza a quella dichiarazione; l'importanza pesa in modo significativo nella cascata.

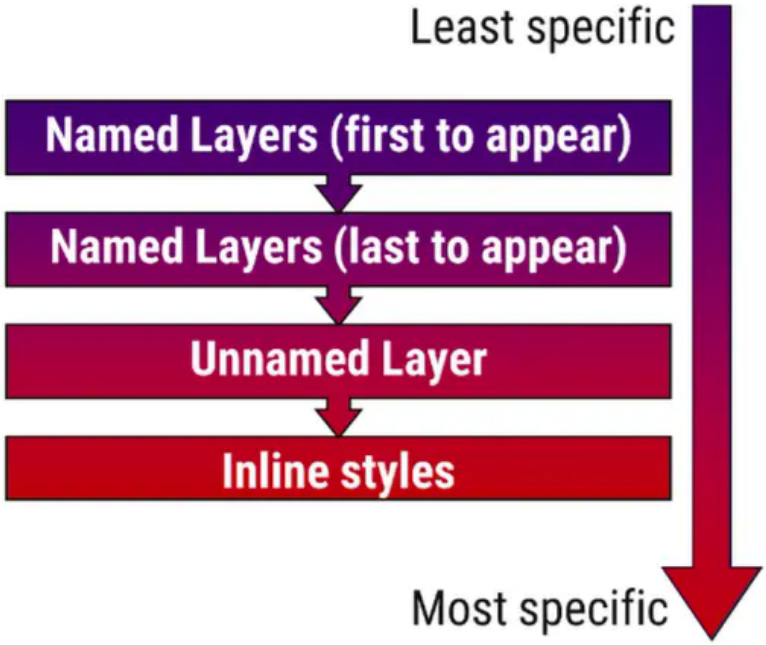
```css
h1 { color: red !important; }
```

- Gli origin bucket vanno dal **meno specifico al più specifico**; l'importanza (`!important`) crea ulteriori sotto-bucket all'interno di ciascuna origine.




### Layers

- Dentro ogni bucket origine/importanza possono esserci più **cascade layers**:
  - Negli stili authored si possono definire **layer personalizzati** con la regola `@layer`; i layer dichiarati **più tardi hanno priorità più alta**.
  - Tutto il CSS in `<style>` o importato con `<link>` appartiene a un **layer senza nome**.
  - Gli **stili inline** appartengono a un layer separato e hanno la **priorità più alta**.




### Specificità

**Definition**

**Specificità**

Una terna numerica **(A, B, C)** calcolata da un selettore: si ignora il selettore universale; **A** = numero di selettori di id; **B** = numero di selettori di classe, attributo e pseudo-classi; **C** = numero di selettori di tipo e pseudo-elementi.

- Il calcolo vale quando due regole in conflitto appartengono allo stesso bucket origine/importanza e allo stesso layer: **vince il selettore più specifico**.

| Selettore | Specificità (A, B, C) |
|---|---|
| `*` | (0, 0, 0) |
| `a:hover` | (0, 1, 1) |
| `article.item section p::first-letter` | (0, 1, 4) |
| `em.master[target]` | (0, 2, 1) |
| `#id` | (1, 0, 0) |
| `#navbar ul li a.nav-link[href*='/']` | (1, 2, 3) |

### Confronto tra specificità

- Le tre componenti si confrontano **in ordine**: vince la specificità con A maggiore; a parità di A, quella con B maggiore; a parità anche di B, quella con C maggiore; se tutti i valori sono pari, le specificità sono **uguali** (pareggio).

**Example**

| Selettore #1 | Spec. #1 | Selettore #2 | Spec. #2 | Vincitore |
|---|---|---|---|---|
| `a[target]` | (0, 1, 1) | `.list a` | (0, 1, 1) | Pareggio |
| `#msg` | (1, 0, 0) | `input[type].inp` | (0, 2, 1) | #1 |
| `#nav > #brd a.lk` | (2, 1, 1) | `em.foo.bar.light` | (0, 3, 1) | #1 |
| `[id='nav'] a` | (0, 1, 1) | `#nav a` | (1, 0, 1) | #2 |

### Posizione e ordine di apparizione

- Se due regole appartengono allo stesso bucket origine/importanza, allo stesso layer e hanno la stessa specificità, **l'ultima regola apparsa ha la priorità più alta**.

- La regola vale sia dentro lo stesso stylesheet, sia in base all'ordine in cui i fogli di stile compaiono.

```css
h1 { color: red; }
h1 { color: blue; } /* vince blue */
```

## Ereditarietà (inheritance)

- Alcune proprietà CSS (**inheritable**), ad esempio `color`, `font-size`, `font-family`, `font-weight`, `font-style`, vengono **ereditate dagli antenati** se all'elemento non è impostato un valore specifico.
- Le proprietà ereditate **non prendono parte alla cascata**: vengono considerate solo quando la cascata non ha prodotto alcun valore per quell'elemento; in un certo senso hanno la specificità più bassa di tutti i metodi di stilizzazione.

```css
p {
    font-family: sans-serif;
    color: red;
}
/* l'elemento <em> dentro <p> eredita color e font-family */
```

Info

**Ispezione nel browser**

Negli strumenti di sviluppo (Dev Tools) gli user agent styles sono **nascosti per default**: per vederli occorre premere F1 e cambiare l'impostazione.

# Lezione 06 CSS: Layouts, Responsive Design

## Unità di misura (sizing units)

### Panoramica

- Alcune proprietà CSS cambiano la **dimensione** degli elementi: `width`, `height`, `font-size`, `margin`, `padding`, `border`, …
- Per dimensionare gli elementi si usano lunghezze **assolute** o lunghezze **relative** (dipendono da altre dimensioni: font o viewport).

### Unità assolute

- Le lunghezze assolute si definiscono con un numero e una delle **unità di lunghezza** supportate.

```css
div {
    width: 2.54cm;  /* centimetri */
    height: 1in;    /* pollici */
    border: 2mm solid black; /* millimetri */
    font-size: 24px; /* pixel */
}
```

### Percentuali

- Le **percentuali** definiscono dimensioni relative all'elemento **padre**.
- Si usano con molte proprietà: `width`, `height`, `font-size`, `margin`, `padding`, …

```css
.b { width: 50%; height: 50%; } /* 50% del padre */
```

### Unità relative al font

- **em**: `1em` rappresenta la dimensione di font corrente (`1.5em` è il 50% in più); il nome deriva storicamente dalla larghezza della "M" maiuscola in tipografia.
- **rem**: `1rem` rappresenta il `font-size` calcolato all'elemento **radice** (default 16px).

```css
p { font-size: 1.5rem; }
span { font-size: 1.5em; }
```

### Unità relative al viewport

Definition

**Viewport**
Il **viewport** è la finestra del browser.

- **vw**: `1vw` rappresenta l'1% della **larghezza** del viewport corrente.
- **vh**: `1vh` rappresenta l'1% dell'**altezza** del viewport corrente.

```css
div { height: 50vh; }
.b { width: 50vw; font-size: 10vw; }
```

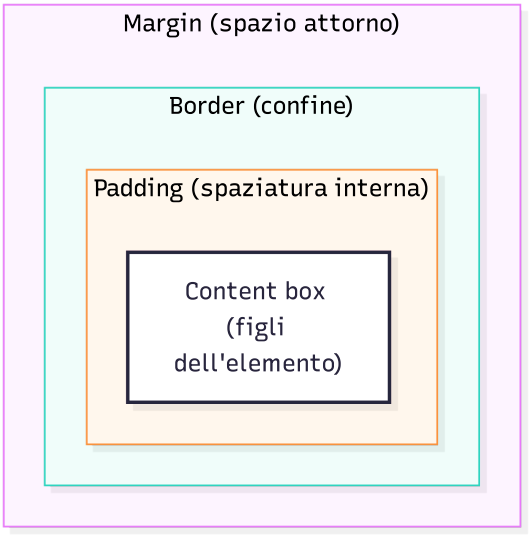
# Il box model

**Definition**

**Box model**

Ogni elemento HTML è un **box** composto da aree distinte.

- **Content box**: l'area dove vivono i figli dell'elemento.
- **Padding**: spaziatura interna, separa il content box dal bordo.
- **Border**: il confine dell'elemento.
- **Margin**: crea spazio attorno agli elementi.




- La dimensione di ogni area si definisce con dichiarazioni CSS.
- Comportamento e aspetto dei box sono determinati da: **layout mode**, **contenuto** e **proprietà del box model**.
- Se il contenuto non entra nel box, la parte in eccesso si chiama **overflow**.

# Layout

### Flow layout (normal flow)

Per default i box sono visualizzati con il **flow layout** (a.k.a. **normal flow**). Esistono due tipi di elementi:

- **inline** si dispongono nella direzione inline, come parole in una frase.
- **block** si dispongono uno dopo l'altro, ognuno su una nuova riga dedicata.

### La proprietà display

- `display: inline` posiziona il box accanto al precedente nella direzione inline; **ignora** `width` e `height`.
- `display: block` posiziona il box su una nuova riga dedicata; se non specificato, i block si espandono su tutta la dimensione inline.
- `display: inline-block` è come `inline`, ma **permette** di impostare `width` e `height`.
- `display: none` rimuove completamente l'elemento dalla visualizzazione.

```css
em { display: inline-block; width: 50px; height: 50px; }
```

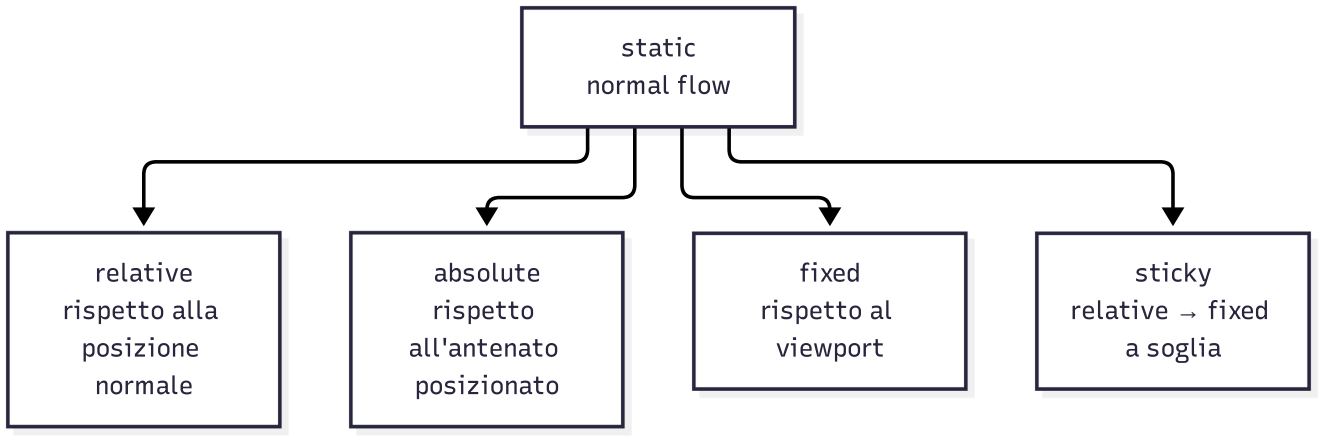
### Floats

- La proprietà `float` fa "galleggiare" un elemento nella direzione indicata, con i fratelli successivi che vi **scorrono attorno** (wrap).
- La proprietà `clear` impedisce a un fratello successivo di fare wrap attorno a un elemento flottante.

```css
img { float: left; margin-right: 1rem; }
p+p { clear: both; }
```

### Positioning

- La proprietà `position` cambia il comportamento dell'elemento nel normal flow.
- Il valore di default è **static**; gli altri valori sono `relative`, `absolute`, `fixed`, `sticky`.
- **relative**: l'elemento è posizionato rispetto alla sua posizione normale.
- **absolute**: l'elemento è posizionato rispetto all'**antenato posizionato più vicino** (nearest relative-positioned ancestor).
- **fixed**: l'elemento è posizionato rispetto al **viewport**.
- **sticky**: ibrido relative/fixed, relativo finché non supera una soglia (`top`, …), poi fixed finché non raggiunge il confine del proprio padre.



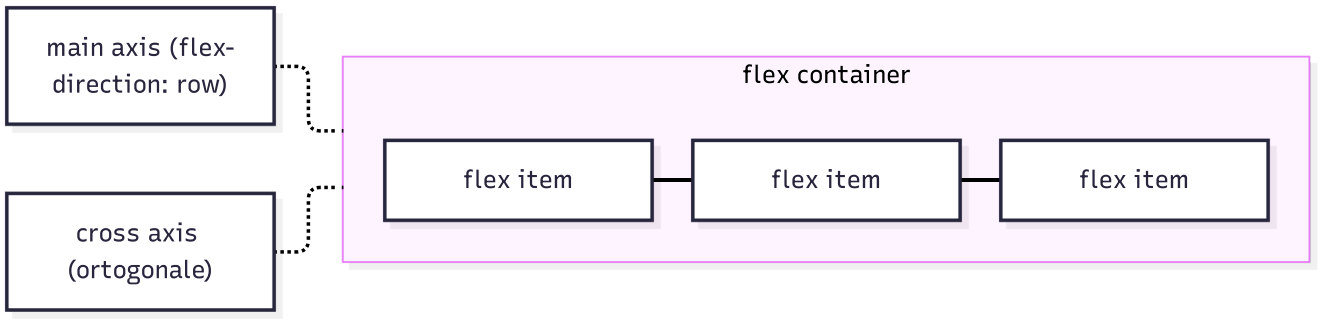

```css
nav { position: sticky; top: 0px; }
a  { position: fixed; bottom: 2rem; right: 2rem; }
```

## Layout moderni: Flexbox e Grid

- Oltre al normal flow, il CSS moderno ha due meccanismi di layout: **Flexbox** e **Grid**.
- **Flexbox** è pensato per layout **unidimensionali** (orizzontale o verticale).
- **Grid** è pensato per layout **bidimensionali**.

### Flexbox: flex container

- I flex container si dichiarano con `display: flex`; sono elementi block-level con figli **flex item**.
- Ogni container ha un **main axis** e un **cross axis** (ortogonale).
- Il main axis è impostato da `flex-direction` (default: `row`).




### Flexbox: flex items

I figli del container diventano **flex item** e si collocano lungo il main axis.

- **flex-grow**: fa espandere l'item in tutto lo spazio disponibile lungo il main axis.
- **flex-shrink**: controlla se l'item può ridurre la propria dimensione di base per stare nel container.

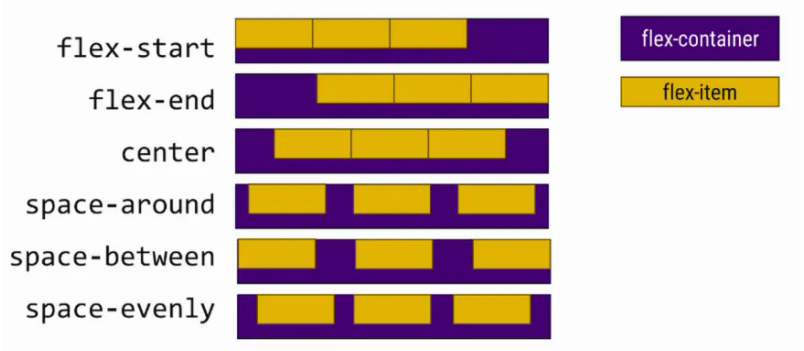
Gli overflow sono possibili e si controllano con `flex-wrap` sul container.

### Flexbox: gestione dello spazio libero

- **justify-content**: gestisce lo **spazio libero lungo il main axis**.
- **align-content**: gestisce lo **spazio libero lungo il cross axis**.
- Valori possibili includono: `flex-start`, `flex-end`, `center`, `space-between`, `space-around`.

CSS

```css
.container {
    display: flex;
    justify-content: space-around;
    align-items: center;
}
```




### Grid

Grid è un layout **bidimensionale** basato su **righe e colonne**, si dichiarano con `display: grid`; i figli diretti sono **grid item**. Il container definisce **numero e dimensione** di righe e colonne (`grid-template-columns`, `grid-template-rows`, `gap`). Per default gli item sono collocati lungo le righe; `grid-auto-flow: column` li colloca lungo le colonne.

CSS

```css
.container {
    display: grid;
    grid-template-columns: 1fr 100px;
    grid-template-rows: 20px 50px 100px;
    gap: 10px;
}
```

### L'unità fr

**fr** è un'unità relativa specializzata che funziona solo nei grid layout. Rappresenta una lunghezza flessibile corrispondente a una quota dello spazio disponibile, in modo simile alla proprietà `flex`.

- Esempio: `grid-template-columns: 1fr 1fr 1fr` definisce tre colonne con la stessa quota di spazio.

### Template areas

- Si possono assegnare nomi ad **aree** (insiemi di celle) con `grid-template-areas` e collocare item in un'area con `grid-area`.

```css
body {
    display: grid;
    grid-template-columns: 70vw 1fr;
    grid-template-areas:
      "header header"
      "navbar navbar"
      "main sidebar"
      "footer footer";
}
header { grid-area: header; }
aside  { grid-area: sidebar; }
```

## Media queries

Definition

**Media query**

Una media query applica stili CSS solo quando il **dispositivo** che visualizza il contenuto ha **caratteristiche specifiche**.

- Le media query si aprono con la keyword `@media`.

```css
@media print {
    body { color: black; background: transparent; }
}
```

### Tipi di output

- **print**: materiale paginato e documenti in anteprima di stampa.
- **screen**: dispositivi che visualizzano il documento su schermo.
- **all**: tutti i dispositivi di output.

### Media features

- Le **media features** verificano caratteristiche dello user-agent, del dispositivo di output o dell'ambiente.
- Sintassi completa: `@media media_type and (media_feature: value) and ....`
- Se `media_type` è omesso si usa `all`; le feature vanno racchiuse tra parentesi e sono opzionali.

```css
@media screen and (orientation: landscape) { /* ... */ }
@media (min-width: 768px) { /* ... */ }
```

## Responsive design

### Fixed-width layouts

- Progettare per una singola larghezza specifica (640, 800, 1024px) si chiama **fixed-width design**.
- Funziona bene solo alla larghezza considerata: schermi più piccoli richiedono scrolling orizzontale, schermi più grandi mostrano molto spazio sprecato.

### Liquid (fluid) layouts

- I **liquid layouts** usano le **percentuali** come larghezza delle colonne, senza flexbox, grid o media queries.
- Reggono su un intervallo più ampio di larghezze, ma degradano agli estremi: **stretched** su schermi molto larghi, **squashed** su schermi molto stretti.

### Siti separati (mobile)

- Con l'arrivo dei dispositivi mobili (schermi fino a 240px) un'opzione era un **sito separato** per mobile.
- Tipicamente basato su **user agent sniffing** per reindirizzare gli utenti mobili a una versione su sottodominio.

Warning

**Contro dei siti separati**

- Lo UA sniffing è inaffidabile (può mancare i mobile o reindirizzare i desktop).
- Ogni sito separato va mantenuto.
- La distinzione mobile/non-mobile oggi è sfumata.

### Adaptive layouts

- Con le media query (~2009), i designer passavano tra un **piccolo numero di design fixed-width** usando media query: è l'**adaptive design**.
- L'utente vede il layout più vicino alla dimensione del proprio browser; per le dimensioni intermedie il design non è ottimale.

CSS

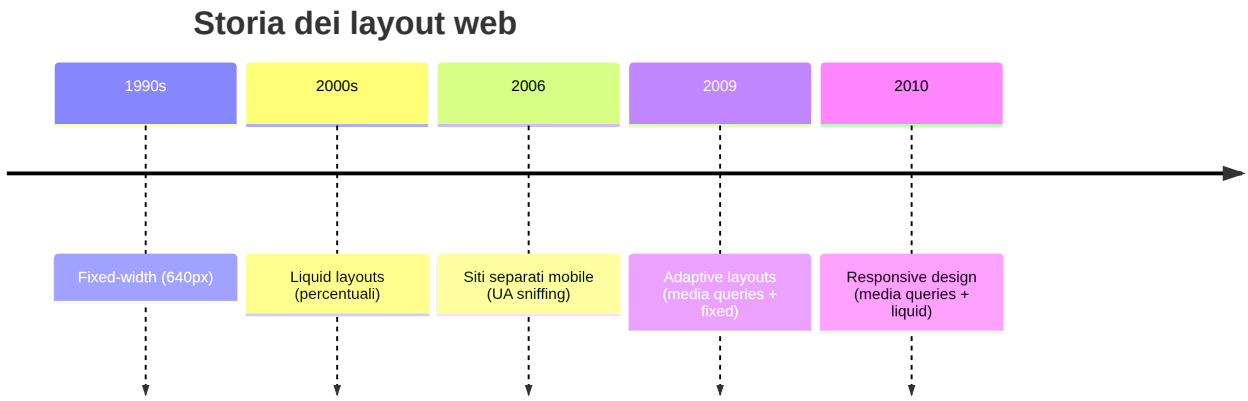
```css
@media (max-width: 800px) { main { width: 400px; } }
@media (min-width: 800px) { main { width: 500px; } }
```

### Responsive layouts

- **Adaptive** = media queries + fixed-width; **responsive** = media queries + liquid layouts (termine coniato da Ethan Marcotte, 2010).
- Il responsive design è caratterizzato da: **container fluidi**, **media fluidi**, **media queries**.
- Layout e immagini di un sito responsive devono apparire ben resi su qualsiasi dispositivo.

CSS

```css
body { display: flex; flex-wrap: wrap; }
@media (min-width: 768px) { main { width: 70%; } aside { width: 20%; } }
img { width: 20%; min-width: 100px; }
```




### Viewport meta tag

- I primi browser mobili assumevano che i siti fossero progettati per uno schermo di **980px**.
- Rendering in un **virtual viewport** di 980px, poi **scalatura** della pagina alla larghezza reale dello schermo.




- Il meccanismo aiuta i siti non ottimizzati per mobile, ma **non funziona** per i siti mobile-optimized: le media query che scattano a 640px non si attivano mai a 980px.
- Il **meta tag viewport** permette di controllare il meccanismo; le pagine mobile-optimized dovrebbero includere in `<head>`:

```HTML
<meta name="viewport" content="width=device-width, initial-scale=1">
```

- `width=device-width`: assumere che la larghezza di progettazione sia la larghezza del device.
- `initial-scale=1`: nessuna scalatura.

Capitolo 7

# Lezione 07: Modern, Adaptive CSS

Estensione dei concetti già visti (selettori, cascata, box model, normal flow, posizionamento, Flexbox, Grid, media query) tramite nuove funzionalità del CSS moderno.

## Pseudo-classi funzionali

Info

**Contesto**

I selettori semplici possono combinarsi in relazioni complesse, ma le liste di selettori ripetitive allungano i fogli di stile (meno efficienti da caricare) e rendono le modifiche più soggette a errori. Strumenti come **Sass** (pre/post-processore) si sono diffusi proprio per organizzare i fogli di stile; il CSS moderno ha incorporato direttamente nel linguaggio capacità analoghe.

Definition

**Pseudo-classi funzionali**

Pseudo-classi che accettano selettori come argomenti e consentono di scrivere selettori in modo più conciso.

- `:is()` raggruppa selettori alternativi.
- `:where()` crea default facilmente sovrascrivibili.
- `:not()` esclude gli elementi che corrispondono.
- `:has()` seleziona elementi in base a elementi correlati.

### `:is()` (matches-any)

- `:is()` corrisponde a un elemento se uno dei suoi selettori argomento corrisponde; è utile quando in un selettore complesso cambia solo una parte.

```css
:is(header, main, footer) a em { color: red; }
button:is(:hover, :focus, :active) { color: red; }
```

### `:where()`

- `:where()` seleziona come `:is()`, ma contribuisce **specificità zero** alla cascata, per sé e per tutti i suoi argomenti.
- La conseguenza è che gli stili risultanti sono volutamente facili da sovrascrivere (utile per regole di default).

### `:not()`

- `:not()` corrisponde agli elementi che non corrispondono a nessuno dei suoi argomenti.

### `:has()`

- `:has()` corrisponde a elementi in base ai loro discendenti o elementi correlati, anche fratelli successivi.

```css
ol:has(.active) { background-color: lightgray; }
li:has(+ .selected) { color: red; }
```

#### Specificità delle pseudo-classi funzionali

- Le pseudo-classi funzionali **non contano come pseudo-classi** nel calcolo della specificità (non entrano in B come `:focus` o `:hover`).

- `:where()` e il suo contenuto contribuiscono sempre **zero specificità**.
- Per `:is()`, `:not()` e `:has()` la specificità è quella dell'argomento più specifico.

# Custom properties (variabili CSS)

- I valori ripetuti nel CSS sono difficili da mantenere: se un valore cambia, va modificato in ogni occorrenza.

Definition

**Custom properties**

Entità definite dagli autori CSS che rappresentano valori specifici riutilizzabili; si definiscono con la sintassi delle proprietà personalizzate (o con l'at-rule `@property`) e si recuperano con `var()`.

```css
:root {
  --primary-color: #663399;
}
h1 { color: var(--primary-color); }
```

### Ereditarietà

- Le custom properties **partecipano alla cascata**, sono **normalmente ereditate** e possono essere **sovrascritte**.
- La pseudo-classe `:root` corrisponde alla radice dell'albero del documento e permette di definire custom property «globali»; un elemento antenato può poi ridefinire la stessa proprietà, e i discendenti ereditano il nuovo valore.

```css
:root { --accent: #cfa2fb; }
button { background: var(--accent); }
.warning { --accent: orangered; }
```

### Fallback

- **Fallback**: `var()` accetta un valore di riserva come secondo argomento, usato se la custom property non è definita.

```css
.notice { color: var(--notice-color, darkred); }
```

- I fallback sono utili quando un componente ammette **personalizzazione opzionale**.

### At-rule `@property`

- `@property` permette di personalizzare il comportamento di ereditarietà di una custom property, dichiarandone sintassi, ereditarietà e valore iniziale.

```css
@property --accent {
  syntax: "<color>";
  inherits: false;
  initial-value: teal;
}
```

# Valori calcolati

### `calc()`

- **`calc()`** combina valori e unità compatibili con gli operatori aritmetici `+`, `-`, `*`, `/`.

```css
.item { width: calc(100% - 20px); margin: auto; }
```

### `min()` e `max()`

- **`min()`** e **`max()`** selezionano rispettivamente il più piccolo e il più grande dei loro argomenti.
- Con `width: min(100px, calc(100% - 2rem))` l'elemento non supera mai 100px, si restringe quando lo spazio disponibile è minore e conserva almeno 1rem per lato.
- Con `font-size: max(1rem, 10vw)` la dimensione del font non scende mai sotto 1rem e può crescere su viewport più larghi.

### `clamp()`

- **`clamp()`** vincola un valore tra un minimo e un massimo: il valore cresce fluidamente ma non supera mai i limiti.

## Unità viewport su dispositivi mobili

- **`100vh`** rappresenta l'altezza del viewport; su desktop si basa sulla dimensione corrente della finestra, ma su mobile i controlli del browser (es. barra degli indirizzi) possono apparire e scomparire, complicando il calcolo.

Info

**Unità viewport moderne**

- **`svh`** (small viewport height): altezza del viewport calcolata con la barra di navigazione visibile.
- **`lvh`** (large viewport height): calcolata sull'altezza massima possibile del viewport (come `vh`).
- **`dvh`** (dynamic viewport height): rappresenta dinamicamente l'altezza corrente del viewport; con i controlli visualizzati corrisponde a `lvh` meno l'altezza dei controlli (`svh`), senza controlli coincide con `lvh`.

## Pattern di layout moderni

### `gap`

- La proprietà **`gap`** si usa sui container **flex** e **grid** per aggiungere spazio tra i figli.
- A differenza dei margini sui singoli figli, `gap` aggiunge spazio **solo tra gli elementi** (non ai bordi del container).

#### Shorthand `flex`

- Lo shorthand **`flex`** controlla quanto un item cresce (**`flex-grow`**), quanto si restringe (**`flex-shrink`**) e la sua dimensione base iniziale (**`flex-basis`**).

```css
.item { flex: 1 1 100px; } /* equivale a flex-grow:1; flex-shrink:1; flex-basis:100px */
```

## Container queries

- Le **media query** osservano il viewport, ma lo stesso componente può trovarsi in container di dimensioni diverse (area principale oppure sidebar/colonna di griglia più piccola): in un container piccolo il componente dovrebbe comportarsi come in un viewport piccolo, anche se il viewport è grande.

Definition

**Container queries**

Regole che permettono di stilare un componente in base alla dimensione di un antenato; richiedono la dichiarazione della proprietà `container-type` sull'antenato e l'uso dell'at-rule `@container` per la query.

```css
aside { container-type: inline-size; }
@container (max-width: 300px) {
  .card { background-color: lightgreen; }
  .card :is(h2, p) { font-size: 1em; }
}
```

## Candidati immagine responsive

- Il CSS determina quanto grande viene mostrata un'immagine, ma scaricare un'immagine ad alta risoluzione e scalarla su schermi piccoli è inefficiente.
- L'**HTML** può fornire più file per la stessa immagine: il browser seleziona il migliore candidato in base al dispositivo, scaricando solo la versione necessaria, tramite l'attributo **`srcset`** o l'elemento **`<picture>`**.

### `srcset`

- L'attributo **`src`** funge da fallback (quando `srcset` non è supportato); `srcset` specifica una lista di nomi di file con **descrittori**:
  - **descrittori di larghezza** (numero seguito da `w`): dimensione intrinseca dell'immagine;
  - **descrittori di densità di pixel** (numero seguito da `x`): densità di pixel del dispositivo rispetto alla densità standard.

```html
<img src="fu-tzu-400.jpg"
     srcset="fu-tzu-400.jpg 400w, fu-tzu-800.jpg 800w, fu-tzu-1200.jpg 1200w"
     alt="Portrait of the great master Fu-Tzu">
```

### `<picture>` e art direction

Definition

**Art direction**

Processo di selezione del media migliore per condizioni specifiche: non solo diverse risoluzioni della stessa immagine, ma anche contenuti diversi (es. un grafico con meno dettagli su mobile e completo su desktop).

```html
<picture>
  <source media="(max-width: 200px)" srcset="fu-tzu-400.png">
  <source media="(max-width: 400px)" srcset="fu-tzu-800.png">
  <img src="fu-tzu-1200.png" alt="Portrait of the great master Fu-Tzu">
</picture>
```

## Media query e preferenze utente

- Le media query possono testare più delle dimensioni del viewport: riflettono anche le **preferenze dell'utente**, normalmente originate dalle impostazioni del browser o del sistema operativo:
  - **schema di colori preferito**: `@media (prefers-color-scheme: dark)`;
  - **movimento ridotto**: `@media (prefers-reduced-motion: reduced)`;
  - **contrasto aumentato o ridotto**: `@media (prefers-contrast: more)`;
  - **trasparenza ridotta**: `@media (prefers-reduced-transparency: reduced)`.

```css
:root { --bg-color: white; --text-color: indigo; }
@media (prefers-color-scheme: dark) {
  :root { --bg-color: indigo; --text-color: white; }
}
```

## Query `@supports`

- `@supports` applica gli stili solo quando certe coppie proprietà/valore sono supportate; la regola base funge da **fallback** e il layout avanzato (es. griglia) è applicato solo se il browser lo supporta.

```css
.cards { display: block; }
@supports (display: grid) {
  .cards { display: grid; grid-template-columns: 1fr 1fr; }
}
```

## Nesting nativo del CSS

- Nel CSS moderno i selettori correlati possono essere **nidificati** all'interno di altre regole, evitando di ripetere il selettore padre.

```css
.card {
  background-color: lightblue;
  :is(h2, p) { font-size: 1.25em; }
}
```

### Selettore di nesting: `&`

- Il selettore `&` in un selettore nidificato si riferisce esplicitamente al selettore della regola padre.

```css
.card {
  background: white;
  &:hover { background: lavender; }
  & > h2 { color: rebeccapurple; }
}
```

Warning

**Prudenza con il nesting**

Mantenere il nesting poco profondo: un eccesso di nidificazione rende i selettori e il comportamento della cascata più difficili da capire.

# Parte V

# JavaScript

Capitolo 8

# Lezione 8 JavaScript

## Il linguaggio

Definition

**JavaScript**

**Linguaggio di programmazione** scripting di alto livello, debolmente tipato (loosely-typed). Introdotto dal browser Netscape per rendere dinamici i documenti HTML: i programmi (script) sono interpretati come puro testo dal **JS Engine**. Implementa la specifica **ECMAScript**.

- Oggi è usato anche fuori dalle pagine web: backend, applicazioni desktop e mobile, scripting generico.
- Supporta i paradigmi **imperativo**, **funzionale** e **orientato agli oggetti**, con feature dedicate alla programmazione **asincrona**.
- Le pagine HTML+CSS sono intrinsecamente **statiche**: JavaScript permette al contenuto di cambiare durante la navigazione.

### Includere JavaScript in una pagina

- **JavaScript interno**: codice dentro un elemento `<script>` in `<head>` o `<body>`.
- **JavaScript esterno**: file con estensione `.js` referenziato tramite `<script src="...">`.

HTML

```
<script> console.log("Hello World!"); </script>
<script src="script.js"></script>
```

### Strict mode

- Fino al 2009 (ECMAScript 5) il linguaggio è evoluto senza **breaking changes**; ES5 ne ha introdotti alcuni, disattivati di default per retro-compatibilità.
- Si abilitano con la direttiva `"use strict"` alla prima riga dello script. Nel corso si usa sempre la versione moderna «strict».

JavaScript

```
"use strict";
/* JavaScript code here */
```

## Sintassi di base

- Un programma è una **sequenza di statement**, composti da valori, operatori, espressioni, keyword e commenti.
- La sintassi è simile a Java e C ma più permissiva; gli statement sono delimitati da newline e/o **punto e virgola** (uso preferito come buona pratica).
- Con `typeof` si ispeziona il tipo di un valore: `undefined`, `number`, `string`, `boolean`.

### Operatori

| Operatore | Descrizione |
|---|---|
| `=` | Assegnamento |
| `+ - * /` | Somma, sottrazione, moltiplicazione, divisione |
| `**` | Elevamento a potenza (da ECMAScript 2016) |

| Operatore | Descrizione |
|---|---|
| `%` | Modulo (resto della divisione) |
| `++ --` | Incremento, decremento |

| Operatore | Descrizione |
|---|---|
| `==` | uguaglianza debole (con conversione di tipo) |
| `===` | uguaglianza stretta (stesso valore e stesso tipo) |
| `!=` | diverso (debole) |
| `!==` | diverso valore o diverso tipo |
| `> < >= <=` | confronti |
| `?` | operatore ternario |

Info

**Uguaglianza debole vs stretta**

- `==` converte gli operandi di tipo diverso prima di confrontarli: `"1" == 1` e `true == "1"` sono `true`.
- `===` non converte: se i tipi differiscono restituisce subito `false` (`"1" === 1` è `false`).

### Controllo di flusso

`if`, `if-else`, `for`, `while`, `do-while` e `switch` hanno stessa sintassi e semantica di Java; anche `continue` e `break` funzionano allo stesso modo.

## Variabili e scope

### Ciclo di vita

- **Dichiarazione**: il nome della variabile è legato allo scope corrente.
- **Inizializzazione**: alla variabile viene assegnato un valore (di default `undefined`).
- **Uso**: la variabile può essere referenziata.

Definition

**Temporal Dead Zone (TDZ)**

Intervallo tra la dichiarazione e l'inizializzazione in cui la variabile è **inaccessibile**: referenziarla solleva un `ReferenceError`.

### Dichiarazione

- `let`: variabili «standard», con **scope a livello di blocco** (il blocco `{} più vicino`).
- `const`: costanti con scope di blocco; non possono essere riassegnate ( tentativo → `TypeError`).
- Le variabili sono accessibili solo dopo l'esecuzione della riga in cui sono dichiarate; nello stesso blocco non si può ridichiarare lo stesso nome.

JavaScript

```
let bool;
bool = true;
if (bool) {
    let msg = "Hello"; // visibile solo dentro il blocco
    console.log(msg);
}
```

- Una variabile dichiarata in un blocco interno **oscura** (shadowing) l'omonima dello scope esterno.

### Hoisting

Definition

**Hoisting**

Comportamento predefinito per cui le dichiarazioni di variabili (e funzioni) sono implicitamente spostate all'inizio del loro scope. Con `let`/`const` è sollevata solo la **dichiarazione**, non l'inizializzazione, che avviene alla riga originaria: l'uso precedente cade nella TDZ.

JavaScript

```
{
    console.log(x); // ReferenceError: can't access 'x' before initialization
    let x = 1;
}
```

### `var` e dichiarazione implicita (pre-ECMAScript 6)

Warning

**var è deprecato**

- `var` ha scope di funzione/globale (nessuno scope di blocco) ed è hoistato **con inizializzazione a `undefined`**: usare la variabile prima della riga di dichiarazione stampa `undefined` invece di un errore, e la variabile resta visibile fuori dal blocco.
- **Dichiarazione implicita** (assegnamento senza keyword): crea una **variabile globale**, pratica error-prone, vietata in strict mode (`ReferenceError`).

JavaScript

```
if (true) {
    console.log(n); // undefined (non errore!)
    var n = 1;
}
console.log(n); // 1
```

## Scope

- **Scope globale**, **scope di funzione**, **scope di modulo**; le variabili `let`/`const` aggiungono lo **scope di blocco**.

## Funzioni

JavaScript

```
function greet(name, message = "Hello") {
    console.log(`${message} ${name}`);
}
```

- La **dichiarazione di funzione** vale nello scope in cui avviene; le dichiarazioni di funzione sono **hoistate** e quindi richiamabili prima della riga che le definisce.
- Se un argomento non è passato, il parametro è `undefined`; i parametri possono avere **valori di default**.
- I **template literals** (backtick) permettono l'interpolazione `${...}`.

### Scope interno e visibilità

- Una funzione crea il proprio scope: le variabili e funzioni dichiarate al suo interno non sono visibili all'esterno.
- Una funzione può accedere alle variabili degli scope esterni (**closure**), purché non mascherate nel proprio scope.

### Function expressions

- Le funzioni sono **valori**: possono essere assegnate a variabili (**function expression**), anche con sintassi **arrow function**.
- Si applicano le regole di hoisting delle variabili: la funzione non è usabile prima della riga di assegnazione (TDZ), a differenza delle dichiarazioni.

```JavaScript
let greet = function(name) { console.log(`Hello ${name}!`); };
let salute = (name) => { console.log(`Howdy ${name}!`); };
```

### Funzioni annidate e closure

Definition

**Funzione annidata**

Funzione creata dentro un'altra funzione. Può essere **restituita** e usata fuori dalla funzione originaria e, ovunque venga usata, conserva l'accesso al contesto esterno (**closure**) della funzione che l'ha creata.

```JavaScript
function getGreeter(message) {
    let sep = ",";
    return function(name) {
        console.log(`${message}${sep} ${name}!`);
    }
}
let helloGreeter = getGreeter("Hello"); // "cattura" message e sep
```

## Oggetti

Definition

**Oggetto**

Contenitore di dati **chiave: valore**. Si crea con la sintassi **object constructor** (`new Object()`) o, più comunemente, **object literal** (`{}`). Una **proprietà** ha una chiave (nome) prima di `:` e un valore a destra; le proprietà sono separate da virgole.

### Proprietà

- **Accesso con dot notation**: `pet.name`; proprietà inesistente → `undefined`.
- **Aggiunta/cancellazione**: `pet.nickname = "..."` aggiunge, `delete pet.age` rimuove.
- **Square brackets notation**: necessaria per chiavi con spazi o espressioni: `dog["is a good boy"]`, `dog["is the " + "best boy"] = true`.

### Riferimenti

- Una variabile assegnata a un oggetto memorizza un **riferimento**, non l'oggetto: due variabili che puntano allo stesso oggetto sono uguali sia con `==` che con `===` (condividono lo stesso valore di riferimento).

### Cloning

- **Copia per riferimento**: assegnare un oggetto a un'altra variabile non copia nulla; le modifiche sono visibili da entrambe le variabili.
- **Shallow clone**: copia proprietà per proprietà (es. con un ciclo `for...in`); gli oggetti annidati restano **condivisi** (modificare `copy.owner.name` modifica anche l'originale).
- **Deep clone**: richiede **ricorsione**, se il valore di una proprietà è un oggetto, si clona ricorsivamente.
- Alternative built-in: `Object.assign()` (shallow), `structuredClone()` (deep).

```JavaScript
function clone(object) {
    let copy = {};
    for (let key in object) {
        copy[key] = (typeof object[key] === "object") ? clone(object[key]) : object[key];
    }
    return copy;
}
```

### Metodi

- Una proprietà il cui valore è una funzione è un **metodo**; definibili in assegnamento, con sintassi classica `greet: function() {...}` o **shorthand** `greet() {...}`.

## this

**Definition**

**this**

Riferimento valutato **a run-time** in base al contesto: dentro un metodo, punta all'oggetto che contiene il metodo, permettendogli di accedere alle altre proprietà dello stesso oggetto.

```JavaScript
let john = { name: "John", greet() { console.log(`Hi, I'm ${this.name}!`); } };
```

- La stessa funzione assegnata come metodo a oggetti diversi vede ogni volta l'oggetto chiamante tramite `this`.
- **Le arrow function non hanno `this`**: il `this` al loro interno è preso dal contesto esterno, quindi non riflette l'oggetto chiamante.

### Costruttori

**Definition**

**Funzione costruttore**

Funzione regolare invocata con la keyword **`new`** per creare molti oggetti simili. Convenzioni: nome con iniziale maiuscola e invocazione solo con `new`.

Quando una funzione è eseguita con `new`: 1) si crea un nuovo oggetto vuoto assegnato a `this`; 2) viene eseguito il corpo, che tipicamente modifica `this`; 3) `this` è restituito.

```JavaScript
function Pet(name, species) {
    this.name = name;
    this.species = species;
}
let rick = new Pet("Richard", "Lizard");
```

### Optional chaining

- Accedere una proprietà di una proprietà `undefined` solleva un errore; l'operatore **`?.`** «cortocircuita»: se la parte sinistra è `undefined` restituisce subito `undefined` invece di un errore.
- Sostituisce il controllo esplicito `pet.owner ? pet.owner.name : undefined`.
- Varianti: **`?.()`** per chiamare una funzione che potrebbe non esistere (`mike.greet?.()` → `undefined`), **`?.[chiave]`** con la notazione a parentesi.

### Configurazione delle proprietà

- Ogni proprietà ha tre **flag**: **Writable** (il valore può cambiare), **Enumerable** (la proprietà compare nei cicli), **Configurable** (la proprietà può essere cancellata e i flag modificati).

- Con la creazione «tradizionale» tutti i flag sono `true`; con `Object.defineProperty(obj, key, { value, writable, enumerable, configurable })` si configurano esplicitamente (es. proprietà read-only → `TypeError` alla riassegnazione o alla cancellazione se non configurabile).

### Getter e setter

Info

**Proprietà dati vs accessorie**

- **Proprietà dati**: memorizzano un valore.
- **Proprietà accessorie**: funzioni eseguite quando la proprietà è letta (**getter**, keyword `get`) o assegnata (**setter**, keyword `set`).

JavaScript

```
let p = {
    first: "John", last: "Smith",
    get fullName() { return `${this.first} ${this.last}`; },
    set fullName(name) { [this.first, this.last] = name.split(" "); }
};
p.fullName; // si accede come proprietà, senza ()
```

- Getter e setter si usano **come proprietà** (senza `()`): dall'esterno non c'è differenza tra proprietà dati e accessorie.

# Lezione 9 JavaScript

## Prototipi ed ereditarietà

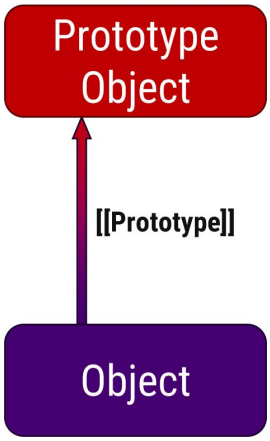
### Il prototype

Definition

**Ereditarietà prototipale**

Ogni oggetto possiede una proprietà nascosta **`[[Prototype]]`**, che è `null` oppure riferimento a un altro oggetto. Quando si accede a una proprietà mancante, JavaScript la cerca risalendo la **catena dei prototipi**.

- **`__proto__`** è un getter/setter per la proprietà `[[Prototype]]`; in JavaScript moderno si preferiscono `Object.getPrototypeOf()` e `Object.setPrototypeOf()`.
- I prototipi sono usati **solo in lettura**: le operazioni di **scrittura e cancellazione** agiscono direttamente sull'oggetto, non sul prototipo.

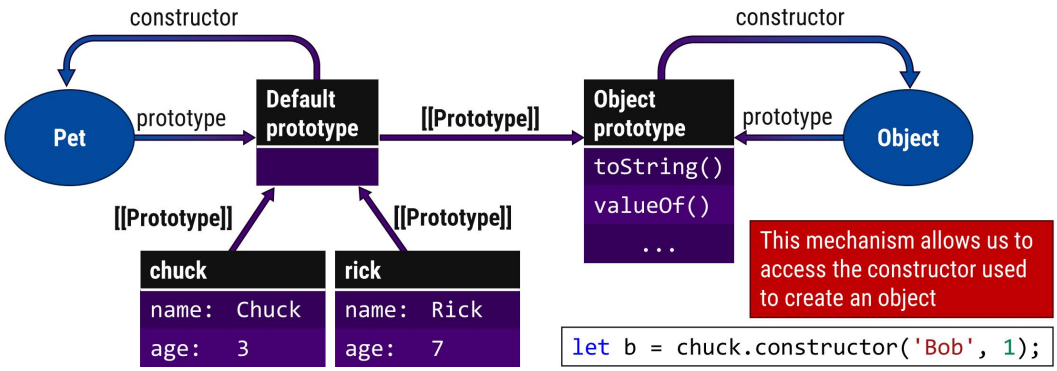



```JavaScript
let pet = { legs: 4 };
let cat = { name: "Garfield" };
Object.setPrototypeOf(cat, pet);
cat.legs;      // 4 (dalla catena)
snake.legs = 0; // snake ora ha una proprietà propria legs
```

L'API standard moderna raccomandata rispetto a **proto**, ma va usata con cautela perché fa crollare le ottimizzazioni del motore JS. ### Costruttori e prototype

- Con una **funzione costruttore** invocata con `new`, l'operatore usa `Constructor.prototype` per impostare il `[[Prototype]]` del nuovo oggetto.
- Se il costruttore non ha una proprietà `prototype`, viene usato un **prototipo di default**: un oggetto con la proprietà `constructor` che punta alla funzione costruttore stessa (`Pet.prototype = { constructor: Pet }`).
- Questo meccanismo consente di risalire al costruttore tramite `obj.constructor`.

```JavaScript
function Duck(name) { this.name = name; }
Duck.prototype = { eats: true };
let chuck = new Duck("Chuck");
chuck.eats; // true, via prototype
```




### Metodi: due alternative

Esistono due modi per dotare di metodi gli oggetti creati da un costruttore:

- Assegnare il metodo a `this` dentro il costruttore: ogni oggetto ha una **copia propria** del metodo.
- Assegnarlo a `Constructor.prototype`: il metodo è **condiviso** da tutti gli oggetti tramite la catena dei prototipi.

```JavaScript
Pet.prototype.describe = describe; // metodo condiviso sul prototype
```

## Strutture dati

### Array

- Un **array** memorizza una sequenza ordinata di valori, dichiarabile con la sintassi letterale `[...]` o con il costruttore `new Array(...)`.
- Gli array sono **indicizzati da 0**; l'accesso in lettura/scrittura usa la notazione a parentesi quadre.
- La proprietà **`length`** non è il conteggio dei valori: contiene l'indice massimo + 1, ed è **scrivibile** (assegnare un indice alto la aumenta, ridurla tronca l'array).
- Un array può contenere **tipi eterogenei** (stringhe, oggetti, numeri, funzioni).
- Gli array possono contenere altri array, formando **array multidimensionali** (es. matrici, accessibili con `matrix[i][j]`).

```JavaScript
let a = [1, 2, 3];
a[999] = 998;   // a.length diventa 1000
a.length = 3;   // tronca l'array
```

### Metodi di aggiunta/rimozione

- **`push()`**: aggiunge in coda; **`unshift()`**: aggiunge in testa.
- **`pop()`**: estrae dalla coda; **`shift()`**: estrae dalla testa.

### Iterazione

- **`for` classico** sugli indici e **`for..of`** percorrono tutti gli elementi (inclusi eventuali *buchi* come `undefined`).
- **`forEach()`** riceve una callback `(value, index, array)`.
- **`for..in`** è sconsigliato sugli array: è 10–100 volte più lento e può includere altre proprietà enumerabili.

### Destructuring

**Definition**

**Destructuring assignment**

Sintassi che permette di **spacchettare** un array in variabili: `let [x, y] = ...`. Gli elementi rimanenti sono scartati, oppure raccolti con l'operatore **rest** `...`.

```JavaScript
let [a, b, ...rest] = [1, 2, 3, 4, 5]; // rest = [3, 4, 5]
```

- Una sintassi simile vale per i **parametri di funzione**: `function greet(msg, ...names)` raccoglie gli argomenti extra nell'array `names`.

### Iterabili e Symbol.iterator

- Il **`for..of`** funziona con oggetti **iterabili**: array e stringhe lo sono.
- Per rendere iterabile un oggetto custom occorre implementare il metodo speciale **`Symbol.iterator`**.

Info

**Protocollo di iterazione**

- All'avvio del ciclo, `for..of` chiama `Symbol.iterator` una volta (in sua assenza lancia un `TypeError`).
- Da quel momento lavora solo con l'**iterator** restituito.
- Per ogni valore chiama `next()`, che restituisce `{ done: boolean, value: any }`: `done: true` termina il ciclo, altrimenti `value` è il prossimo valore.

```JavaScript
range[Symbol.iterator] = function() {
    return { current: this.min, max: this.max, step: this.step,
        next() {
            let n = { done: false, value: this.current };
            if (n.value > this.max) n.done = true;
            this.current += this.step;
            return n;
        } };
};
```

- È possibile usare l'iteratore **esplicitamente**: chiamare `next()` in un ciclo `while` fino a `done` replica il comportamento di `for..of`.

### Map

Definition

**Map**

Collezione di **coppie chiave-valore** a differenza degli oggetti, ammette **chiavi di qualsiasi tipo** (non solo stringhe).

| Metodo / proprietà | Descrizione |
|---|---|
| `new Map()` | crea la map |
| `map.set(key, value)` | memorizza il valore associato alla chiave |
| `map.get(key)` | restituisce il valore, o `undefined` se assente |
| `map.has(key)` | `true` se la chiave esiste |
| `map.delete(key)` | rimuove la coppia |
| `map.clear()` | svuota la map |
| `map.size` | numero di elementi |

- In un oggetto le chiavi sono **coercite a stringhe**: `{1: "Num"}` e `{"1": ...}` collidono; in una `Map` i tipi restano distinti.

### Iterazione sulle Map

- **`map.keys()`**, **`map.values()`** e **`map.entries()`** restituiscono iterabili su chiavi, valori e coppie.
- Iterare direttamente la map (`for (let [key, value] of map)`) equivale a usare `entries()`.

### Set

**Definition**

**Set**
Struttura per collezioni di **valori unici**, senza ripetizioni.

| Metodo / proprietà | Descrizione |
|---|---|
| `new Set([iterable])` | crea il Set, copiando i valori di un iterabile |
| `set.add(value)` | aggiunge il valore e restituisce il set |
| `set.delete(value)` | elimina il valore; `true` se esisteva |
| `set.has(value)` | `true` se il valore è presente |
| `set.clear()` | svuota il set |
| `set.size` | numero di elementi |

# Classi

### La sintassi class

- Una **classe** è un template per creare oggetti; `new Pet(...)` crea l'oggetto e invoca `constructor()` con gli argomenti, che imposta le proprietà sull'oggetto nuovo, poi lo restituisce.

```JavaScript
class Pet {
    constructor(name) { this.name = name; }
    greet() { /* ... */ }
}
let pet = new Pet("Chuck");
```

**Info**

**Cosa fa davvero class**
Una classe **non è una nuova entità del linguaggio**: `typeof Pet` è `"function"`.
1. Crea una funzione costruttore il cui codice è preso da `constructor()`.
2. Memorizza gli altri metodi in `Pet.prototype`.

- Come gli oggetti, le classi supportano **proprietà** di istanza e **getter/setter** (`get`), e possono definire metodi speciali come `[Symbol.iterator]`.

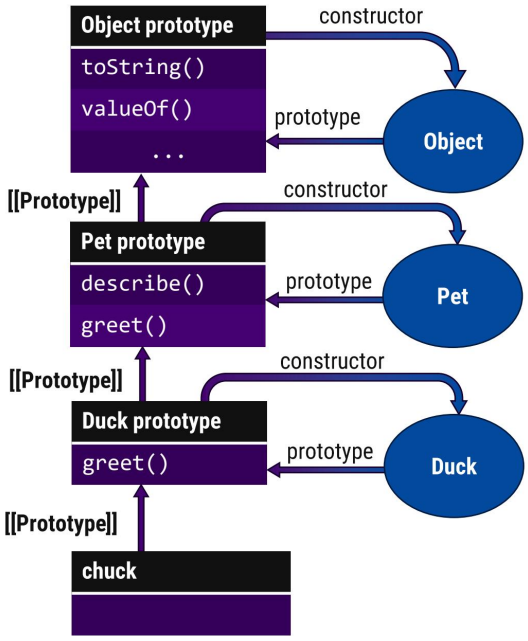
```JavaScript
class Duck {
    species = "Duck";           // proprietà
    get fullDescription() { return `${this.name} the ${this.species}`; }
}
```

### Ereditarietà: extends

- Una classe può ereditare da un'altra con la keyword **`extends`**; i metodi della classe base restano accessibili, quelli ridefiniti vengono **sovrascritti** (override).

```JavaScript
class Duck extends Pet {
    greet() { console.log("Quack!"); } // override
}
```

- Internamente, `extends` è implementato tramite la proprietà **`[[Prototype]]`**: il `<prototype>` di `Duck` punta a `Pet`, che a sua volta punta a `Object`.




# Gestione degli errori

### Errori e try/catch/finally

- A runtime possono verificarsi **errori** (errori del programmatore, input inattesi): normalmente interrompono lo script con un messaggio in console.
- Il costrutto **`try/catch/finally`** gestisce gli errori: `try` contiene il codice a rischio, `catch` riceve l'oggetto `error` (con `name` e `message`), `finally` è eseguito sempre.

```JavaScript
try {
    x = 1; // errore
} catch (error) {
    console.log(error.name, error.message);
} finally {
    console.log("done");
}
```

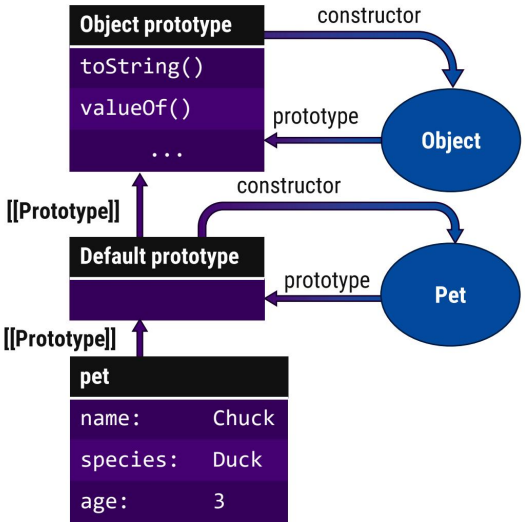
- In JavaScript c'è **al massimo un blocco `catch`** (`try/finally` da solo è ammesso); i diversi tipi di errore si distinguono dentro il blocco, ad esempio con `instanceof`.
- L'oggetto `error` è gestito tipicamente verificando `error instanceof ReferenceError`.

### instanceof

**Definition**

**Operatore instanceof**
Verifica se la proprietà `prototype` di un costruttore **compare nella catena dei prototipi** di un oggetto.

```JavaScript
pet instanceof Pet;    // true
pet instanceof Object; // true
```




### throw

- Gli errori **si propagano verso l'alto** nella call tree finché non sono catturati o si raggiunge la cima (in tal caso lo script si interrompe).
- Si possono lanciare errori con la keyword **`throw`**, tipicamente `throw new Error("...")`.
- È possibile definire **classi di errore custom** estendendo `Error` e sovrascrivendo `this.name` nel costruttore (chiamando `super(message)`).

```JavaScript
class MyError extends Error {
    constructor(message = "Whoops") { super(message); this.name = "MyError"; }
}
```

# Moduli

### Modularità

**Definition**

**Modulo**

Un modulo è semplicemente un **file JavaScript**; i moduli si caricano a vicenda tramite le direttive **`export`** ed **`import`**, per scambiare funzionalità.

- La modularità ** migliora la manutenibilità** e promuove la separazione delle *concerns*.
- Aiuta a prevenire **conflitti di naming**: identificatori uguali in moduli diversi non collidono.

**Warning**

**Perché gli script semplici non bastano**

Includere più `<script>` nella pagina mette tutto in un unico scope globale: due `let msg` con lo stesso nome provocano un `SyntaxError: redeclaration of let msg`. Il riuso del codice richiederebbe di rinominare gli identificatori, vanificando lo scopo.

### export e import

- **`export`** etichetta le variabili e le funzioni accessibili dall'esterno del modulo; tutto il resto rimane **privato**.
- **`import`** importa funzionalità specifiche da altri moduli (solo se esportate), specificandone l'URL; si può rinominare con `import { greet as greetPet }`.
- Nell'HTML basta includere lo script principale come modulo: `<script type="module" src="./script-module.js"></script>`.

JavaScript

```javascript
// pet-module.js
export function Pet(name, age) { this.name = name; this.age = age; }
let msg = "Hello"; // privato, non esportato

// script-module.js
import { Pet, greet as greetPet } from './pet-module.js';
```

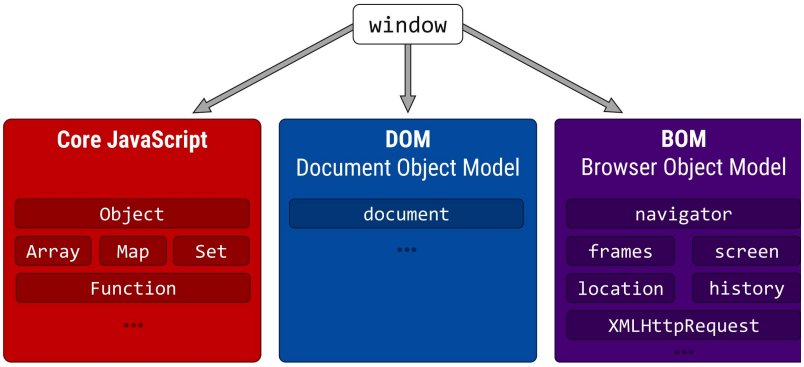
# Lezione 11 — JavaScript in a Browser Environment

## L'ambiente browser

### Host environment

JavaScript può girare in diversi **host environment** (ambienti host):

- **Browser web**: per cui JavaScript è stato originariamente creato.
- **Web server**: es. tramite il runtime **Node.js**.
- Ogni host environment fornisce i propri **oggetti** e **funzioni**, in aggiunta a quelli inclusi nel **core del linguaggio**.




### window

L'ambiente browser ha un oggetto «radice» chiamato `window`.

- È l'**oggetto globale** per il codice JavaScript.
- Rappresenta la finestra del browser e fornisce **metodi per controllarla**.
- Le funzioni e le variabili globali sono **proprietà di** `window`.

```JavaScript
let msg = "Hello";
function greet(name) { console.log(`${msg}, ${name}!`); }

greet("Web Technologies");     // richiamata come funzione globale
window.greet("window object"); // la stessa funzione, come proprietà di window
```

### DOM

**Definition**

**DOM (Document Object Model)**

Rappresenta il **contenuto del documento corrente**. L'oggetto `document` è il punto d'ingresso principale alla pagina web e fornisce modi per **accedere** e **manipolare** i contenuti.

### BOM

**Definition**

**BOM (Browser Object Model)**

Insieme di oggetti aggiuntivi forniti dal browser per lavorare **sul browser stesso**, non sui contenuti del documento.

Gli oggetti principali del BOM sono i seguenti:

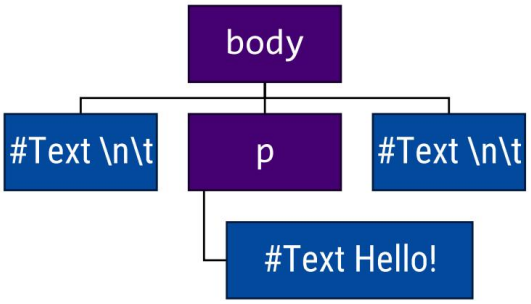
- `location`: contiene informazioni sull'URL del documento corrente (`location.href`).
- `screen`: informazioni sul display usato dall'utente (`screen.availWidth`).
- `navigator`: dettagli aggiuntivi su browser e piattaforma (`navigator.userAgent`).
- `history`: rappresenta la cronologia di navigazione (`history.back()`, come il pulsante «back» del browser).

# Il DOM

### Struttura ad albero

Un documento HTML può essere visto come un **albero** (con discendenti, figli, fratelli): il **DOM formalizza** questa intuizione.

- Ogni **tag HTML è un oggetto** nel DOM.
- I tag annidati sono **figli** del tag che li racchiude.
- Il contenuto testuale di un tag è un oggetto (`#text`) a sua volta **figlio del tag**.
- Gli **attributi** di un tag sono accessibili come **proprietà** dell'oggetto corrispondente.
- Il DOM reale include anche **spazi e a-capo** tra i tag come nodi `#text`; nelle rappresentazioni si omettono per brevità.




I **commenti HTML** non sono mostrati dal browser, ma vengono comunque analizzati e aggiunti al DOM come speciali **nodi commento**.

### Tipi di nodo

La specifica DOM definisce **12 tipi di nodo**; tipicamente si lavora con quattro tipi di nodo:

- **Document node**: è la radice e il punto d'ingresso del DOM.
- **Element node**: rappresenta un elemento HTML.
- **Text node**: rappresenta il contenuto testuale.
- **Comment node**: rappresenta un commento.

### Trovare elementi

L'oggetto `document` fornisce diversi **metodi per selezionare elementi** da un documento HTML.

- `querySelector` e `querySelectorAll` sono i metodi più versatili e ricevono in input un **selettore CSS**.
- `querySelector` restituisce solo il **primo elemento** corrispondente al selettore; `querySelectorAll` restituisce la **lista completa** delle corrispondenze.
- Tutti i metodi restituiscono `null` quando non trovano elementi.

```JavaScript
document.getElementById("a");            // elemento per attributo id
document.getElementsByClassName("foo");  // HTMLCollection
document.getElementsByName("c");        // NodeList
document.querySelector("ul > li");       // primo <li> che corrisponde
document.querySelectorAll("ul > li");    // tutti i <li> che corrispondono
```

Definition

**HTMLCollection («live collection»)**

Tutti i metodi della famiglia `getElementsBy*` restituiscono una **HTMLCollection**: una collezione aggiornata automaticamente, che riflette sempre lo stato corrente del documento.

```JavaScript
let links = document.getElementsByTagName("a"); // HTMLCollection (1)
// ...il documento viene esteso con altri <a>...
console.log(links); // HTMLCollection (3): la collezione si è aggiornata da sola
```

Invocati su `document`, i metodi di ricerca cercano nell'**intero documento**; se gli elementi cercati sono discendenti di un altro elemento, è più efficiente invocarli **sull'elemento antenato**.

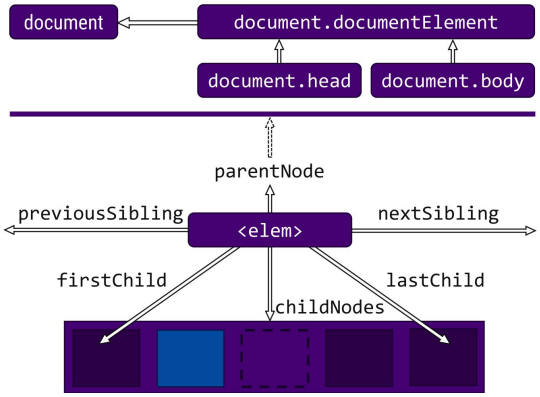
```JavaScript
let sidebar = document.querySelector(".sidebar");
let links = sidebar.querySelectorAll("a"); // cerca solo dentro sidebar
```

| Metodo | Cerca per… | Chiamabile sugli elementi? | Restituisce live collection? |
|---|---|---|---|
| `querySelector` | selettore CSS | ✓ | – |
| `querySelectorAll` | selettore CSS | ✓ | – |
| `getElementById` | attributo `id` | – | – |
| `getElementsByName` | attributo `name` | – | ✓ |
| `getElementsByTagName` | nome del tag opp * | ✓ | ✓ |
| `getElementsByClassName` | attributo `class` | ✓ | ✓ |

### Navigare nel DOM

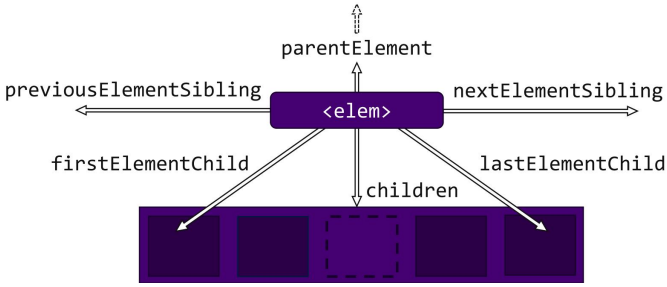
Gli elementi più in alto (`html`, `head`, `body`) sono disponibili anche come **proprietà di `document`**; inoltre, i nodi del DOM sono collegati tra loro da riferimenti di navigazione.

- I nodi del DOM contengono **riferimenti a padre, fratelli e figli**: `parentNode`, `previousSibling` / `nextSibling`, `firstChild` / `lastChild`, `childNodes`.
- Questi riferimenti sono **di sola lettura**.




### Navigazione solo elementi

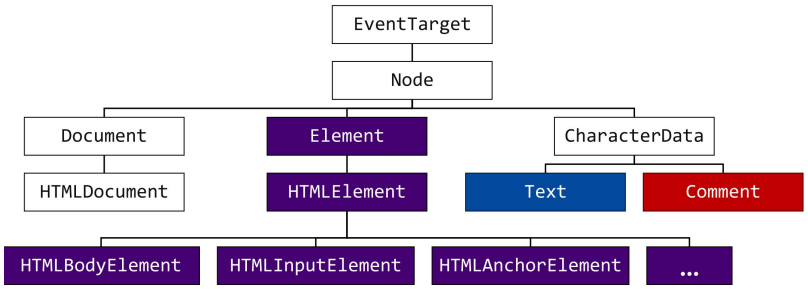
Per ignorare i nodi di testo e commento si usano le proprietà che considerano **solo i DOM element**: `parentElement`, `children`, `firstElementChild` / `lastElementChild`, `previousElementSibling` / `nextElementSibling`.




```HTML
<main><h1>Hello DOM!</h1><!-- comment --><p>DOM is <em>FUN</em>!</p></main>
<script>
  document.querySelector("main").childNodes; // [#text, h1, #text, #comment, #text, p, #text]
  document.querySelector("main").children;   // HTMLCollection { h1, p }
</script>
```

### Gerarchia delle classi dei nodi

I nodi del DOM appartengono a una **gerarchia di classi**: da `EventTarget` deriva `Node`, da cui derivano `Document`, `Element` e `CharacterData` (con `Text` e `Comment`); da `HTMLElement` derivano le classi specifiche di ogni tag (`HTMLInputElement`, `HTMLAnchorElement`, …).




### Proprietà dei nodi

#### `nodeName` / `tagName`

I nodi del DOM hanno alcune proprietà utili; le prime riguardano il tipo di nodo:

- **`nodeName`** e **`tagName`** danno informazioni sul **tipo di nodo** e sono di **sola lettura**.
- Per gli elementi valgono il nome del tag (es. `"H1"`); per i nodi di testo e commento valgono `#text` e `#comment`, mentre `tagName` è `undefined` (non sono elementi).

```JavaScript
let body = document.body;
body.firstChild.nodeName;         // "#text"
body.firstChild.tagName;          // undefined (non è un elemento)
body.childNodes[3].nodeName;      // "#comment"
body.firstElementChild.tagName;   // "H1"
```

### Attributi come proprietà

Gli attributi **standard** vengono analizzati e resi disponibili come **proprietà** dell'elemento corrispondente; gli attributi non standard non lo sono (per quelli si usa `getAttribute`).

```JavaScript
// <input id="x" class="inp ok" name="field" custom="y">
let i = document.body.firstElementChild;
i.id;                     // "x", attributo standard
i.classList;               // DOMTokenList ["inp", "ok"]
i.custom;                  // undefined, non è un attributo standard
i.getAttribute("custom");  // "y"
```

Queste proprietà sono anche **scrivibili**:

```JavaScript
i.id = "w";
i.classList.remove("ok"); i.classList.add("err");
i.setAttribute("custom", "k");
i.removeAttribute("name");
// → <input id="w" class="inp err" custom="k">
```

#### `innerHTML` / `outerHTML`

Un'altra coppia di proprietà riguarda il codice HTML dell'elemento:

- **`innerHTML`** restituisce il codice HTML **contenuto** in un elemento; **`outerHTML`** il codice HTML dell'**intero elemento**.
- Sono entrambi **scrivibili**: assegnare loro un valore cambia la pagina, `innerHTML` cambia il contenuto, `outerHTML` sostituisce l'elemento.

```JavaScript
let h1 = document.querySelector("h1"); // <h1 id="h">Hello <em>DOM!</em></h1>
h1.innerHTML; // "Hello <em>DOM!</em>"
h1.outerHTML; // "<h1 id=\"h\">Hello <em>DOM!</em></h1>"

```

```
5  h1.innerHTML = "DOM: <em>Wow!</em>";      // nuovo contenuto dell'elemento
6  h1.outerHTML = `<p>${h1.innerHTML}</p>`;  // l'<h1> diventa un <p>, stesso contenuto
```

# Manipolare il DOM

Non ci si limita a modificare gli elementi esistenti (es. cambiandone gli attributi): la struttura stessa del DOM può cambiare.

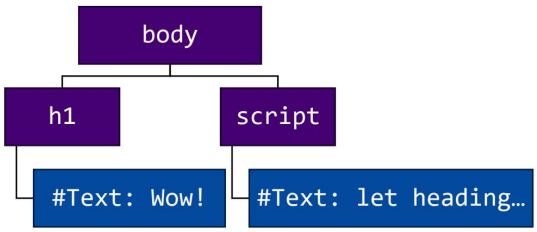
- Si possono **creare** nuovi elementi e **rimuovere** quelli esistenti, cambiando la **struttura del DOM**.
- L'aggiornamento dinamico del DOM è la chiave per pagine web reattive, «vive».

### Creare nuovi elementi

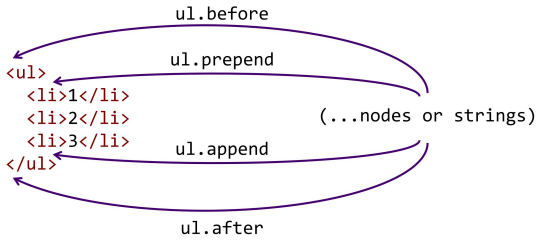
`document.createElement(tag)` crea un nuovo elemento, mentre `document.createTextNode(value)` crea un nuovo nodo `#text`; i nodi appena creati **non sono ancora nel DOM** e vanno inseriti con i metodi di inserimento.

JavaScript

```
1  let heading = document.createElement("h1");  // nuovo <h1>, non ancora nel DOM
2  let title = document.createTextNode("Wow!"); // nuovo #text, non ancora nel DOM
3  heading.append(title);          // #text come figlio dell'<h1>
4  document.body.prepend(heading); // <h1> (con il suo figlio) in cima al <body>
```




### Metodi di inserimento




| Metodo | Descrizione |
|---|---|
| `document.createElement(tag)` | crea un elemento con il tag indicato |
| `document.createTextNode(value)` | crea un nodo di testo (raramente usato) |
| `elem.cloneNode(deep)` | clona l'elemento; se `deep == true` con tutti i discendenti |
| `node.append(...nodi o stringhe)` | inserisce dentro il nodo, alla fine |
| `node.prepend(...nodi o stringhe)` | inserisce dentro il nodo, all'inizio |
| `node.before(...nodi o stringhe)` | inserisce subito prima del nodo |
| `node.after(...nodi o stringhe)` | inserisce subito dopo il nodo |
| `node.replaceWith(...nodi o stringhe)` | sostituisce il nodo |
| `node.remove()` | rimuove il nodo |

### Interagire con l'utente

`alert`, `prompt` e `confirm` sono funzioni del browser utili per interagire con l'utente.

- `alert(message)` mostra un messaggio all'utente.
- `confirm(question)` chiede una conferma e restituisce `true`/`false`.
- `prompt(question)` chiede un input e restituisce la stringa inserita dall'utente.

```JavaScript
alert("This page will ask for your input");
if (confirm("Do you want to continue?") === true) {
  let name = prompt("State your name:");
} else {
  console.log("user cancelled");
}
```

# Eventi

### Definizione e classificazione

Definition

**Evento**

Un **evento** è un segnale che indica che è successo qualcosa.

Gli eventi possono essere legati a diverse aree:

- **Comportamento dell'utente**: click, pressioni di tasti sulla tastiera, movimenti del mouse.
- **Form**: invio (submission), focus (o perdita di focus) su un campo di input.
- **Eventi del documento**: documento o elementi caricati, richieste di rete completate.

| Categoria | Evento | Significato |
|---|---|---|
| Mouse | `click` | l'utente clicca (tap su dispositivi touchscreen) su un elemento |
| | `contextMenu` | clic destro del mouse su un elemento |
| | `mouseover` / `mouseout` | il cursore entra in / esce da un elemento |
| | `mousedown` / `mouseup` | il pulsante principale del mouse è premuto / rilasciato su un elemento |
| | `mousemove` | il cursore del mouse si muove |
| Keyboard | `keydown` / `keyup` | un tasto della tastiera è premuto / rilasciato |
| Forms | `submit` | l'utente invia un form |
| | `focus` | l'utente dà focus a un elemento (es. `<input>`) |
| Document | `DOMContentLoaded` | l'HTML è stato analizzato, il DOM è completamente costruito |
| Window | `load` | la pagina è stata completamente caricata e renderizzata |

### Handler

Per reagire agli eventi si definiscono **handler**: funzioni eseguite quando un evento scatta.

- Il modo più semplice per definirli sono gli **attributi HTML** `on<evento>`, il cui valore è il codice JavaScript da eseguire.

```HTML
<input type="button" value="Click me" onclick="handleClick();">
```

Gli handler si possono anche definire **dinamicamente** via JavaScript, in due modi:

- Assegnando la **proprietà** `on<evento>` dell'elemento: **sovrascrive** l'handler definito tramite attributo HTML.
- Con il metodo `addEventListener`: invece **aggiunge** un nuovo handler.

```JavaScript
input.onclick = () => console.log("Tap");     // sovrascrive l'handler da attributo
input.addEventListener("click", handleClick); // aggiunge una nuova funzione handler
```

### `this` negli handler

Dentro un event handler, `this` vale l'elemento a cui l'handler invocato è **assegnato**.

```JavaScript
function handleClick(){ console.log(this.outerHTML); }
// assegnato all'<input>: stampa l'<input>; assegnato al <p>: stampa il <p>
```

### L'oggetto event

Tutti i dettagli dell'evento (es.: quale tasto è stato premuto? a quali coordinate è avvenuto il click?) sono forniti da un **oggetto event**, passato agli handler. Ogni tipo di evento ha le sue **proprietà**: es. gli eventi legati alla tastiera hanno la proprietà `key` con il tasto premuto.

```JavaScript
function handler(event){ console.log(`You pressed ${event.key}`); }
document.querySelector("input").addEventListener("keyup", handler);
```

### Event bubbling

Definition

**Event bubbling**

Quando un evento scatta su un elemento, prima vengono eseguiti tutti gli handler registrati **su quell'elemento**, poi quelli del **padre**, e così via risalendo tutti gli antenati fino al **nodo radice** del DOM.

Il meccanismo ha due caratteristiche da tenere a mente:

- Si applica alla **maggior parte** degli eventi, ma non a tutti: gli eventi `focus` non fanno bubbling.
- Un evento in bubbling si propaga fino all'oggetto globale `window`.

```HTML
<section onclick="console.log('section');">
  <article onclick="console.log('article');">
    <p onclick="console.log('p');">Bubbling</p>
  </article>
</section>
<!-- output in console dopo un click su <p>: p, article, section, document -->
```

### Event delegation

Info

**Event delegation**

Se molti elementi possono generare eventi simili, invece di assegnare un handler a ciascuno se ne usa **uno solo sul loro antenato comune**: gli elementi **delegano** la gestione dell'evento a un antenato.

- La proprietà `target` dell'oggetto event permette di determinare **su quale elemento** l'evento è stato generato.

```HTML
<table onclick="handler(event);"><!-- ...righe con celle <td>... --></table>
<script>
  function handler(event){
    if (event.target.nodeName === "TD") {  // gestisce solo le celle
```

```
      let oldValue = parseInt(event.target.innerHTML);
      event.target.innerHTML = ++oldValue;
    }
  }
</script>
```

### Fermare il bubbling

- Un handler può decidere che l'evento è stato **completamente processato** e non va propagato oltre: il bubbling si ferma invocando `stopPropagation()` sull'oggetto event.

Warning

**Regola pratica**

Non fermare la propagazione senza un buon motivo: in futuro potrebbe servire catturare l'evento su un antenato.

JavaScript

```
function handler(event){
  console.log(this.tagName);
  event.stopPropagation();
}
```

### Generare eventi (dispatching)

- Da JavaScript è possibile **generare eventi**: con il costruttore `new Event(...)` e il metodo `dispatchEvent`.

Info

**isTrusted**

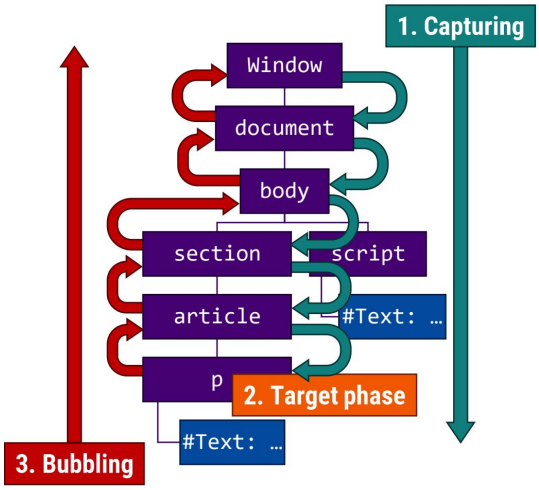
La proprietà `isTrusted` dell'evento vale `false` per gli eventi «artificiali» generati via JavaScript.

JavaScript

```
let click = new Event("click");
input.dispatchEvent(click); // isTrusted === false per gli eventi generati via JS
```

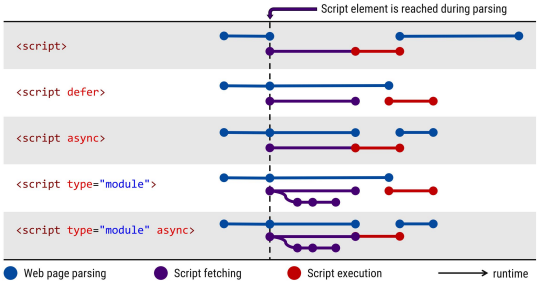
### Capturing

- Esiste un'altra fase di elaborazione dell'evento, il **capturing**, che procede «verso il basso» fino all'elemento target.
- L'elaborazione di un evento ha tecnicamente **tre fasi**: 1. capturing, 2. target, 3. bubbling.
- I listener nella fase di capturing si registrano con `elem.addEventListener(..., { capture: true });`.



# Ordine di esecuzione degli script

- Man mano che gli script diventano complessi, interagiscono con il DOM e generano dinamicamente handler, serve conoscere **come viene eseguito il codice JavaScript** al caricamento di una pagina.
- **Default**: gli script vengono scaricati ed eseguiti **nell'ordine in cui compaiono**, mentre il parsing della pagina è **in pausa**.
- `defer`: gli script esterni vengono scaricati **in parallelo** ed eseguiti **dopo** che la pagina ha finito di essere analizzata.
- `async`: gli script esterni vengono scaricati in parallelo ed eseguiti **appena scaricati**.
- Gli script **module** sono `defer` di default e possono essere dichiarati `async`.




- L'ordine di esecuzione **conta**: uno script nel `<head>` senza attributi viene eseguito prima che il `<body>` esista, quindi `document.body` è `null` (TypeError).

| Script nel `<head>` che accede a un elemento del `<body>` | Esito |
|---|---|
| `<script src="load.js">` | eseguito subito: `document.body` è `null` → TypeError |
| `<script defer src="load.js">` | eseguito a parsing concluso: il DOM è pronto ✓ |
| `<script async src="load.js">` | non prevedibile: dipende dal tempo di scaricamento |
| handler di `DOMContentLoaded` | eseguito quando il DOM è completamente costruito ✓ |

JavaScript

```
document.addEventListener("DOMContentLoaded", () => {
  let h1 = document.body.querySelector("h1"); // ora il body esiste
  console.log(h1.innerHTML);
});
```

Capitolo 11

# Lezione 11 JavaScript asincronismo e richieste di rete

## Storage dei dati nel browser

I browser moderni forniscono diverse API, accessibili via JavaScript, per memorizzare e recuperare dati dell'applicazione: **cookie**, **Web Storage** (`localStorage` e `sessionStorage`) e **IndexedDB**.

### Cookie

**Definition**

**Cookie**

**Piccole stringhe di dati** appartenenti al protocollo HTTP, usate per «superarne» l'assenza di stato (*statelessness*). Vengono tipicamente impostati in una risposta HTTP tramite l'intestazione `Set-Cookie`; il browser li memorizza e li invia in tutte le richieste successive allo stesso dominio, tramite l'intestazione `Cookie` della richiesta.

I cookie del sito corrente sono accessibili da JavaScript tramite la proprietà `document.cookie`.

- Il valore di `document.cookie` è una stringa di coppie `name=value` delimitate da `;`.
- Ogni coppia è un cookie separato.

```javascript
document.cookie = `name=${name}; max-age=10`; // scade dopo 10 secondi
```

### localStorage

Gestire i cookie via JavaScript non è molto immediato, soprattutto con più cookie: occorre manipolare stringhe, separarle, usare espressioni regolari. L'oggetto `localStorage` fornisce modi semplici per memorizzare coppie chiave/valore nel browser.

- I dati non vengono inviati con ogni richiesta, quindi si possono memorizzare molti più dati.
- È abbastanza simile a una `Map` e può memorizzare solo valori stringa.
- Esiste un oggetto `localStorage` diverso per ogni combinazione di dominio/protocollo/porta.

### sessionStorage

L'oggetto `sessionStorage` fornisce le stesse funzionalità di `localStorage`, ma è usato molto più raramente.

- A differenza di `localStorage`, i cui dati sopravvivono anche a un riavvio completo del browser, `sessionStorage` esiste solo nella scheda corrente del browser.
- I dati sopravvivono al ricaricamento della pagina, ma non alla chiusura e riapertura della scheda.

### IndexedDB

**Definition**

**IndexedDB**

**Database integrato nel browser**, che supporta diversi tipi di chiave, transazioni e query per intervalli di chiavi (*key-range queries*), e può memorizzare ancora più dati di `localStorage`.

Eccessivamente potente per le tradizionali applicazioni client-server: è pensata soprattutto per le applicazioni offline.

## Sincronismo e asincronismo

### Codice sincrono

Tipicamente il codice JavaScript viene eseguito in modo **sincrono**: ogni istruzione attende il completamento delle precedenti.

- È un modo intuitivo di programmare, ma ha alcuni svantaggi.
- Alcune istruzioni possono richiedere tempo per completarsi: attesa di input dall'utente, attesa del completamento di una richiesta di rete, attesa del termine di un calcolo complesso.
- Le istruzioni successive, anche se importanti, possono rimanere bloccate in attesa: l'**asincronismo** aiuta ad affrontare questi problemi.

### Codice asincrono

L'esecuzione di JavaScript è ordinariamente **single-threaded**: è il browser a fornire le capacità asincrone, come i timer e le richieste di rete.

JavaScript esegue le callback quando può, una volta completate le operazioni pertinenti.

JavaScript

```
1 console.log("A");
2 setTimeout(() => { console.log("B"); }, 2000); // dopo 2000 ms
3 console.log("C"); // l'output è "A", "C" e infine "B"
```

### Callback

Definition

**Callback**

**Funzioni passate come argomenti** ad altre funzioni, con l'aspettativa che vengano invocate al momento appropriato. I gestori di eventi (*event handler*) sono infatti callback.

### Limiti delle callback

Quando si gestiscono molte callback annidate dentro altre callback, il codice diventa molto disordinato molto rapidamente, e la gestione degli errori lo complica ulteriormente.

JavaScript

```
1 firstOperation(result, (r1) => {
2   secondOperation(r1, (r2) => {
3     thirdOperation(r2, (r3) => { console.log(r3); });
4   });
5 });
```

- Per questo si parla di **Callback Hell** o *Pyramid of Doom*.
- Il vecchio JavaScript gestiva l'asincronismo con le callback; in quello moderno si usa tipicamente un nuovo costrutto: la **Promise**.

## Promise

### Produttore e consumatore

In programmazione capita spesso che esistano un codice **produttore**, che fa qualcosa che richiede tempo (attende input dell'utente, scarica dati dalla rete, esegue calcoli complessi), e un codice **consumatore**, che ha bisogno dei risultati del produttore.

Definition

**Promise**

**Costrutto del JavaScript moderno** che serve a collegare produttori e consumatori.




### Creazione di una Promise

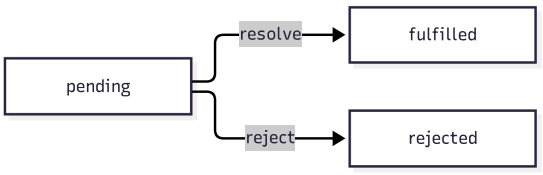
Un oggetto `Promise` si crea con l'apposito costruttore, passando come argomento l'**executor**: la funzione viene invocata immediatamente, appena la Promise viene creata.

- `resolve` e `reject` sono callback fornite da JavaScript.
- Quando il codice dell'executor termina, deve chiamare una delle due.
- `resolve(value)`: se l'operazione è completata con successo, con risultato `value`.
- `reject(error)`: se si è verificato un errore, con l'oggetto errore `error`.

```JavaScript
let promise = new Promise(function(resolve, reject) {
  // executor: qui il codice produttore
});
```

### Ciclo di vita

Una Promise nasce nello stato **pending**; quando viene stabilita (*settled*) diventa **fulfilled** (mantenuta) oppure **rejected** (rifiutata).




```JavaScript
console.log(promise); // Promise { <state>: "pending" }
```

### then

Il codice consumatore riceve una Promise che certi dati saranno resi disponibili in un certo momento del futuro, e registra le azioni da eseguire tramite metodi specifici offerti dall'oggetto Promise; il più importante è `.then()`.

- `.then()` prende in input due callback: la prima è invocata quando la Promise è mantenuta, la seconda quando è rifiutata.
- È anche possibile registrare solo la funzione da eseguire in caso di Promise mantenuta.

```JavaScript
promise.then(
  (result) => { /* Promise mantenuta */ },
  (error) => { /* Promise rifiutata */ }
);
```

### catch

Quando interessano solo gli scenari di errore, si può passare `null` come primo argomento di `.then()`, oppure usare il metodo equivalente `.catch()`.

```javascript
promise.catch((error) => { /* Promise rifiutata */ });
```

### finally

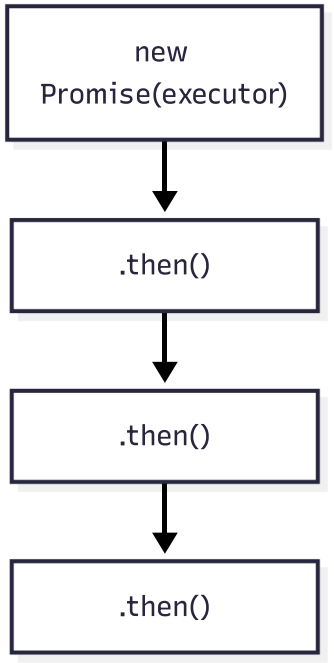
Come i blocchi `try/catch`, anche le Promise offrono un metodo `.finally()`, che prende una callback senza input.

- È invocato appena la Promise si stabilisce, sia con `resolve` sia con `reject`.
- È pensato per eseguire pulizie (*clean-up*) e altre operazioni da svolgere a prescindere dallo stato della Promise.

### Chaining

Quando serve eseguire una sequenza di passaggi asincroni si usa il **concatenamento** (*chaining*) delle Promise: la Promise originale viene risolta una sola volta, ma ogni invocazione di `.then()` restituisce in realtà una nuova Promise, che rappresenta il completamento del gestore stesso.

- Quando un gestore restituisce un valore, questo diventa il risultato della nuova Promise.
- Un `throw` all'interno di un gestore equivale a una `reject()` implicita.




```javascript
promise
  .then((r) => { console.log(r); return r * 2; }) // 1
  .then((r) => { console.log(r); return r * 2; }) // 2
  .then((r) => { console.log(r); return r * 2; }); // 4
```

Error

**Concatenamento errato**

Chiamare più volte `.then()` sulla stessa Promise non è concatenamento: ogni gestore riceve il risultato della Promise originale, non il valore restituito dal gestore precedente.

```javascript
// con risultato originale 1: stampa 1, 1, 1
promise.then((r) => { console.log(r); return r * 2; });
promise.then((r) => { console.log(r); return r * 2; });
```

### Metodi statici dell'API Promise

L'oggetto `Promise` espone anche metodi statici per lavorare con più Promise contemporaneamente.

- **`Promise.all(array)`**: restituisce una nuova Promise che rappresenta il completamento di tutte le Promise di ingresso; il risultato è un array in cui ogni Promise di ingresso contribuisce un elemento; se una sola Promise viene rifiutata, `Promise.all` si rifiuta immediatamente.

- **`Promise.allSettled(array)`**: aspetta che tutte le Promise di ingresso si stabilizzano, ma restituisce come risultato un array di oggetti che rappresentano lo stato di ciascuna (`status: "fulfilled"` con `value`, oppure `status: "rejected"` con `reason`).
- **`Promise.race(array)`**: simile a `Promise.all()`, ma si stabilisce appena una delle Promise di ingresso si stabilizza, e ne assume il risultato o l'errore.
- **`Promise.any(array)`**: simile a `Promise.race()`, ma attende la prima Promise risolta; se tutte le Promise di ingresso vengono rifiutate, si rifiuta con un `AggregateError` contenente tutti gli errori.
- **`Promise.resolve(value)`** e **`Promise.reject(error)`**: creano Promise già stabilite (rispettivamente risolta o rifiutata); a volte sono usate per compatibilità, ad esempio quando una funzione è attesa che restituisca una Promise.

```JavaScript
Promise.all([
  new Promise(res => setTimeout(() => res(3), 3000)),
  new Promise(res => setTimeout(() => res(2), 2000)),
  new Promise(res => setTimeout(() => res(1), 1000)),
]).then((result) => console.log(result)); // [3, 2, 1]
```

## async e await

Il JavaScript moderno include una sintassi speciale per lavorare con le Promise, composta da due parole chiave: `async` e `await`.

### async

La parola chiave `async`, posta prima di una funzione, garantisce che la funzione restituisca sempre una Promise.

- Qualsiasi altro valore restituito da una funzione `async` viene implicitamente incapsulato in una Promise risolta.
- Gli errori lanciati vengono incapsulati in una Promise rifiutata.

```JavaScript
async function g() {
  return 1; // equivale a restituire Promise.resolve(1)
}
```

### await

La parola chiave `await` si può usare solo all'interno di funzioni `async`, e mette in pausa l'esecuzione del codice finché una Promise non si stabilisce.

```JavaScript
async function f() {
  let promise = new Promise((res) => setTimeout(() => res("Done"), 2000));
  let result = await promise; // l'esecuzione si mette in pausa qui
  return result;
}
```

# Richieste di rete

JavaScript può anche scaricare dati da Internet.

### XMLHttpRequest

Nel vecchio JavaScript i programmatori usavano oggetti `XMLHttpRequest`, con callback per gestire gli eventi di stato.

### fetch

**Definition**

**fetch()**

**Modo moderno di eseguire richieste HTTP** in JavaScript: prende in ingresso un URL e un array opzionale di opzioni, e restituisce una Promise che si risolve in un oggetto `Response`.

```JavaScript
let promise = fetch("url", [options]);
```

Le opzioni permettono di specificare metodo HTTP, intestazioni della richiesta, eccetera.

### Lavorare con la risposta

**Warning**

**Risoluzione anticipata**

La Promise restituita da `fetch()` si risolve appena arrivano le intestazioni della risposta: in quel momento si possono controllare stato e intestazioni HTTP, ma il corpo della risposta potrebbe non essere ancora disponibile.

Sull'oggetto `Response` si possono controllare stato e intestazioni.

- `response.status` contiene il codice di stato HTTP.
- `response.ok` indica se la richiesta ha avuto successo.
- `response.headers.get('Content-Type')` legge una specifica intestazione; le intestazioni della risposta sono iterabili.

```JavaScript
let response = await fetch("./example.json");
console.log(response.headers.get('Content-Type'));
if (response.ok) { /* richiesta riuscita */ } else { /* errore */ }
```

### Corpo della risposta

L'oggetto `Response` fornisce diversi metodi basati su Promise per accedere al corpo in vari formati, non appena diventa disponibile: ad esempio `.json()` interpreta il corpo come JSON.

| Metodo | Risultato |
|---|---|
| `.text()` | Legge il corpo della risposta e lo restituisce come testo |
| `.json()` | Interpreta il corpo della risposta come JSON |
| `.blob()` | Restituisce un Blob (dati binari con tipo) |
| `.arrayBuffer()` | Restituisce un ArrayBuffer (rappresentazione binaria di basso livello) |
| `formData()` | Restituisce un FormData (dati inviati tramite form) |

```JavaScript
let response = await fetch("https://catfact.ninja/fact");
if (response.ok) {
  let fact = await response.json(); // corpo interpretato come JSON
}
```

**Warning**

**Corpo consumabile una sola volta**

Si può scegliere **un solo** metodo di lettura del corpo: se è già stato chiamato `response.text()`, `response.json()` non funziona perché il contenuto del corpo è già stato consumato.

# Parte VI

# Server Side

Capitolo 12

# Lezione 12 Web statico e dinamico

## Richiami su Promise e Fetch

### Promise

Un rapido richiamo sul ciclo di vita e sul consumo delle promise.

- **Creazione**: `new Promise(executor)` crea la promise; l'executor riceve le funzioni `resolve` e `reject` per stabilirne l'esito.
- **`setTimeout`**: prende in input una callback e un tempo in millisecondi.
- **Consumo**: `.then(onResult, onError)` gestisce rispettivamente successo ed errore; `.finally()` esegue una callback quando la promise si è *settled*.
- **Asincronia**: le callback della promise vengono eseguite dopo il codice sincrono: `Here` e `There` vengono stampati prima di `Promise settled` e del risultato.

```JavaScript
let promise = new Promise((resolve, reject) => {
    // l'executor chiama resolve(value) o reject(new Error(...))
});
promise.finally(() => { /* eseguita quando la promise si settle */ });
promise.then(
    (result) => { /* successo */ },
    (error) => { /* errore: error.message */ });
```

### Fetch API

`fetch()` restituisce una promise che risolve con un oggetto `Response`.

- **Header specifico**: si legge con `response.headers.get(name)`.
- **Tutti gli header**: `response.headers` è iterabile con `for(let [key, value] of response.headers)`.
- **Stato**: `response.status` contiene il codice di stato HTTP; `response.ok` è `true` quando la richiesta ha avuto successo.
- **Corpo**: quando la promise di `fetch()` si risolve, il corpo potrebbe non essere ancora disponibile; la `Response` fornisce metodi *promise-based* per accedere al corpo in vari formati.
  - **`.json()`**: elabora il corpo come JSON appena diventa disponibile.

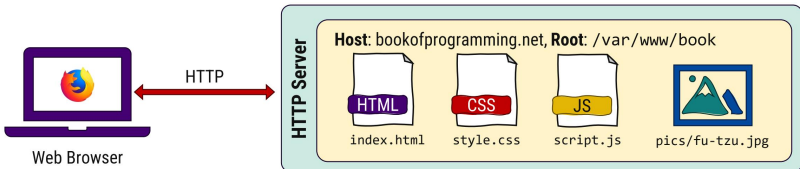
```JavaScript
async function f(){
    let response = await fetch("./example.json");
    response.headers.get('Content-Type');   // un singolo header
    for(let [key, value] of response.headers){ /* iterazione */ }
    let status = response.status;           // codice di stato
    if(response.ok){ /* richiesta riuscita */ }
    let fact = await response.json();       // corpo come JSON
}
```

## Il web statico

Finora abbiamo imparato le tecnologie core del web, **HTML**, **CSS** e **JavaScript**, progettando pagine moderne, responsive e dinamiche: gestione di eventi, modifica del DOM al volo e richieste HTTP asincrone per recuperare dati da mostrare nelle pagine.

### App web statiche

- **Dinamicità nel browser**: il dinamismo dato da JavaScript avviene interamente dentro il browser.
- **Documento identico**: ogni richiesta a una stessa pagina riceve sempre lo stesso documento identico, servito dal **document root** del server.




### Limitazioni del web statico

L'approccio statico è semplice ed efficace per applicazioni con un numero limitato di pagine che cambiano raramente, ma soffre di alcuni limiti.

- **Personalizzazione**: non permette di adattare una pagina a un utente specifico.
- **Contenuti variabili**: non è adatto quando il contenuto di una pagina cambia molto frequentemente.
- **Scala**: non regge siti costituiti da milioni di pagine.
- **Azioni**: non permette di agire in base a una richiesta (es. salvare un ordine).

Info

**Costo della duplicazione**

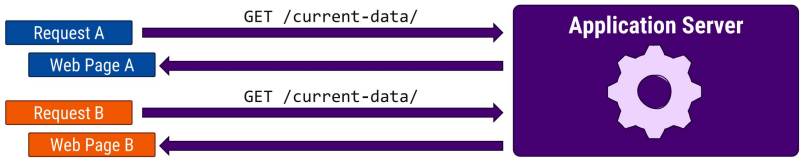
In un sito di ~60 milioni di pagine con la stessa struttura da ~150kB ripetuta in ogni pagina si accumulano ~9TB di dati duplicati; un elemento comune (es. la *navbar*) andrebbe modificato in tutte le pagine.

## Programmazione server-side

### Generazione on-the-fly

Un web server può fare molto più che servire file: può **generare** pagine web **on-the-fly**, tipicamente in base ai parametri ricevuti nella richiesta HTTP. È l'idea chiave dietro la **programmazione server-side**.

- **Parametri della richiesta**: richieste diverse, anche verso la stessa risorsa, producono pagine diverse.
- **Contenuti sempre aggiornati**: la pagina viene costruita al momento della richiesta, non preparata in anticipo.




## CGI

### Definizione

Definition

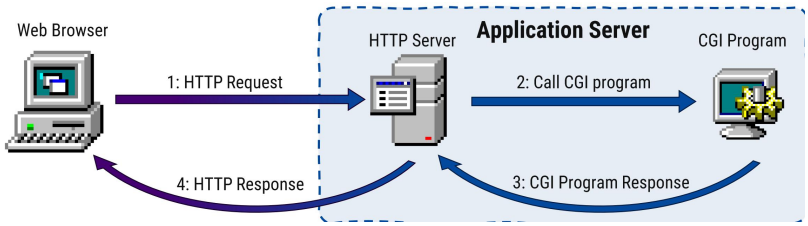
**Common Gateway Interface**

**Specifica di interfaccia** che consente a un web server di eseguire programmi esterni per processare una richiesta HTTP. Diffusa nei primi anni '90, prima ancora dell'esistenza di JavaScript, fu il primo strumento per ottenere pagine web dinamiche.

### Flusso di una richiesta

Il CGI definisce come comunicano il web server e i programmi **CGI-compliant**.

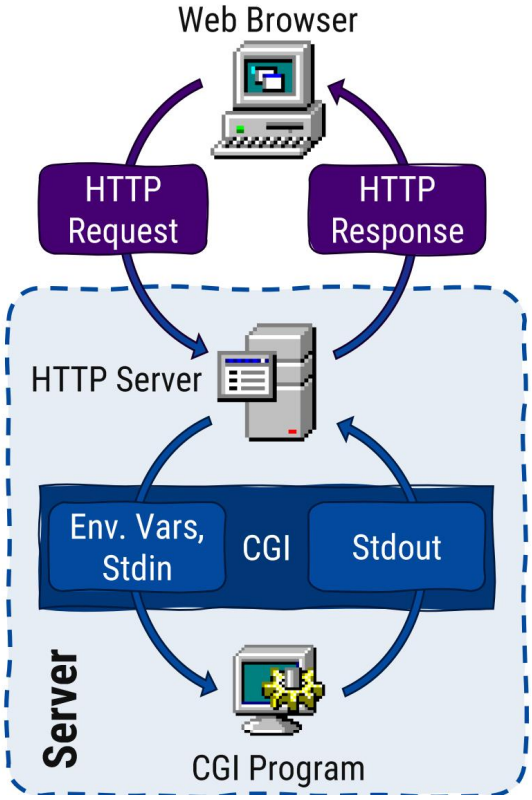
- **Browser → Server**: il browser invia la richiesta HTTP al web server.
- **Server → Programma**: il server invoca il programma CGI per processarla.
- **Ritorno**: l'output del programma torna al server, che lo invia al browser come risposta HTTP.




### Input e output del programma

Il programma CGI accede ai suoi input e costruisce la risposta secondo regole precise.

- **Header e query string della richiesta**: disponibili tramite **variabili d'ambiente**.
- **Corpo della richiesta**: disponibile al programma sullo stream **stdin**.
- **Corpo della risposta**: è l'output del programma sullo stream **stdout**.




### Primo script CGI

Il primo script CGI mostra la struttura essenziale della risposta: prima gli header (es. `Content-type`), poi una riga vuota, poi il corpo HTML; un contenuto generato al volo (la data corrente) dimostra la dinamicità della pagina.

```bash
#!/bin/bash
echo "Content-type: text/html; charset=utf-8"
echo ""
echo "<h1>Hello CGI!</h1>"
date +"%d/%m/%Y at %H:%M:%S"
```

### Limiti dell'approccio CGI

Scrivere programmi CGI non è un'esperienza piacevole.

- **HTML su stdout**: scrivere interi documenti HTML sullo standard output è macchinoso.
- **Parsing manuale**: analizzare a mano query string e corpi delle richieste è tutt'altro che divertente.
- **Processi**: creare un nuovo processo per ogni richiesta non è molto efficace in termini di risorse.

### Linguaggi di scripting server-side

Per fortuna sono emersi linguaggi e framework specializzati per lo scripting server-side.

- **PHP**: acronimo ricorsivo di *PHP Hypertext Preprocessor*, 1995.
- **ASP**: *Active Server Pages*, linguaggio di scripting server-side di Microsoft, 1997.
- **JSP**: *Java Server Pages*, rilasciato originariamente da Sun, 1999.

L'idea era «mixare» codice server-side e HTML: un interprete speciale elabora il mix per produrre codice HTML «puro».

### PHP

#### Delimitatori

PHP usa **delimitatori** speciali per separare codice e HTML.

- `<?php ... ?>`: racchiudono il codice PHP.
- **Testo esterno ai delimitatori**: resta così com'è nell'output.
- **Codice interno ai delimitatori**: viene interpretato e l'output risultante è inserito nell'HTML.
- `<?=`: shorthand per `<?php echo`.

PHP

```php
<title><?php echo "Hello PHP" ?></title>
<?php $array = ["Hello", "PHP"] ?>
<ul>
    <?php foreach($array as $item): ?>
        <li><?= $item ?></li>
    <?php endforeach; ?>
</ul>
```

#### Gestione delle richieste

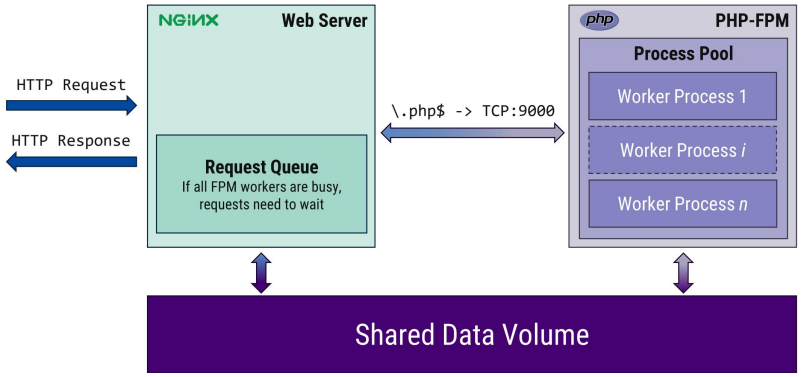
Le soluzioni di scripting server-side supportano lo sviluppatore nei compiti tediosi tipici della gestione delle richieste HTTP: le richieste vengono pre-processate automaticamente.

- **Parametri della richiesta**: disponibili negli array associativi `$_GET` e `$_POST`; ad esempio `$_POST['todo']` accede al parametro `todo`.
- **Cookie**: già elaborati nell'array associativo `$_COOKIES`.
- **Risposta**: non è necessario scrivere esplicitamente l'intera risposta su `stdout`.

#### Architettura NGINX e PHP-FPM

Un'applicazione PHP moderna può essere distribuita con **NGINX** come web server e **PHP-FPM** (*FastCGI Process Manager*) come gestore dei processi PHP.

- **NGINX**: web server che riceve le richieste HTTP in arrivo e le smista.
- **Pool di worker**: PHP-FPM gestisce un pool di processi worker che eseguono il codice PHP.
- **Comunicazione**: NGINX inoltra a PHP-FPM le richieste PHP via TCP sulla porta `9000`; i due componenti condividono un volume di dati comune.


NGINX
Web Server
HTTP Request
HTTP Response
Request Queue
If all FPM workers are busy, requests need to wait
\.php$ -> TCP:9000
php
PHP-FPM
Process Pool
Worker Process 1
Worker Process i
Worker Process n
Shared Data Volume

Capitolo 13

# Lezione 13 Node.js

## JavaScript in ambiente server

Col *server-side scripting* le pagine web possono essere generate al volo da programmi in esecuzione sul server; JavaScript, finora usato solo in un ambiente browser, può essere eseguito anche in un ambiente server.

Per eseguire JavaScript lato server sono disponibili diversi **runtime**, tra cui **Node.js**, **Deno** e **Bun**.

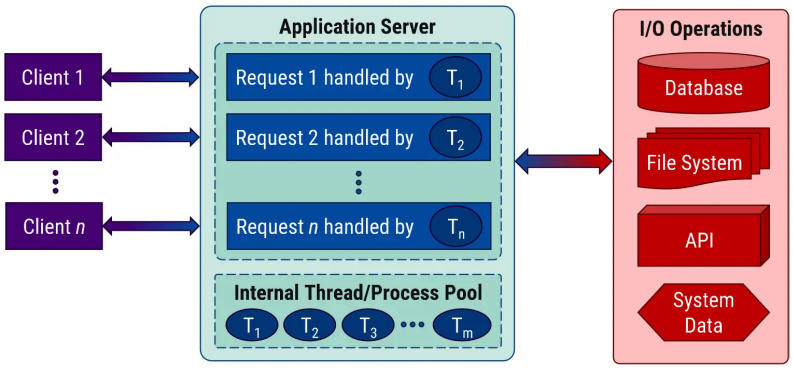
**Definition**

**Node.js**

**Runtime Environment** per JavaScript *open-source* e *cross-platform*, che esegue il motore **V8** di Google proveniente dal progetto Chromium. Si basa su un modello *event-based*, asincrono per default e a I/O *non-blocking*, ed è di gran lunga l'ambiente di esecuzione JavaScript più popolare.

## Gestione multi-threaded delle richieste

Negli altri ambienti di esecuzione e linguaggi, ogni richiesta è gestita da un thread o processo dedicato, prelevato da un pool interno all'*application server*.




Nel servire le richieste, le web application devono spesso eseguire operazioni I/O **bloccanti** e time-consuming: **accedere a un database**, **accedere a un'API esterna** (es. con `fetch()`), **leggere un file** dal file system locale, **leggere il body della richiesta**. Di conseguenza, i thread o processi che gestiscono una richiesta passano molto tempo in attesa del completamento di queste operazioni.

Il modello multi-threaded ha due limiti:

- **Inefficienza**: gran parte del tempo dei thread è speso in attesa delle operazioni bloccanti.
- **Scalabilità limitata**: il thread pool è pre-allocato a una dimensione predefinita, quindi non si possono gestire più richieste concorrenti di quante il pool ne permetta.

## La filosofia di Node.js

Node.js adotta una filosofia alternativa al modello multi-threaded, orientata all'efficienza.

- Un programma Node.js viene eseguito in **un singolo processo**: non serve creare un nuovo thread per ogni richiesta.
- Il runtime fornisce nella standard library un insieme di **primitive di I/O asincrone** che impediscono al codice JavaScript di bloccarsi.
- La maggior parte delle librerie sfrutta il paradigma *non-blocking* ogni volta che è possibile.
- Vasta adozione del **callback pattern**: la funzione da eseguire al termine dell'operazione viene registrata al momento della chiamata.

- Le operazioni sincrone sono generalmente un'eccezione più che la norma.

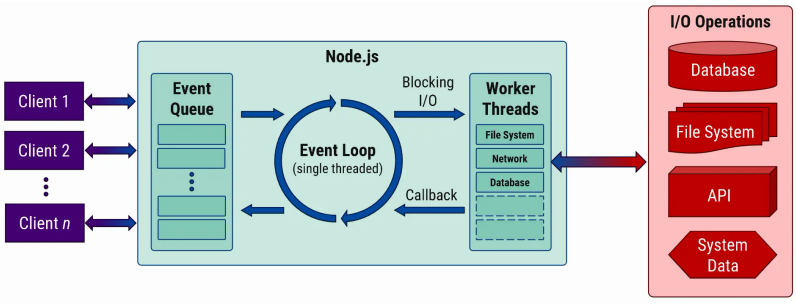
### Il single-threaded event loop

**Definition**

**Event loop**

**Ciclo a thread singolo** al centro del runtime Node.js: preleva gli eventi in coda, esegue i callback e delega le operazioni I/O bloccanti ai *worker thread*.

- **Event queue**: le richieste dei client arrivano come eventi in coda, in attesa di essere processati.
- **Worker thread**: eseguono le operazioni I/O bloccanti (file system, rete, database) al posto del thread dell'event loop.
- **Callback**: al termine dell'operazione la callback torna all'event loop, che può completare la risposta al client.




## Installazione

**Lab**

Node.js si può installare in tre modi:

- **Installer ufficiali** disponibili sul sito web del progetto.
- **Package manager** di sistema.
- **Version manager** come `nvm`, `nvs` o `nvm4w`, comodi per sperimentare versioni diverse e passare frequentemente dall'una all'altra.

La corretta installazione si verifica con `node --version`, che stampa la versione di Node.js in uso.

## Node.js vs browser

L'ambiente di esecuzione determina gli oggetti e le API disponibili:

- **Ambiente browser**: JavaScript manipola principalmente il DOM, con oggetti e API della piattaforma web come `window`, `document`, Cookies, LocalStorage.
- **Ambiente Node.js**: questi oggetti e API non sono disponibili, ma si possono sfruttare i moduli della standard library per l'accesso al file system e alla rete.
- In Node.js è possibile **leggere e scrivere file** sul filesystem e **ascoltare su una socket** le richieste in arrivo.

## npm

**Definition**

**npm**

**Node Package Manager**, gestore di pacchetti che fornisce i modi per scaricare e gestire le dipendenze dei progetti Node.js. È uno dei principali motivi del successo di Node.js: a settembre 2022 contava oltre 2,1 milioni di pacchetti, il più grande repository monolinguaggio al mondo, con un pacchetto per quasi tutto.

- npm è installato di default insieme a Node.js; la sua disponibilità si verifica con `npm --version`.
- Un pacchetto npm si crea nella directory corrente con `npm init`, che chiede le informazioni sul pacchetto: nome, versione, repository (se presente), licenza, autore, *entry point* (il primo file JavaScript eseguito) e comando di test (se presente).
- Per creare un pacchetto compatibile con i **moduli ES** si usa `npm init es6`.

### Il file package.json

Definition

**package.json**

**File descrittore del pacchetto** generato da `npm init` nella stessa directory: contiene tutte le informazioni sul pacchetto, incluse le dipendenze e i comandi per eseguirlo. È la controparte npm del `pom.xml` di Maven.

Le sue proprietà principali sono:

- **`main`**: file (es. `app.js`) eseguito quando il codice client importa il pacchetto; in genere esporta le variabili e le funzioni pubbliche.
- **`type`**: imposta il pacchetto a modulo ES con il valore `"module"`.
- **`scripts`**: task da riga di comando, eseguibili con `npm <taskname>`; il task `test` è definito di default, mentre un task `start` si aggiunge modificando la proprietà.

JSON

```
{
  "name": "hello-web-technologies",
  "version": "0.0.1",
  "main": "app.js",
  "type": "module",
  "scripts": { "start": "node app.js" },
  "dependencies": { "cowsay": "^1.5.0" }
}
```

### Installare le dipendenze

Per installare una dipendenza si esegue `npm install <nomepacchetto>`, con due effetti:

- La dipendenza viene tracciata nella nuova sezione `dependencies` del `package.json`.
- I pacchetti vengono scaricati nella directory `node_modules`, comprese le loro dipendenze ricorsive.

L'importazione di una dipendenza usa la sintassi dei moduli ES:

JavaScript

```
import cowsay from 'cowsay';
console.log(cowsay.think({ text: 'Hello Web Tech!' }));
```

### Distribuire un pacchetto

Per distribuire un pacchetto npm bastano il codice proprio e il descrittore `package.json`: la directory `node_modules` non va inclusa nel versioning.

Il comando `npm install`, eseguito senza argomenti, installa tutte le dipendenze elencate nel `package.json`.

## Una prima applicazione web

La standard library include il modulo `http`, con cui un server Node.js resta in ascolto su una porta e genera risposte HTML al volo:

JavaScript

```
import http from 'http';
const PORT = 3000;
http.createServer(function(request, response) {
  response.writeHead(200, { 'Content-Type': 'text/html' });
  response.write('<!DOCTYPE html><html><body><h1>Hello Web Tech</h1></body></html>');
  response.end();
}).listen(PORT);
```

# Debugging in VS Code

Lab

### Auto Attach

L'Auto Attach si attiva e configura così:

- Il modo più semplice per fare debugging è abilitare la funzione **Auto Attach**, che collega automaticamente un debugger ai processi Node.js avviati da VS Code.
- Si attiva con `CTRL+SHIFT+P` aprendo la palette dei comandi e cercando «Toggle Auto Attach».
- La modalità **Smart** collega il debugger solo agli script eseguiti fuori da `node_modules`.

A debugging avviato si lavora così:

- I **breakpoint** si aggiungono cliccando accanto al numero di riga; quando l'esecuzione li raggiunge, il codice si mette in pausa.
- Il pannello **Run and Debug** permette di ispezionare le variabili, osservare espressioni (*watch*), consultare lo *call stack* e altro, in modo più efficace del debugging via `console.log`.

### Azioni di debug

Le **debugging actions** permettono un controllo fine del flusso di esecuzione:

| Azione | Descrizione |
|---|---|
| Resume | Riprende il normale flusso di esecuzione, fino al prossimo breakpoint |
| Step Over | Esegue l'istruzione successiva come unità singola, senza ispezionarne i passaggi interni |
| Step Into | Entra nell'istruzione successiva per seguirne l'esecuzione riga per riga |
| Step Out | Da dentro un metodo, completa il metodo corrente come un unico comando e torna al contesto precedente |
| Restart | Termina l'esecuzione e riavvia il debugging con la configurazione corrente (non funziona con Auto Attach) |
| Disconnect | Termina la sessione di debug corrente |

# Live reloading

A ogni modifica del codice l'intero server va riavviato manualmente (`npm start`, `node app.js`, …), il che rende lo sviluppo poco agevole.

- Le utility di **live reloading** monitorano il codebase e riavviano il server di sviluppo quando serve.
- **nodemon** è una di queste utility: si installa con `npm install nodemon` e si usa sostituendo `node app.js` con `nodemon app.js`.

Capitolo 14

# Lezione 14

Durante la lezione 14 sono stati discussi esercizi d'esempio in ottica della prima prova intercorso.

Successivamente, è stata presentata la traccia del progetto dell'a.a. 2026/2027 per gli Insegnamenti di Tecnologie Web e Ingegneria del Software.

Capitolo 15

# Lezione 15 Web app con Node.js

## Il caso di studio

Come caso di studio per lo **scripting server-side** si implementa una **To-do list web app** in Node.js puro (v. 20.9.0), usando solo moduli standard di Node.js e il template engine Pug; l'app include un meccanismo di **session tracking** implementato da zero.

Warning

**App didattica**

L'app è intesa come primo esempio basilare per mostrare lo scripting server-side: non è rappresentativa delle pratiche moderne di sviluppo web.

## Il server HTTP in Node.js puro

Node.js permette di implementare in modo semplice ed efficiente un **server HTTP** sfruttando i moduli built-in e il **single-threaded event loop**. Il server si crea a partire dal modulo `http`:

JavaScript

```
import http from 'http';
const PORT = 3000;
let server = http.createServer();
server.listen(PORT);
server.on('request', handleRequest);
```

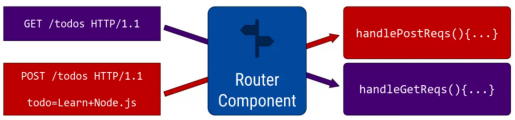
Node.js effettua anche del **pre-processing sulle richieste** (es. il parsing degli header) e fornisce un'astrazione per richieste (`http.IncomingMessage`) e risposte (`http.ServerResponse`). Al di là di questo, si è da soli nell'implementare l'app: i tre problemi da affrontare sono il **routing**, il **templating** e il **parsing del body delle richieste**.

## Routing

Definition

**Routing**

**Processo** che, data una richiesta HTTP, decide quale codice eseguire per servirla. Il componente che svolge questo compito è il **router**.




### Percorsi

Le richieste a percorsi diversi dell'app (homepage, pagina della lista, pagina di reset, …) possono essere gestite da funzioni diverse, selezionate sull'URL della richiesta:

```JavaScript
function handleRequest(request, response) {
  switch (request.url) {
    case "/":
      renderHomepage(request, response);
      break;
    case "/todo":
      handleTodoListRequest(request, response);
      break;
    default:
      handleError(request, response, 404, "Web page not found");
  }
}
```

Quando nessuna delle rotte note si applica, si restituisce un errore **404** (pagina non trovata).

### Metodi HTTP

Le richieste allo stesso percorso possono essere gestite diversamente a seconda del **metodo HTTP** usato: un `GET` a `/todos` restituisce la lista degli elementi, un `POST` a `/todos` prova a salvarne uno nuovo.

```JavaScript
switch (request.method) {
  case "GET":
    handleTodoListRequestGet(request, response);
    break;
  case "POST":
    handleTodoListRequestPost(request, response);
    break;
  default:
    handleError(request, response, 405, "Unsupported method");
}
```

Un metodo non supportato viene segnalato con un errore **405**.

### File statici

Se l'output HTML include file statici (immagini, CSS, JS), il router va configurato per servirli leggendoli dal filesystem con `fs.readFile` e impostando il `Content-Type` appropriato:

```JavaScript
case "/css/bootstrap.css":
  fs.readFile('./static/css/bootstrap.css', function (err, data) {
    response.writeHead(200, {'Content-Type': 'text/css'});
    response.end(data, 'utf-8');
  });
  break;
```

> *`function (err, data){...}` è una call-back*

### Oltre il caso semplice

Il routing sembra semplice, ma può crescere in complessità insieme all'app.

Info

**Complessità del routing**

- Alcune rotte possono essere accessibili solo ad alcuni utenti (es. amministratori).
- Alcune rotte dipendono da pattern o parametri nella richiesta (es. `GET /users/42/friends`, dove `42` è un qualunque id utente).
- Una richiesta può richiedere l'esecuzione di due o più pezzi di codice (es. una funzione che registra i dati e una diversa che prepara la risposta).

# Template engine

L'app deve produrre codice HTML da inviare nel body della risposta, così che il browser possa renderizzarlo. Nell'esempio più semplice l'HTML si scrive manualmente concatenando stringhe, ma parti di HTML si ripetono su più pagine (navbar, footer) e, al crescere della complessità, costruire risposte manipolando e concatenando stringhe diventa rapidamente invasibile.

**Definition**

**Template engine**

**Software** che genera HTML a partire da **template**, affrontando il problema del riuso delle parti comuni e della crescita della complessità delle pagine.

## Pug

**Definition**

**Pug**

**Template engine** ad alte prestazioni implementato in JavaScript (port disponibili in molti altri linguaggi), con sintassi **whitespace-sensitive** (simile a Python) per scrivere template HTML, facilmente compilabili in codice HTML.



L'installazione avviene con `npm install pug`; l'uso più semplice richiede il modulo e chiama `pug.renderFile()` sul file del template, che restituisce l'HTML generato:

```JavaScript
import pug from "pug";
let html = pug.renderFile("./templates/basic_example.pug");
```

Nella sintassi di Pug l'indentazione esprime l'annidamento degli elementi HTML:

```Text
doctype html
html(lang="en")
  head
    title Hello Pug!
  body
    h1(class="foo") First Pug template
    p This is our first Pug template!
```

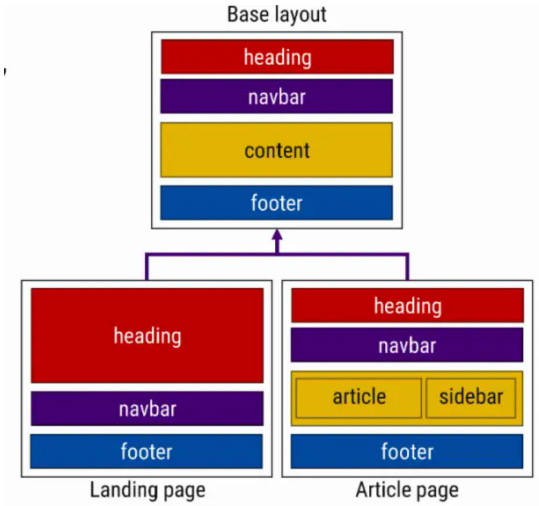
## Include

Un template può includere altri template con la keyword `include`: è un grande aiuto per il riuso dei template.

```Text
//- index.pug
doctype html
html
  include partials/head.pug
  body
    h1 Hello Pug!
    include partials/footer.pug
```

I partial inclusi (es. `partials/head.pug` con `head`, `title`, `script` e `link`; `partials/footer.pug` con il `footer`) vengono inseriti nel punto dell'include nell'HTML finale.

### Ereditarietà dei template




Nella progettazione di web application è comune un **layout comune** condiviso da tutte le pagine, che ciascuna pagina può eventualmente ridefinire in alcune parti; Pug supporta questi scenari con l'**ereditarietà dei template**, basata su due keyword:

- `block`: blocco di template che i template figli possono sostituire; può contenere un contenuto di default.
- `extends`: permette a un template di estenderne un altro e sovrascrivere alcuni dei blocchi definiti nel genitore; i blocchi non sovrascritti mostrano il contenuto di default.

```Text
//- base-layout.pug
body
  block heading
    h1 Pug Inheritance
  block content
  block footer
    footer This is the default footer.
```

```Text
extends ./base-layout.pug
block content
  p The actual content of the homepage.
block footer
  footer This is a specialized footer.
```

Nel secondo template i blocchi `content` e `footer` vengono sovrascritti, mentre `heading` non viene sovrascritto e mostra il valore di default.

### Locali

**Definition**

**Locali (locals)**

**Oggetto di dati** passato alle funzioni `render()` o `renderFile()` su cui il template può basare il contenuto da renderizzare.

```JavaScript
let html = pug.renderFile("./locals.pug", {
  "product": { "name": "Samsung S24", "description": "Smartphone" }
});
```

Il codice contenuto tra `#{` e `}` viene valutato, escapato e inserito nel buffer di output:

```Text
h1 #{product.name.toUpperCase()}
p Description: #{product.description}
```

### Condizionali

Spesso i template vengono renderizzati diversamente a seconda di certe condizioni; a questo scopo Pug supporta il familiare costrutto `if/else`:

```Text
if !user
  a(href="/login") Please, authenticate yourself.
else if user.isAdmin
  p At your command, #{user.name}
else
  p Welcome back, #{user.name}
```

### Iterazioni

Un task comune nell'uso dei template è iterare su sequenze di dati, con `each ... in` e un ramo `else` per la sequenza vuota:

```Text
ul
  each item in items
    li #{item.name} - € #{item.price.toFixed(2)}
  else
    li No pizza is available at the moment!
```

### Altri template engine

Pug è solo uno dei tanti template engine disponibili; altri ben noti sono elencati di seguito.

Info

**Altri template engine**

- **EJS** (ex Jake): sintassi in qualche modo simile a JSP/PHP.
- **SquirellyJS**, **Nunjucks**, **LiquidJS**, **Eta**.
- **Haml** (nato per Ruby).

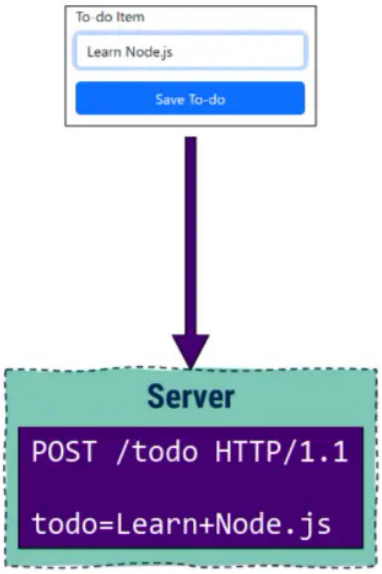
## Parsing del body delle richieste

Gli utenti salvano nuovi elementi riempiendo il form sulla pagina della To-do list, inviato con il metodo `POST` allo stesso URL della pagina:

```Text
form(action="/todo" method="POST")
  label(for="todo") To-do Item
  input(type="text" id="todo" name="todo" required)
  button(type="submit") Save To-do
```

Per salvare l'elemento serve leggere il body della richiesta ed estrarne i nomi dei parametri e i loro valori: il body è una stringa nella forma `p1=v1&p2=v2&...`, non banale come potrebbe sembrare.

### Lettura del body




Il body potrebbe non essere ancora disponibile quando si processa la richiesta (l'utente ha una connessione lenta o sta caricando file grandi) e diventa disponibile più tardi. L'oggetto richiesta (`http.IncomingMessage`) estende `stream.Readable`: genera un evento **`data`** quando un nuovo chunk di dati diventa disponibile nel body e un evento **`end`** quando non c'è più dati da consumare dallo stream.

La lettura del body va quindi operata in modo asincrono, sfruttando questi due eventi:

```JavaScript
function parseRequestBody(request) {
  return new Promise((resolve, reject) => {
    let body = [];
    request
      .on('data', chunk => { body.push(chunk); })
      .on('end', () => {
        body = Buffer.concat(body).toString();
        resolve(parseRequestBodyString(body));
      });
  });
}
```

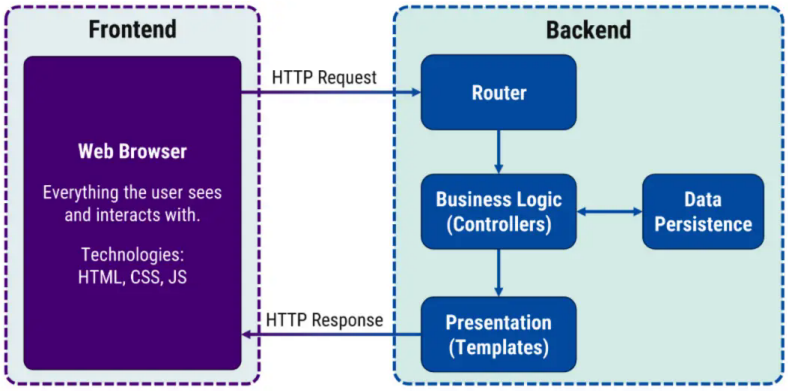
### Parsing della stringa

Una volta letto il body come stringa, si tratta di dividerla per ricavare i nomi e i valori dei parametri, sostituendo i `+` con spazi e decodificando i valori con `decodeURIComponent`:

```JavaScript
function parseRequestBodyString(body) {
  let data = {};
  for (let slice of body.split("&")) {
    let [paramName, paramValue] = slice.split("=");
    paramName = decodeURIComponent(paramName.replace(/\+/g, " "));
    paramValue = decodeURIComponent(paramValue.replace(/\+/g, " "));
    data[paramName] = paramValue;
  }
  return data;
}
```

## Anatomia di una web application tipica

Lo schema riassume l'organizzazione dei componenti di una web application tipica.


Frontend
Web Browser
Everything the user sees and interacts with.
Technologies: HTML, CSS, JS
HTTP Request
HTTP Response
Backend
Router
Business Logic (Controllers)
Data Persistence
Presentation (Templates)

Capitolo 16

# Lezione 16 Sessioni in Node.js

## HTTP e session tracking

Il protocollo HTTP, per come funziona «out of the box», non permette a un server di ricordare nulla delle richieste precedenti.

**Definition**

**HTTP stateless**

**Protocollo stateless**: una singola richiesta HTTP non contiene, di per sé, alcuna informazione sulle richieste precedenti. Senza meccanismi aggiuntivi il server non può sapere che un dato utente ha già effettuato il login.

**Definition**

**Session tracking**

**Tecnica** cruciale dello sviluppo web che consiste nel mantenere informazioni di stato (*stateful*) sulle interazioni di un utente attraverso richieste HTTP multiple. Permette al server di «riconoscere» gli utenti e le loro azioni, così da adattare le risposte a utenti e situazioni differenti.

## Sessioni basate su cookie

Un primo modo per far «memorizzare» dati al server tra richieste diverse è impostare **cookie**. L'idea dell'approccio ingenuo (*naive*) è la seguente.

1. L'utente invia una richiesta a una pagina dinamica con username e password come parametri.
2. La pagina verifica le credenziali: se corrette, il server imposta cookie di tracciamento (es. `Set-Cookie: auth=true ;username=luigi`); se errate, mostra una pagina di errore senza impostare alcun cookie.
3. Le pagine dinamiche che richiedono autenticazione controllano il cookie `auth`: se è impostato mostrano il contenuto, altrimenti reindirizzano alla pagina di login.

### Gestione della richiesta di login

Il login avviene con un form con campi `usr` e `pwd` via `POST`: il server parsifica il corpo della richiesta e controlla le credenziali. Se sono corrette, reindirizza a `/` con codice `302` e imposta due cookie, uno con lo username e l'altro con un valore booleano, entrambi con scadenza di 1 ora.

JavaScript

```
if (isAuthenticated) {
  response.writeHead(302, {
    "Location": "/",
    "Set-Cookie": [
      `auth=true; max-age=${60*60}`,
      `username=${user.username}; max-age=${60*60}`
    ]
  });
  response.end();
}
```

Se le credenziali non sono corrette, il server rende la pagina di login con un messaggio di errore e risponde con il codice `401` (Unauthorized).

### Verifica dell'autenticazione

Le richieste vanno gestite diversamente a seconda che l'utente sia autenticato o meno: un utente non autenticato che visita una rotta protetta riceve una pagina di errore con `401`, mentre un utente autenticato vede il contenuto riservato. Lo stato si verifica controllando i cookie presenti nella richiesta HTTP.

```JavaScript
function checkUserAuthentication(request) {
  let cookies = parseCookies(request);
  if (cookies.auth) return [true, cookies.username];
  return [false, undefined];
}
```

Node.js non offre un parser di cookie, quindi va scritto a mano: l'intestazione `Cookie` si spezza sul separatore `;` e ogni cookie sul carattere `=`, avendo cura di applicare `trim` e `decodeURIComponent` ai valori.

```JavaScript
function parseCookies(request) {
  const list = {};
  const header = request.headers?.cookie;
  header?.split(';').forEach(cookie => {
    const [name, ...rest] = cookie.split('=');
    list[name?.trim()] = decodeURIComponent(rest.join('=').trim());
  });
  return list;
}
```

### Propagazione del contesto di autenticazione

Lo stato di autenticazione influenza tutte le pagine dell'app, per esempio la navbar che mostra «Welcome, User» al posto di «Login». L'informazione va quindi resa disponibile a tutti i controller: il metodo `handleRequest` calcola una volta sola lo stato e lo passa ai gestori delle rotte dentro un oggetto `context` con i campi `isUserAuthenticated` e `username`. Ogni gestore di rotta controlla `context.isUserAuthenticated` e risponde con `401` se l'utente non è autenticato; solo in caso contrario instrada la richiesta in base al metodo HTTP, rispondendo `405` per i metodi non supportati.

## Limiti dell'approccio con cookie

L'approccio a cookie funziona ed è semplice da implementare, ma soffre di problemi critici.

Warning

**Cookie controllati dal client**

I cookie sono interamente controllati dal client ed è banale manometterli, il che rende di fatto inutile l'autenticazione basata su di essi.

- Un utente può cambiare a piacere il valore del cookie `username`, oppure impostarlo da solo senza mai visitare la pagina di login.
- Il problema del *tampering* si può affrontare con **cookie firmati** (*signed cookies*): il server firma i cookie con la propria chiave privata e, finché la chiave resta sicura, riconosce i cookie manomessi.

Info

**Limiti imposti dai browser**

I browser impongono limitazioni ai cookie per evitare problemi di prestazioni.

- **Dimensione**: un cookie è generalmente limitato a 4096 byte.
- **Quantità**: si possono memorizzare generalmente da 20 a 50 cookie per dominio.
- Se i dati di sessione da memorizzare superano questi limiti, i cookie non bastano.

## Sessioni lato server

Per tracciare informazioni sensibili delle interazioni precedenti mitigando il *tampering*, e per memorizzare più dati di quanti i cookie consentano, si usa il meccanismo di **sessione web** (RFC 6265).

Definition

**Sessione web**

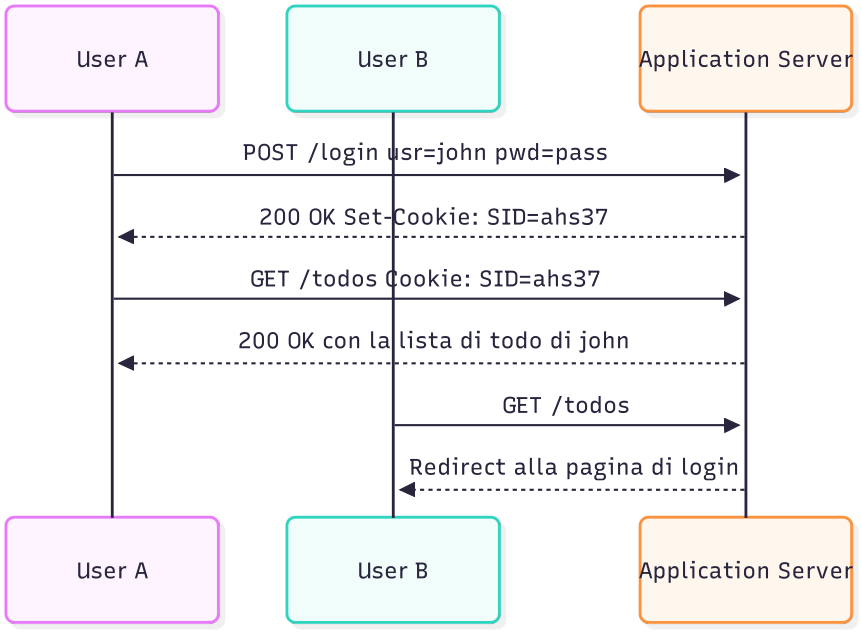
**Struttura dati** che memorizza coppie chiave-valore, gestita e archiviata sul server: quando serve mantenere informazioni tra richieste, il server crea una sessione per un dato client. La sessione è identificata da un unico

**session id** (o *session token*), passato al client generalmente come cookie.

La conseguenza chiave è che i dati di sessione restano sul server e il client non può manipolarli.

### Esempio di autenticazione con sessioni

Il server mantiene una sessione separata per ogni client, identificata dal proprio session id e contenente coppie chiave-valore come `auth = true` e `username = john`. Il flusso tipico è il seguente.




### Implementazione da zero in Node.js

Un meccanismo di sessione di base si implementa facilmente con i `Map` nativi di JavaScript: una prima `Map` associa ogni session id a una sessione, che a sua volta è una `Map` di coppie `<chiave, valore>`.

```JavaScript
class Session {
  constructor() { this.sessionStore = new Map(); }
  createSession() {
    let sessionId = this.generateNewSessionId();
    this.sessionStore.set(sessionId, new Map());
    return sessionId;
  }
  storeSessionData(sessionId, key, value) {
    return this.getSessionById(sessionId)?.set(key, value);
  }
}
```

Quando un utente effettua il login si crea una nuova sessione, vi si memorizzano i dati rilevanti e si passa il session id al client come cookie.

```JavaScript
if (isAuthenticated) {
  let sessionId = session.createSession();
  session.storeSessionData(sessionId, "username", user.username);
  session.storeSessionData(sessionId, "auth", true);
  response.writeHead(302, {
    "Location": "/",
    "Set-Cookie": [`sessionId=${sessionId}; max-age=${60*60}`]
```

```
  });
  response.end();
}
```

Per accedere ai dati di sessione a ogni richiesta, il metodo `getSessionFromRequest` legge il cookie `sessionId` tramite `parseCookies` e recupera la sessione corrispondente con `getSessionById`; i valori si leggono poi con `userSession?.get("auth")` e `userSession?.get("username")` e finiscono nel `context` condiviso dai controller.

### Metodi di session tracking

L'uso di un cookie per trasportare il session id è l'approccio più popolare perché semplice e ampiamente supportato, ma esistono alternative.

- **URL rewriting**: il session id viene aggiunto a ogni URL come parametro di query.
- **Campi form nascosti** (*hidden form fields*): il session id è memorizzato in campi nascosti dei form e viene inviato insieme agli altri dati quando l'utente sottomette il form.

Capitolo 17

# Lezione 17 Web Frameworks: Express

## Dalla programmazione server-side ai framework

Le applicazioni web sviluppate con la sola programmazione server-side (CGI con Bash, PHP 8.2, Node.js "puro") condividono attività ricorrenti: gestione della **routing**, rendering dei **template**, gestione delle **sessioni** web, gestione di **autenticazione/autorizzazione**. Senza un framework, queste attività presentano alcuni punti deboli:

- Molte operazioni **tediose** da reimplementare ogni volta, come il parsing dei cookie e la lettura e l'analisi dei corpi delle richieste.
- Nessuna gestione delle sessioni out-of-the-box, salvo PHP.
- Nessun supporto out-of-the-box per i **middleware** (es. autenticazione) e nessun **templating** out-of-the-box.

L'obiettivo di un framework è non reinventare la ruota in ogni progetto.

## Framework e librerie

Framework e librerie software sono cose diverse, e si distinguono per chi è al controllo del flusso di esecuzione.

### Libreria software

**Definition**

**Libreria software**

**Insieme di funzioni** richiamabili per far svolgere un certo lavoro: ogni chiamata esegue il compito e poi **restituisce il controllo al chiamante**. Il codice utente (es. il metodo `main`) è **al controllo del flusso di esecuzione**.

### Framework

**Definition**

**Framework**

**Insieme predefinito di componenti software, strumenti e buone pratiche** che serve da fondamenta per lo sviluppo di software; **incarna un progetto astratto** con molto comportamento già integrato. Il codice scritto dall'utente può essere **collegato (plugged-in)** al framework, che è **al controllo del flusso di esecuzione** e invoca il codice utente quando appropriato.

Un **web framework** è un framework progettato specificamente per semplificare e snellire lo sviluppo di applicazioni web: fornisce un modo (strutturato) di costruirle e organizzarle, riducendo la necessità di ripartire da zero.

### Inversion of Control

**Definition**

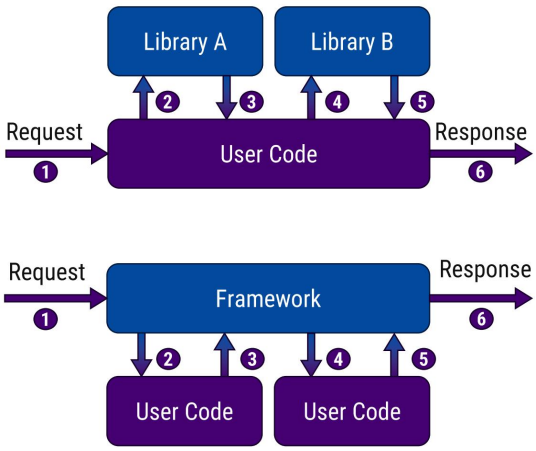
**Inversion of Control**

**Principio** in base al quale il flusso di controllo è **invertito** rispetto alla programmazione imperativa tradizionale: la responsabilità di gestire il flusso di esecuzione si sposta dallo sviluppatore al framework.

L'IoC è una **caratteristica distintiva** dei framework, nota anche come *legge di Hollywood*:

> *"Don't call us, we'll call you."*
>
> – Hollywood's Law

Il confronto tra i flussi di controllo in una libreria e in un framework:




## Componenti dei web framework

Un web framework tipicamente include tre componenti:

- **Funzionalità core**: routing, parsing delle richieste, validazione degli input, gestione dei cookie, gestione delle sessioni.
- **Template engine**: aiuta il rendering di contenuto dinamico, separando la logica del codice dal livello di presentazione.
- **ORM** (*Object-Relational Mapping*): semplifica le interazioni col database permettendo di lavorare con oggetti invece di query SQL.

### Framework opinionati

I framework possono essere più o meno **opinionati**.

Definition

**Framework opinionato**

**Framework** dotato di un insieme rigido di convenzioni, buone pratiche e decisioni predefinite prese dai suoi autori: impone uno specifico modo di fare le cose e lascia meno libertà agli sviluppatori. Un framework **unopinionated** non impone alcuno specifico modo di fare le cose.

- **Pro**: garantiscono livelli più alti di **consistenza** tra progetti diversi; possono velocizzare lo sviluppo riducendo la *decision fatigue*.
- **Contro**: curva di apprendimento potenzialmente più ripida; possono non essere sufficientemente flessibili per un certo progetto.

## Express

Esistono molti web framework server-side tra cui scegliere; come esempio del corso si vede **Express**, descritto come *fast, unopinionated, minimalist web framework* per **Node.js**: di gran lunga il framework backend più popolare per Node.js.

### Prima applicazione

Il pacchetto si installa con `npm install express`; un'applicazione minima crea l'app, definisce una route e mette il server in ascolto:

JavaScript

```
1  import express from "express";
2  const app = express(); // crea un'applicazione Express
3  app.get('/', (req, res) => { res.send('Hello World!'); });
4  app.listen(3000);
```

### Routing

La **routing** è il modo in cui il framework determina come gestire una richiesta a un dato **endpoint**; una route si definisce con metodi appropriati sull'oggetto `app`, nella forma `app.<METHOD>(<PATH>, callback)`, dove `<METHOD>` e `<PATH>` identificano un endpoint e `callback` è la funzione eseguita per gestire le richieste a quell'endpoint.

```JavaScript
app.get("/hello", (req, res) => { res.send("Hello"); });
app.post("/hello", (req, res) => { res.send("Hello Post"); });
```

### Route path

I path delle route possono essere stringhe, pattern di stringhe o espressioni regolari complete; in caso di corrispondenze multiple si applica la prima route che fa match.

```JavaScript
app.get("/webtech", (req, res) => { res.send("Web Technologies!"); });
// corrisponde a /webtech, /wirelesstech, /whatever-tech, ...
app.get(/^\w.*tech$/, (req, res) => { res.send("Regex!"); });
```

### Parametri di route

Le route possono catturare segmenti URL con nome, detti **parametri**, dichiarati nel `<PATH>` con `:paramName`; i valori catturati sono resi disponibili nell'oggetto `req.params`.

```JavaScript
// GET /student/42/exams/TecWeb
app.get("/student/:student_id/exams/:exam_name",
    (req, res) => { res.send(req.params); }
); // {"student_id":"42","exam_name":"TecWeb"}
```

### Più handler per route

Una route può avere più funzioni **handler**, che si comportano come middleware; `next` è il callback che invoca la funzione handler successiva da eseguire.

```JavaScript
let f1 = function (req, res, next) { console.log("f1"); next(); };
app.get("/handlers", [f1, f2, f3], (req, res) => {
    res.send("Done with all handlers");
});
```

### Route concatenabili

Con `app.route()` il path è specificato in un'unica posizione (meno ridondanza) e gli handler specifici per metodo vengono concatenati:

```JavaScript
app.route('/book')
    .get((req, res) => { res.send('Get a random book'); })
    .post((req, res) => { res.send('Add a book'); })
    .delete((req, res) => { res.send('Delete a book'); });
```

### Router modulari

`express.Router()` crea un **router modulare**, caricabile all'occorrenza; si monta con `app.use(<path>, router)`, e le richieste al path indicato (es. `/custom/echo/:value`) vengono gestite dal router.

```JavaScript
// router.js
export const router = express.Router();
```

```
router.get("/echo/:value", (req, res) => { res.send(req.params.value); });
// index.js
app.use("/custom", echoRouter);
```

### Metodi della risposta

Per inviare una risposta al client vanno chiamati metodi appropriati sull'oggetto `res`, altrimenti la richiesta resta appesa:

| Metodo | Descrizione |
|---|---|
| `res.download()` | Richiede il download di un file. |
| `res.end()` | Termina il processo di risposta. |
| `res.json()` | Invia una risposta JSON. |
| `res.jsonp()` | Invia una risposta JSON con supporto JSONP. |
| `res.redirect()` | Redirige una richiesta. |
| `res.render()` | Fa il rendering di un template. |
| `res.send()` | Invia una risposta di vario tipo. |
| `res.sendFile()` | Invia un file come octet stream. |
| `res.sendStatus()` | Imposta il codice di stato e ne invia la rappresentazione testuale come corpo. |

### Rendering dei template

Il rendering di un template (es. **Pug**) è un processo diretto: si imposta il template engine con `app.set()` e si renderizza una vista con `res.render()`; inoltre, i dati impostati in `res.locals` sono disponibili a tutte le viste che renderizzano `res`.

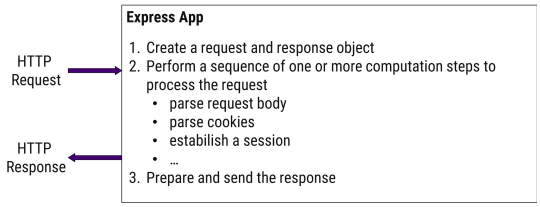
JavaScript

```
app.set("view engine", "pug");
app.get("/example", (req, res) => {
    res.locals.items = [{name: "Web tech.", grade: "A+"}];
    res.render("example"); // items è disponibile nella vista
});
```

### Ciclo richiesta-risposta

Il ciclo richiesta-risposta è il cuore del framework Express:

1. Creazione degli oggetti richiesta e risposta.
2. Esecuzione di una sequenza di uno o più passi di calcolo per processare la richiesta, come il parsing del corpo, il parsing dei cookie e l'istituzione di una sessione.
3. Preparazione e invio della risposta.



# Middleware

Definition

**Middleware**

**Funzione** che può manipolare gli oggetti richiesta e risposta correnti o eseguire codice arbitrario; è così chiamata perché eseguita tra la ricezione di una richiesta e l'invio della relativa risposta. I middleware sono eseguiti **nell'ordine di dichiarazione** e ricevono in input `req` (la richiesta HTTP), `res` (la risposta HTTP) e `next`, un callback che invoca il middleware successivo nello stack.

Un middleware si monta con `app.use()`, che lo registra su un dato path (default `/`):

JavaScript

```
export function addTimestamp(req, res, next) {
    req.timestamp = new Date();
    next();
}
app.use(addTimestamp); // monta il middleware sul path "/"
app.get('/', (req, res) => { res.send(`Ts: ${req.timestamp}`); });
```

### Gestione degli errori

È importante assicurarsi che ogni errore occorso durante l'elaborazione di middleware o route sia gestito correttamente da Express:

- Un errore **lanciato** durante l'esecuzione di codice **sincrono** è catturato automaticamente dal default error handler di Express.
- Gli errori restituiti da **funzioni asincrone** invocate da route handler e middleware devono essere passati alla funzione `next()`.

JavaScript

```
app.get('/readfile', (req, res, next) => {
    fs.readFile('/file-does-not-exist', (err, data) => {
        if (err) next(err); // passa l'errore a Express
        else res.send(data);
    });
});
```

A partire da **Express 5**, i route handler e i middleware che restituiscono una **promise** invocano automaticamente `next(value)` quando la promise si respinge o quando lanciano un errore: non è necessario farlo esplicitamente.

JavaScript

```
app.get('/user/:id', async (req, res) => {
    const user = await getUserById(req.params.id);
    res.send(user);
});
```

### Error handler personalizzati

Express include un **default error handler** che stampa l'intero stack trace solo in *development mode*; è possibile definire error handler personalizzati, middleware speciali con **quattro parametri** in input:

JavaScript

```
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).send('Something broke!');
});
```

# Middleware utili

Oltre a quelli scritti dal programmatore, esistono middleware pronti all'uso, built-in o disponibili via npm.

### express.static

Middleware built-in progettato per servire **file statici**: usa `req.url` per determinare se una richiesta corrisponde a un file o path della directory root specificata (es. `public`).

```JavaScript
app.use(express.static("public")); // montato nella root "/"
app.use("/static", express.static("public")); // path virtuale "/static"
```

### express.urlencoded

Middleware built-in che fa il parsing del corpo delle richieste con payload **urlencoded** (es. quelle inviate tramite form), rendendo i parametri disponibili nell'oggetto `req.body`; si registra con `app.use(express.urlencoded({extended: false }))`.

### express.json

Middleware built-in dal comportamento analogo a `express.urlencoded`, ma progettato per fare il parsing di corpi **JSON**: anche qui i valori finiscono in `req.body`.

### express-session

Middleware disponibile tramite **npm** che semplifica la gestione di una sessione conservata sul server: i dati di sessione sono conservati sul server, mentre l'**id di sessione** è conservato in un cookie.

Warning

**Storage di sessione**

Lo storage server-side di default **non è production-ready**: per la produzione va configurato uno store di sessione dedicato (es. basato su *memcached* o *redis*).

```JavaScript
app.use(session({ secret: "s3cr37", resave: false, saveUninitialized: false }));
app.get("/session-counter", (req, res) => {
    if (!req.session.count) { req.session.count = 1; } // dati in req.session
    else { req.session.count = req.session.count + 1; }
});
```

### cookie-parser

Middleware disponibile tramite npm che fa il parsing dell'header `Cookie` delle richieste in arrivo e popola `req.cookies` con un oggetto le cui proprietà sono i nomi dei cookie.

### Flash message

Spesso, in un'applicazione web, l'utente viene rediretto dopo aver eseguito un'azione (es. alla homepage dopo una cancellazione): è una buona regola fornire un riscontro, così che sappia se l'operazione è riuscita o meno. La difficoltà è che dopo la redirezione inizia un **nuovo ciclo richiesta-risposta**: è una variante del solito problema di **HTTP** senza stato, e si risolve negli stessi modi visti in precedenza (cookie, sessioni).

- `connect-flash` è un semplice middleware per flash message, disponibile su npm, che si registra con `app.use(flash())`.
- `req.flash(chiave, messaggio)` salva un flash message nella sessione corrente.
- `req.flash(chiave)` consuma i flash message con la chiave data.

```JavaScript
app.get("/redirect", (req, res) => {
    req.flash("info", "You have been redirected."); // salva
    res.redirect("/showflash");
});
app.get("/showflash", (req, res) => {
    let infos = req.flash("info"); // consuma i messaggi con chiave "info"
});
```

Poiché i flash message sono **consumati alla lettura**, ricaricando la pagina (o visitandola direttamente senza essere stati rediretti) nessun flash message è presente nella sessione e non viene mostrato nulla.

Capitolo 18

# Lezione 18 Web app con Express

## Object-Relational Mapping

Quando si sviluppa software object-oriented che persiste dati in un RDBMS si gestiscono due rappresentazioni degli stessi dati: **oggetti nello heap** e **righe in tabelle relazionali**.

Le due rappresentazioni sono molto diverse e vanno tenute allineate: le modifiche ai domain object devono essere mappate sullo schema del RDBMS, e le modifiche allo schema possono impattare il codice esistente. Servono una buona quantità di lavoro e di codice boilerplate ripetitivo.

Definition

**Libreria ORM**

**Soluzione pronta all'uso** che supporta gli sviluppatori nel tenere allineate la rappresentazione a oggetti e quella relazionale, consentendo di lavorare con astrazioni ad alto livello invece che con query e statement SQL.

In genere una libreria ORM permette di:

- **Definire dichiarativamente** come le tabelle del database vengono mappate su oggetti.
- **Creare e aggiornare automaticamente** lo schema del RDBMS quando serve.
- **Eseguire facilmente le operazioni CRUD** (Create, Read, Update, Delete).
- **Comporre ed eseguire query complesse** in modo dichiarativo.
- **Gestire transazioni** e **supportare strategie di caching**.

I benefici sono minore boilerplate, più produttività e cambio più facile di RDBMS. Esempi noti: **Hibernate** (Java), **Entity Framework** (C#), **SQLAlchemy** (Python), **Propel** (PHP), **Active Record** (Ruby on Rails), **Sequelize**, **Prisma** e **TypeORM** (Node.js). I framework opinionati a volte includono un proprio ORM.

## Sequelize

Definition

**Sequelize**

**ORM per Node.js** che supporta Postgres, MySQL, MariaDB, SQLite, SQL Server e altri DB, con un'elegante API basata su Promise.

Si installa con `npm install sequelize` più il driver del RDBMS scelto (per il corso si usa SQLite, con il driver `sqlite3`). La connessione si crea istanziando `Sequelize` con la stringa di connessione:

JavaScript

```
1 import { Sequelize } from "sequelize";
2 const database = new Sequelize("sqlite:mydb.sqlite");
3 await database.authenticate();
```

### Modelli

Definition

**Modello**

**Classe che estende `Model`** e definisce come una tabella del database viene mappata su oggetti; è l'essenza di ogni ORM.

Esistono due modi equivalenti di definire un modello:

- Chiamare il metodo `define` sulla connessione al database.
- Creare una classe che estende `Model` e chiamare poi il suo metodo `init`.

```JavaScript
const User = database.define('User', {
  firstName: { type: DataTypes.STRING, allowNull: false },
  age: { type: DataTypes.INTEGER }, // allowNull default: true
  id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true }
});
```

### Sincronizzazione dei modelli

All'avvio il database è vuoto: **model synchronization** è il meccanismo con cui Sequelize allinea lo schema del database ai modelli definiti, tramite il metodo asincrono `model.sync()`, che ritorna una Promise.

Alcuni aspetti del processo:

- Sequelize nomina le tabelle col **plurale** del nome del modello, salvo override esplicito.
- Prima verifica che la tabella esista (`SELECT` di controllo, poi `CREATE TABLE IF NOT EXISTS`); se esiste già, non fa nulla.
- Chiamando `sync()` sulla connessione si sincronizzano tutti i modelli insieme.

Le modalità di sincronizzazione sono tre:

- `User.sync()`: crea la tabella solo se non esiste.
- `User.sync({ force: true })`: ricrea la tabella, droppandola se esiste, è distruttivo.
- `User.sync({ alter: true })`: controlla lo stato della tabella (colonne, tipi…) e la altera per farla combaciare col modello.

### Creazione, lettura, modifica, cancellazione

Un'entità si può creare in due modi: istanziare il modello con `new` e chiamare `save()`, oppure usare il metodo statico `Model.create()`:

```JavaScript
let riff = await User.create({ name: "Riff Raff", age: 26 });
```

La lettura dei dati usa metodi dedicati sui modelli:

- `findAll()` recupera tutte le istanze; accetta un oggetto `where` con operatori della costante `Op` (es. `Op.gte`, `Op.like`) per filtrare.
- `include: [User]` carica anche le associazioni, che per default non vengono recuperate.
- `findByPk(pk)` recupera un'entità a partire dalla sua chiave primaria.

Per aggiornare un'entità si usa `setDataValue()` seguito da `save()`, che riscrive la riga sul database; per cancellarla si chiama `destroy()`, che elimina la riga.

### Associazioni

Sequelize supporta associazioni **uno-a-uno**, **uno-a-molti** e **molti-a-molti**:

- `A.hasOne(B)`: A ha una B.
- `A.belongsTo(B)`: A appartiene a B.
- `A.hasMany(B)`: A ha molte B.
- `A.belongsToMany(B, { through: 'C' })`: A è collegata a molte B tramite la tabella di giunzione C.

Definendo un'associazione, Sequelize crea automaticamente una colonna foreign key sul modello target, per default chiamata `NomeModello` + `NomePK` (es. `UserId` su `Todo`). Un'istanza può essere creata insieme alle sue associazioni annidate in un solo passo, purché tutti gli elementi siano nuovi, passando le entità figlie e l'opzione `include: [Modello]`.

### Pro e contro dell'ORM

I vantaggi principali dell'approccio ORM sono:

- **Astrazione**: si lavora con oggetti e modelli invece che con SQL.
- **Produttività**: meno boilerplate, schema generato automaticamente.
- **Portabilità**: è facile passare tra RDBMS supportati.
- **Leggibilità e consistenza**: codice più leggibile del SQL grezzo, con pattern e convenzioni uniformi.

Gli svantaggi da tenere presenti:

- **Learning curve** più ripida e alcune operazioni più complesse che con SQL puro.
- **Vendor lock-in**: può essere difficile migrare a un ORM diverso.
- **Performance**: possibile overhead, soprattutto con uso improprio.

## Organizzazione di un'app Express

Express è unopinionated: non esiste un unico modo giusto di organizzare un'app. Un'organizzazione possibile prevede queste cartelle:

- `views`: contiene i template.
- `routes`: definizione dei Router, che dirigono le richieste al controller appropriato.
- `controllers`: business logic per la gestione delle richieste.
- `models`: definizione dei modelli dati e della connessione al database.
- `middleware`: middleware custom.
- `public`: file statici organizzati in sottodirectory.

## Configurazione dell'applicazione

Le web app dipendono da informazioni **sensibili** (API key, password del database, credenziali AWS) e da impostazioni **specifiche dell'ambiente** (URL del database di produzione/sviluppo/staging, livello di logging, …).

Hardcodare questi valori nel codice è una cattiva pratica per tre motivi:

- Non è sicuro quando il codice viene versionato in un repository pubblico.
- Cambiare un'impostazione può richiedere modifiche a più file.
- Produce "versioni" diverse dell'app (sviluppo, produzione…) da aggiornare e mantenere separatamente.

Il file di configurazione deve essere il punto unico dove cambiare queste impostazioni:

- Il file reale (es. `.env`) **non va versionato**: va inserito nel `.gitignore`.
- Si può versionare un file **dummy** (es. `.env.dummy`) che ciascuno rinomina e adatta ai propri valori.
- Pacchetti come **dotenv** leggono il file `.env` e ne rendono disponibile il contenuto, es. nell'oggetto globale `process.env`.

## To-do List con Express

Lab

### Configurazione dei middleware

La configurazione tipica dell'app registra, in ordine, template engine, logging, file statici, parsing del body, sessioni e gestione dei messaggi flash:

```JavaScript
app.set("view engine", "pug");
app.use(morgan('dev'));
app.use(express.static("public"));
app.use(express.urlencoded({ extended: false }));
app.use(session({ secret: 'fu-tzu the great master', resave: false,
  saveUninitialized: false, cookie: { maxAge: 10*60*1000 } }));
```

- **Sessione**: `maxAge` del cookie espresso in millisecondi (nell'esempio, 10 minuti).
- Completano la configurazione `flash()` e middleware custom che esportano i messaggi flash e lo stato di autenticazione verso le viste.

### Rotte e gestione degli errori

La configurazione delle rotte monta i vari Router con `app.use(router)` e in coda:

- Una rotta **catch-all** (`app.get('/*splat')`) che, se raggiunta, risponde `404` renderizzando la pagina d'errore.

- Un **error handler** con firma `(err, req, res, next)`: logga lo stack e risponde con status `err.status || 500` renderizzando la pagina d'errore con codice e descrizione.

Ogni router può registrare un middleware comune a tutte le sue rotte con `todoRouter.use(enforceAuthentication)`, e le rotte delegano al controller, che restituisce Promise; gli errori vengono inoltrati con `next(err)`.

### Autenticazione con sessioni

Il controller di autenticazione verifica le credenziali cercando un utente con `findOne` su username e password della richiesta; se le credenziali sono valide crea la sessione impostando `req.session.isAuthenticated = true` e salvandovi lo username, altrimenti fallisce l'autenticazione.
Il middleware che impone l'autenticazione è il seguente:

JavaScript

```
export function enforceAuthentication(req, res, next) {
  if (!req?.session?.isAuthenticated) {
    req.flash("error", "You must be logged in to access this feature.");
    res.redirect("/auth");
  } else { next(); }
}
```

Il middleware redirige gli utenti non autenticati alla pagina di login con un messaggio flash d'errore; altrimenti passa il controllo al successivo. I controller usano lo username in sessione per filtrare i dati dell'utente corrente, es. con `Todo.findAll({ where: { ... } })` sulla chiave esterna dell'utente.

Capitolo 19

# Lezione 19 REST API

## Dal server-centric all'architettura a due componenti

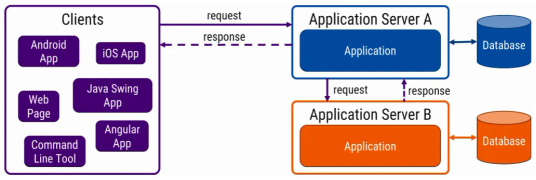
Nelle web app «tradizionali» il server si occupa di tutto: dati (con l'ORM), business logic (rotte, controller, middleware, sessioni) e presentazione (rendering dei template); il browser riceve pagine HTML generate dinamicamente, che una volta arrivate sono statiche.

Questo approccio non è abbastanza flessibile negli scenari moderni, in cui i dati devono essere accessibili anche ad altro software (es. app mobile):

- L'altro software deve scaricare le pagine HTML e **parserle** per estrarre i dati rilevanti.
- Il trasferimento è **inefficiente**: si spostano molti dati non necessari.
- Non è **robusto**: qualsiasi modifica all'HTML può rompere l'estrazione dei dati.

Il trend attuale è dividere la web app in due componenti che comunicano via Internet:

- **Backend** (lato server): responsabile di dati e business logic.
- **Frontend** (lato client): responsabile della UI; può essere un'app nativa mobile o desktop, oppure una pagina HTML "più smart" che rende un'interfaccia interattiva tramite JavaScript (Single Page Application).




## API e REST

Definition

**Application Programming Interface (API)**

**Insieme di modalità** con cui i programmi comunicano tra loro.

Per la comunicazione via Internet si potrebbe definire manualmente un protocollo su TCP/UDP con socket e simili, ma è poco conveniente e poco interoperabile: integrare n API richiederebbe di imparare n protocolli diversi. Protocolli e architetture dedicate come CORBA, Java RMI o SOAP reinventano un'alternativa al web. La scelta del corso è usare direttamente i protocolli del web, cioè HTTP.

Definition

**REST**

**Stile architettonale**, non uno standard né un protocollo, cioè un insieme di principi e linee guida che definisce come usare gli standard del web nelle API, basato su HTTP e URI.

REST fornisce un'interfaccia comune e consistente basata sull'uso «corretto» di HTTP ed è lo stile architettonale prevalente per le web API.

### Fondamenti

Una REST API permette di interagire con **risorse** tramite HTTP:

- Ogni risorsa è associata a una **URI unica**: una risorsa è «qualsiasi cosa sia abbastanza importante da essere referenziata come cosa a sé» e corrisponde tipicamente, ma non necessariamente, a oggetti di dominio persistenti.
- I **verbi HTTP** vanno usati per recuperare o manipolare le risorse.
- L'API deve essere **stateless**: niente sessioni lato server, il che ne migliora la scalabilità.

Il nome Representational State Transfer richiama il **transferimento di stato**: lo stato dell'applicazione vive sul client, lo stato della risorsa vive sul server, e vengono trasferiti tramite rappresentazioni appropriate (es. JSON, XML):

```JSON
{ "todos": [
  { "todo": "Learn REST", "done": false },
  { "todo": "Learn JavaScript", "done": true }
] }
```

L'uso tipico dei verbi HTTP su una collezione di to-do è il seguente:

| Verbo HTTP e URI | Significato |
|---|---|
| `GET /todos` | Recupera la lista di tutti i to-do salvati |
| `GET /todos/<ID>` | Recupera solo il to-do con quell'`<ID>` |
| `POST /todos` | Salva un nuovo to-do (dati nel body della richiesta) |
| `PUT /todos/<ID>` | Sostituisce (o crea) il to-do con quell'ID, usando i dati nel body |
| `DELETE /todos/<ID>` | Elimina il to-do con quell'ID |

#### Risorse nidificate

A volte esiste una gerarchia tra risorse (es. una company ha dipartimenti, che hanno impiegati); si possono allora definire **nested resources**, es. `GET /sellers/{id}/reviews` elenca le recensioni di un dato venditore. Ogni risorsa però deve avere **una sola URI**.

#### Archetipi di risorse

Quando si disegna una API si possono considerare quattro archetipi fondamentali di risorsa:

- **Document**: concetto singolare, corrisponde a un'istanza di un oggetto o a un record del database; es. `/leagues/serie-a/teams/napoli`. > Regola: *i nomi (non numerici) di un document sono singolari.*
- **Collection**: directory di risorse gestita dalla web app; il client può proporre l'aggiunta di nuove risorse, ma è la risorsa a decidere se accettarla; es. `/leagues`, `/leagues/serie-a/teams`. > Regola: *i nomi sono plurali.*
- **Store**: directory di risorse gestita dal client, che può creare, recuperare, aggiornare e cancellare le risorse; es. `/users/1234/favourites`. > Regola: *i nomi sono plurali.*
- **Controller**: modella un concetto procedurale, una sorta di funzione eseguibile invocabile con parametri che produce un output o un effetto collaterale; es. `/alerts/1234/resend`, `/login`. > Regola: *i nomi sono verbi o frasi verbali.*

#### Query string: filtering e paginazione

La componente query della URI serve a **filtrare** collection e store:

- `GET /users` recupera tutti gli utenti; `GET /users?role=admin` solo quelli col ruolo specificato.
- Sono ammessi parametri strutturati: `GET /users?age.gt=18&age.lte=40`.

La stessa componente serve a **paginare** i risultati: `GET /todos?size=10&page=2` restituisce i risultati dall'11° al 20° (cioè la pagina 2).

Quando la complessità dei criteri supera le capacità espressive della query string, si può introdurre un **controller dedicato** nella collection o nello store, es. `POST /users/search`, che riceve i criteri di filtro complessi come oggetto nel body della richiesta.

## Implementazione di una REST API

Una REST API è concettualmente semplice: restare in ascolto di richieste HTTP e gestirle, esattamente come nelle app tradizionali. Le differenze sono:

- Si risponde con una rappresentazione più machine-friendly (es. JSON) invece dell'HTML da renderizzare.
- Si ricevono gli input in rappresentazione machine-friendly invece che tramite form.
- Non ci si può basare sulle sessioni: servono altri schemi di autenticazione.

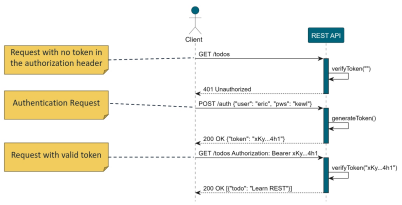
# Sicurezza della API: token e JWT

Una REST API permette di manipolare risorse e in genere non tutti devono poterlo fare:

- **Autenticazione**: solo utenti legittimi possono accedere alle risorse.
- **Autorizzazione**: alcuni utenti possono accedere solo ad alcune risorse.

Uno schema di autenticazione ampiamente usato è **token-based**, con questo flusso:

1. Il client invia una richiesta con username e password alla API.
2. La API valida le credenziali e genera un token.
3. Il token (una stringa) viene restituito al client.
4. Il client rimanda il token alla API in ogni richiesta successiva che richiede autenticazione, nell'header `Authorization`.
5. La API verifica il token prima di rispondere.

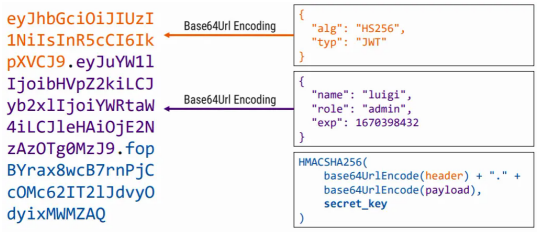



**Definition**

### JSON Web Token (JWT)

**Standard aperto (RFC 7519)** ampiamente adottato che permette di condividere in modo sicuro claim tra due parti; il token è una stringa di tre parti separate da `.`, nella forma `Header.Payload.Signature`.

- **Header**: JSON con il tipo di token e la funzione di hashing usata per firmarlo (es. `{"alg": "HS256", "typ": "JWT"}`), codificato con Base64Url.
- **Payload**: JSON contenente i **claim**, personalizzabili; alcuni claim registrati sono raccomandati, es. `exp` che indica la scadenza del token; anche questo è Base64Url.
- **Signature**: funzione di hash crittografica applicata alla concatenzione di header, payload e una **secret key** nota solo al server emittente; garantisce che il token non sia manomesso o contraffatto.




**Warning**

### Base64Url è codifica, non cifratura

Header e payload di un JWT sono semplicemente codificati e **possono** essere invertiti con facilità: chiunque può leggerne il contenuto. La sicurezza sta nella firma, non nella segretezza del contenuto.

**Definition**

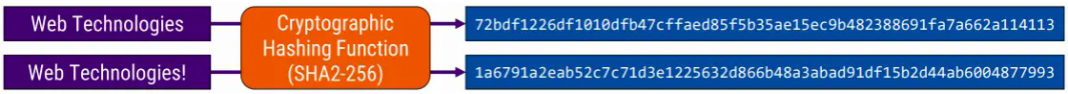
### Funzione di hash crittografica

**Funzione** che mappa una stringa binaria arbitraria (input) a una stringa binaria di dimensione fissa (digest o hash).

Le funzioni di hash crittografiche sicure hanno due proprietà chiave:

- Sono **praticamente senza collisioni**: è impossibile trovare due input diversi con lo stesso digest.
- Sono **facili da calcolare, impossibili da invertire**: dato un input è facile calcolare il digest, ma dato un digest è infeasibile risalire all'input che l'ha generato.

La firma JWT è in realtà un po' più complessa di una semplice hash (es. algoritmi HS256 e RS256).




La configurazione tipica registra questi middleware:

- `morgan('dev')` per il logging.
- `cors()`, che rende l'API accessibile da qualsiasi origine.
- `express.json()`, che parsa le richieste con payload JSON.

L'**error handler** risponde in JSON con status e descrizione dell'errore, usando `err.status || 500`.

L'emissione e verifica dei token usa la libreria `jsonwebtoken`:

```JavaScript
static issueToken(username) {
  return Jwt.sign({ user: username }, process.env.TOKEN_SECRET,
    { expiresIn: `${24*60*60}s` });
}
static isTokenValid(token, callback) {
  Jwt.verify(token, process.env.TOKEN_SECRET, callback);
}
```

Il middleware di autenticazione legge l'header `authorization`, estrae il token e lo verifica:

- Token assente o non valido → `next({ status: 401, message: "Unauthorized" })`.
- Token valido → salva lo username decodificato in `req.username` e chiama `next()`.

**Autorizzazione**: oltre all'autenticazione, occorre verificare che l'utente possa interagire con quella specifica risorsa (es. modificare solo i propri to-do). Un middleware dedicato, es. `ensureUsersModifyOnlyOwnTodos`, controlla con il controller di autenticazione se l'utente ha il permesso su quella risorsa; in caso negativo passa a `next({ status: 403, ... })`: **403 Forbidden**, non 401.

## Regole di design REST

Un buon insieme di regole di base per il design di API RESTful è riportato di seguito; strumenti come RESTRuler rilevano automaticamente le violazioni di queste regole (REST antipatterns) nelle web API.

| Regola | Identificatore | Categoria |
|---|---|---|
| `401` per problemi con le credenziali del client | RC401 | Codici di stato HTTP |
| Nome plurale per collection e store | PluralNoun | URI Design |
| Nome singolare per document | SingularNoun | URI Design |
| Nessuna slash finale nelle URI | NoTrailingSlash | URI Design |
| Verbo o frase verbale per i controller | VerbController | URI Design |
| Niente nomi CRUD nelle URI | NoCRUDNames | URI Design |
| `Content-Type` sempre usato | ContentType | Metadata Design |
| Descrizione della richiesta coerente col tipo di richiesta | DescriptionType | Metadata Design |
| Slash iniziale `/` per la gerarchia | ForwardSlash | URI Design |
| `GET` e `POST` non devono "tunnelare" altri metodi | NoTunnel | Request Methods |
| `GET` solo per recuperare una rappresentazione di una risorsa | GETRetrieve | Request Methods |
| Trattini `-` per migliorare la leggibilità delle URI | Hyphens | URI Design |
| Lettere minuscole preferite nei path delle URI | Lowercase | URI Design |
| Niente underscore _ nelle URI | NoUnderscores | URI Design |

## OpenAPI (Approfondimento)

Definition

**OpenAPI**

**Standard formale e aperto** per descrivere API HTTP, noto un tempo come Swagger; le descrizioni sono documenti strutturati in JSON o YAML che rappresentano un oggetto JSON.

Le ragioni dell'esistenza dello standard:

- **Standardizzazione**: pratiche documentative consistenti.

- **Documentazione**: generazione automatica di documentazione completa.
- **Code generation**: generazione automatica di librerie client e server stub.
- **Leggibilità umana e machine-readable**: abilita la discovery automatizzata e altro.

È lo standard industriale più adottato. Una specifica minima contiene:

- `openapi`: la versione della specifica.
- `info`: informazioni sull'API (es. `title`, `version`).
- `paths`: gli endpoint dell'API, chiamati paths; per ciascun endpoint i metodi supportati e, per metodo, parametri attesi (path, header, …), request body e risposte possibili.

YAML

```yaml
openapi: 3.1.0
info:
  title: A minimal specification
  version: 0.0.1
paths: {} # No endpoints defined
```

Gli strumenti open source principali sono **Swagger Editor** (scrivere la specifica e vedere la documentazione), **Swagger Codegen** (stub server in molti linguaggi/framework: ASP.NET, Go, Java, Node.js, Python, Spring, …; client SDK in C#, Dart, Go, Java, Kotlin, PHP, Python, TypeScript, …) e **Swagger UI** (documentazione interattiva).

Info

**OpenAPI-driven development**

Approccio in cui si definisce la specifica dell'API **prima** di scrivere il codice, sfruttando al massimo la generazione di codice; la definizione dell'API tiene allineati i team coinvolti (front-end, back-end, QA).

In Express si usano due pacchetti: **swagger-jsdoc**, che genera la specifica OpenAPI a partire da annotazioni nei commenti al codice sorgente, e **swagger-ui-express**, che serve la documentazione Swagger UI su un path scelto (es. `/api-docs`):

JavaScript

```javascript
/**
 * @swagger
 * /auth:
 *   post:
 *     description: Authenticate user
 */
authenticationRouter.post("/auth", async (req, res) => { /* ... */ });
```

Capitolo 20

# Lezione 20 TypeScript

## JavaScript e i tipi

JavaScript ha un type system peculiare, con due caratteristiche fondamentali:

- È **dinamicamente tipizzato**: una variabile non ha un tipo fisso, che dipende dal valore assegnato (`undefined`, poi `number`, poi `string`, …).
- È **debolmente tipizzato** ed esegue cast impliciti nelle espressioni, es. in `"100" - 1 + "42"` la sottrazione converte a numero e la somma finale converta a stringa, producendo `"9942"`.

Queste caratteristiche rendono il linguaggio pratico per manipolare piccole pagine web, senza dichiarare tipi o cast espliciti. Al crescere della complessità dei programmi però:

- È più facile introdurre bug subdoli.
- L'esperienza di sviluppo peggiora: scarso supporto IDE, pochissimi errori colti prima del runtime.
- Il codice diventa meno manutenibile e difficile da capire.

Tipici errori rilevabili solo a runtime in JavaScript puro: invocare come funzione una variabile che non lo è (`TypeError`), assegnare a una proprietà col nome sbagliato senza alcun errore, usare un metodo inesistente (es. `add` su un array).

## Transpiler

Definition

**Transpiler**

**Strumento** che traduce codice scritto in un nuovo linguaggio di programmazione, con feature aggiuntive, in codice JavaScript standard, eseguibile ovunque JavaScript sia supportato.

Con l'uso di JS per programmi sempre più complessi, gli sviluppatori hanno sentito il bisogno di superarne i limiti; un nuovo linguaggio non era fattibile perché JS era già ampiamente supportato, quindi sono nati molti linguaggi che compilano verso JS: CoffeeScript, TypeScript, Dart, Elm e centinaia d'altri.

Definition

**TypeScript**

**Superset di JavaScript** nato nel 2012, disegnato e sviluppato da Microsoft (core dev: Anders Hejlsberg), che aggiunge la tipizzazione statica e altro; compila verso JavaScript ed è di gran lunga il "sapore" di JS più popolare.

Si installa come pacchetto npm con `npm install -g typescript`, che rende disponibile il compilatore `tsc`; in alternativa si esegue `tsc` da un pacchetto locale con `npx`. I file TypeScript hanno estensione `.ts`.

## Compilazione e downleveling

Il codice JavaScript standard è anche TypeScript valido: un file `.ts` contenente solo JS può essere eseguito direttamente con Node, ma senza type checking.

Il compilatore rileva i bug prima del runtime anche su codice JS puro, es. un numero di argomenti inferiore a quello atteso da una funzione (`TS2554`). Con `tsc file.ts` il compilatore produce `file.js`, eseguibile con Node come al solito; il codice generato preserva commenti e indentazioni e sembra scritto da un essere umano.

Definition

**Downleveling**

**Riscrittura** da parte del compilatore di feature di versioni JavaScript recenti in codice per versioni più vecchie: di default TypeScript targetizza ES3 (1999), quindi i template string vengono convertiti in concatenazioni `.concat()`; è possibile scegliere il target, es. `tsc --target es6`.

Il downleveling permette di usare le feature moderne pur girando su browser vecchi.

# Dichiarare i tipi

TypeScript permette di dichiarare tipi per variabili, parametri e valori di ritorno; i tipi primitivi più comuni sono `string`, `number` e `boolean`:

```typescript
let courseName: string = "Web Technologies";
let credits: number = 6;
let isGreat: boolean = true;
```

Il codice con annotazioni di tipo non è più JavaScript valido: l'esecuzione diretta con Node fallisce con un `SyntaxError`, quindi occorre compilare il file `.ts` prima di eseguire il `.js` generato. Il compilatore segnala le assegnazioni di tipo incompatibile, es. assegnare una stringa a una variabile `number` (`TS2322`).

# `any` e inferenza dei tipi

- `any`: tipo speciale con cui si può fare qualsiasi cosa senza che il compilatore si lamenti; il codice compila ma gli errori tornano a manifestarsi a runtime.
- **Type inference**: senza annotazione esplicita, TypeScript deduce il tipo dal contesto, es. una stringa letterale diventa `string`, un array di numeri diventa `number[]`; il tipo inferito viene comunque applicato, quindi assegnare dopo un tipo diverso è errore.
- **Implicit any**: quando il tipo non è specificato né deducibile, il compilatore defaulta a `any`; il flag `noImplicitAny` trasforma ogni `any` implicito in errore di compilazione (`TS7006`).

# Tipi strutturali

### Array e object types

- **Array**: si specificano con la sintassi `type[]`, es. `let langs: string[]`; passare un elemento di altro tipo è errore (`TS2345`).
- **Object types**: si definiscono elencando le proprietà e i loro tipi, es. `{name: string, credits: number}`; il compilatore segnala le proprietà mancanti (`TS2453`) e le proprietà extra non previste (`TS2353`).
- **Proprietà opzionali**: si dichiarano aggiungendo `?` dopo il nome, es. `credits?: number`; in tal caso il valore può essere `undefined` e va controllato prima dell'uso.

### Union types

Definition

**Union type**

**Tipo** ottenuto combinando due o più tipi con `|`, che rappresenta valori appartenenti a uno qualsiasi dei tipi combinati.

TypeScript garantisce che le operazioni su un union type siano valide per **ogni** tipo dell'unione: chiamare `toUpperCase()` su `number | string` è errore (`TS2339`).

Definition

**Narrowing**

**Restrizione** del tipo entro un ramo del codice tramite controlli specifici, es. `typeof x === "string"` o `Array.isArray(x)`; TypeScript inferisce il tipo specifico per ciascun ramo.

```typescript
function printError(code: number | string) {
  if (typeof code === "string") {
    console.log(code.toUpperCase()); // in questo ramo code: string
  } else {
    console.log(code); // in questo ramo code: number
  }
```

```
7 }
```

### Type alias e interface

- **Type alias**: assegnano un nome a un tipo con la sintassi `type Nome = definizione`, per riusare lo stesso tipo più volte senza ripetersi (principio DRY).
- **Interface**: altro modo di nominare object types, con la sintassi `interface Nome { ... }`.

```TypeScript
type Credits = number | string;
type Course = { name: string, credits: Credits };
```

**Definition**

**Structural typing**

**Natura di TypeScript**: la compatibilità tra tipi è determinata solo dalla struttura e dalle capacità dei tipi, non dai nomi; un oggetto con `name: string` e `age: number` è compatibile sia con `interface Person` sia con `interface Pet` che abbiano quei membri. Java, al contrario, ha un type system **nominativo**.

Le interfacce si estendono con `interface Bird extends Pet`, che eredita le proprietà; assegnare un oggetto senza la proprietà richiesta della "madre" è errore (`TS2322`).

Per estendere i type alias si usano gli **intersection types**, creati con `&`: combinano più tipi in uno, il cui risultato ha le proprietà di ciascun tipo (es. `type Bird = Pet & { flies: boolean }`).

**Info**

**type vs interface**

Sono molto simili e spesso intercambiabili; la distinzione chiave è che un **type alias non può essere riaperto** per aggiungere proprietà (una seconda dichiarazione omonima è un errore `TS2300`), mentre un'**interfaccia è sempre estendibile**: più dichiarazioni omonime si fondono in una sola.

### Altri strumenti di tipizzazione

- **Type assertions**: sintassi `x as T` per dire al compilatore un tipo più specifico di quello inferibile, es. `document.getElementById("name") as HTMLInputElement`, che altrimenti sarebbe `HTMLElement`.
- **Type literals**: in posizione di tipo si possono usare stringhe e numeri specifici, es. `let alignment: "center"` accetta solo quel valore; poco utili da soli.
- **Literals + unions**: esprimono tipi che corrispondono a un insieme chiuso di valori noti: `type Alignment = "center" | "left" | "right"`; valori fuori dall'insieme sono errore di compilazione.
- **Enum**: permettono di definire un insieme di costanti nominate, es. `enum Priority { Low, Medium, High }`; i membri hanno valori numerici progressivi a partire da 0, quindi `Priority.Medium` vale `1`.
- **Function types**: espressioni che descrivono firme di funzione, es. `type stringDecoratorFunction = (x: string, mode: boolean) => string`; una funzione assegnata deve avere la firma compatibile.

**Warning**

**Type assertion a compile-time**

Un'asserzione di tipo non è un'asserzione in senso di testing e **non è applicata a runtime**: è solo una promessa al compilatore, che non verifica nulla dell'effettivo contenuto del valore.

## Generics

Il problema di partenza: una funzione come l'identità, con parametri e ritorno `any`, è generica ma **perde l'informazione sul tipo** in uscita; dichiararla su un tipo fisso funziona solo con quello.

**Definition**

**Type variable (generics)**

**Variabile speciale** che opera sui tipi anziché sui valori, dichiarata dopo il nome della funzione tra `<` e `>`; è uno degli strumenti principali per scrivere codice che lavora con una varietà di tipi preservando l'informazione su di essi.

```typescript
function identity<Type>(x: Type): Type { return x; }
let s: string = identity<string>("Hello"); // Type impostato esplicitamente
let n: number = identity(42);              // inferenza automatica
```

Usare i generics non equivale a usare `any`: il tipo di input viene **preservato** in output. Altre possibilità:

- **Più variabili di tipo**: `function merge<T1, T2>(x: T1, y: T2)`.
- **Array parametrici**: `function getFirst<T>(arr: T[]): T` ritorna il primo elemento di un array di `T`.
- **Classi generiche**: `class CustomCollection<T>` incapsula `list: T[]` con metodi tipizzati su `T`.
- **Vincoli**: `<T extends Animal>` restringe il type variable ai tipi compatibili con l'animale; per il typing strutturale, anche un oggetto letterale con le proprietà richieste soddisfa il vincolo.

## Altri tipi da conoscere

- **`void`**: il tipo di ritorno delle funzioni che non restituiscono un valore; è il tipo inferito per le funzioni senza `return` o con `return` vuoto.
- **`unknown`**: rappresenta qualsiasi possibile valore; è simile ad `any` ma più sicuro, perché **non consente alcuna operazione** senza un narrowing o un'asserzione (accedere a `a.name` è errore `TS18046`).
- **`never`**: rappresenta valori mai osservati; è il tipo di un ramo irraggiungibile (es. dopo aver escluso con `typeof` tutti i tipi di un union) o di funzioni che non ritornano mai, perché terminano sempre con un'eccezione.

## Livelli di strictness

Di default TypeScript offre un'esperienza **opt-in**: i tipi sono facoltativi e si usa `any` quando non si può dedurre un tipo preciso. Per evitare di essere troppo invasivo, alcuni controlli sono disattivati di default:

- `noImplicitAny`: segnala come errore ogni `any` implicito.
- `strictNullChecks`: di default `null` e `undefined` sono assegnabili a qualsiasi tipo; dimenticare di gestirli esplicitamente è la causa di innumerevoli bug nel mondo (il cosiddetto **billion dollar mistake**), e il flag li obbliga a una gestione esplicita (es. `TS18048`: "possibly `undefined`").

Un caso tipico: `Array.prototype.find` può restituire `undefined`, e chiamare un metodo sul risultato senza controllarlo è esattamente l'errore che `strictNullChecks` intercetta a compile-time.

Capitolo 21

# Lezione 21 Frontend Tooling

## Perché serve del tooling

I siti costruiti con le tecnologie fondamentali (HTML, CSS, JS) avevano un solo foglio di stile, poco client-side JavaScript e nessuna libreria esterna. I frontend moderni sono molto più complessi:

- Il codice client-side JavaScript cresce ed è sempre più complesso.
- Si usano molti moduli e librerie esterne.
- Serve transpilare e fare downleveling del codice.
- Si vogliono ottimizzare i file serviti.
- Si usano pre/post-processori per i fogli di stile, es. SASS.

Per sviluppare frontend complessi serve quindi del **tooling** che automatizzi questi passi: pre/post-processori CSS, framework CSS, bundling e minification, server di sviluppo e build.

## CSS pre e post processori

Definition

**CSS pre-processore**

**Compilatore speciale** che genera codice CSS a partire da file scritti in una sintassi propria, che estende il CSS base con nuove feature (es. variabili) per scrivere fogli di stile mantenibili in modo più efficiente.

I pre-processori più diffusi sono **Sass**, LESS e Stylus.

Definition

**CSS post-processore**

**Strumento** che processa e trasforma automaticamente fogli di stile CSS esistenti.

I post-processori si usano tipicamente per:

- **Downleveling**: riscrivere feature CSS recenti in sintassi compatibile con i browser vecchi (es. `oklch()` convertito in `rgb()`).
- **Autoprefixing**: aggiungere i prefissi vendor (es. `-webkit-`, `-o-`) alle proprietà che li richiedono.
- **Ottimizzazioni**: minificazione o eliminazione delle parti non usate.
- **Applicare convenzioni e rilevare errori**.

Il post-processore più popolare è **PostCSS**.

## SASS

Definition

**Sass**

**Pre-processore CSS** che si autodefinisce «CSS with superpowers», il più popolare in circolazione, attivo dal 2006.

Si installa facilmente via npm con `npm install -g sass`, che rende disponibile il binario `sass`; la compilazione è `sass input.scss output.css`.

Sass supporta **due sintassi**:

- **SCSS** (estensione `.scss`): superset del CSS, la più popolare.
- **Indented syntax** (estensione `.sass`): basata sull'indentazione come Python, era la sintassi originale.

### Variabili e nesting

- **Variabili**: dichiarate con `$nome`, permettono di riusare valori nel foglio di stile; alla compilazione vengono sostituite col loro valore.

- **Nesting**: le regole possono essere annidate seguendo la gerarchia degli elementi HTML, es. `a` dentro `span` dentro `.elem`; il CSS generato espande i selettori combinati (`.elem span a`).

```scss
$primary-color: #333;
nav {
  ul { margin: 0; padding: 0; list-style: none; }
  li { display: inline-block; }
  a { display: block; padding: 6px 12px; text-decoration: none; }
}
```

Warning

**Nesting eccessivo**

Troppo nesting è una cattiva pratica: porta a CSS eccessivamente specifico.

### Mixin

Definition

**Mixin**

**Gruppo riutilizzabile** di dichiarazioni CSS/Sass, dichiarato con l'at-rule `@mixin` seguita da un identificatore e incluso con `@include`; può accettare uno o più parametri opzionali, anche con valori di default.

```scss
@mixin theme($color: DarkBlue) {
  background: $color;
  color: #fff;
}
.info   { @include theme; }
.danger { @include theme($color: DarkRed); }
```

### Moduli e partial

- **Moduli**: un file può usare le definizioni di un altro con `@use 'base'` e accedervi col namespace, es. `base.$primary-color`.
- **Partial**: i file Sass il cui nome inizia con underscore (es. `_variables.scss`) sono pensati per essere inclusi da altri file; Sass **non genera** un CSS standalone per essi, utile, ad esempio, per un file di sole variabili.

### Liste, mappe e control flow

- **Lista**: sequenza di valori separati da virgole, spazi o slash, purché coerente nella lista.
- **Map**: sequenza di coppie `<chiave>: <valore>` con chiavi uniche, es. `("xs": 480px, "md": 768px)`.

Sass fornisce **regole di controllo del flusso** per decidere se e quante volte emettere gli stili, come un template engine per generare CSS:

- `@if` / `@else`: emette stili in modo condizionale.
- `@each`: itera su tutti gli elementi di una lista o mappa, es. per generare media query per ogni breakpoint.
- `@for` e `@while`: le consuete semantiche di iterazione.

## Framework CSS

I framework CSS non sono framework software veri e propri: sono un insieme di **stili general-purpose** pronti all'uso. Risolvono problemi ricorrenti:

- Creare un sistema di layout.
- Creare una tipografia consistente.
- Stilizzare tabelle, liste e componenti UI.
- Azzerare gli stili user agent per un aspetto consistente su ogni browser.

I framework più diffusi sono **Bootstrap** (il più popolare, originario di Twitter), **Tailwind** (noto per l'approccio

*utility-first*), Materialize, Foundation, Bulma e PureCSS.

### Uso e personalizzazione

Il framework è un insieme di file CSS da importare nelle pagine, usando le classi documentate: si possono servire dal proprio server o tramite **CDN**, comodo per il prototipazione, ma solleva qualche problema di privacy.

Due approcci per non avere un sito identico a mille altri:

- **Personalizzare la build**: la maggior parte dei framework è costruita con Sass o simili; si possono cambiare le variabili di build (colori, font, dimensioni…) e ricompilare il framework, es. importare `bootstrap.scss` dopo aver ridefinito `$primary` e `$secondary`.
- **Override CSS**: definire stili custom che sovrascrivono alcuni stili del framework.

## Bundling e minification

Definition

**Bundling**

**Tecnica** che combina più file CSS o JS in un unico file, così il browser esegue meno richieste HTTP per caricare la pagina.

Il bundling è un'ottimizzazione delle prestazioni ampiamente adottata perché ogni richiesta HTTP aggiuntiva introduce latenza: le strategie di caching aiutano i caricamenti successivi, ma il primo caricamento resta penalizzato da molti stylesheet e script.

Warning

**L'ordine conta**

L'ordine con cui i file vengono concatenati nel bundle è significativo: può cambiare il comportamento della pagina.

Nota storica: prima che i moduli ES fossero disponibili nei browser, non c'era modo di scrivere codice JS modulare per il client; i bundler (Webpack, Rollup, Parcel) si occupavano di concatenare i moduli sorgente in un modo comprensibile ai browser.

Definition

**Tree shaking**

**Eliminazione del codice morto** (dead code elimination) eseguita dai bundler: analizzando import ed export, pota dal bundle finale tutto il codice non usato dei moduli importati, con un impatto significativo sulle dimensioni del bundle.

### Minification

Definition

**Minification**

**Processo di ottimizzazione** del trasferimento dei file che rimuove gli spazi "non necessari" e accorcia il codice sorgente senza impattarne la funzionalità; i file risultanti sono detti *minified*.

Gli umani hanno bisogno di indentazione e nomi di variabile significativi per leggibilità, manutenibilità e capibilità; i browser non necessitano di codice leggibile. L'effetto cresce con la dimensione dei file:

- React 18.2.0 pesa ~85 kB, la versione minificata ~25 kB: ~70% di compressione.
- Moment.js 2.29.4 pesa ~151 kB, la versione minificata ~75 kB: ~50% di compressione.

## Vite

Definition

**Vite**

**Strumento di supporto** allo sviluppo e al deploy di progetti web, con due componenti chiave: un **development server** con funzionalità avanzate per lo sviluppo, e un **build tool** che produce asset statici altamente ottimizzati, pronti per la produzione.

Un progetto Vite si crea con `npm create vite@latest`, scegliendo framework e variante (es. TypeScript). Lo scaffolding generato prevede tre script:

- `npm run dev`: avvia il dev server, che osserva i cambiamenti ai sorgenti e ricarica automaticamente l'app quando serve (**live reload**).

- `npm run build`: genera un bundle ottimizzato, pronto per la produzione.
- `npm run preview`: un semplice server statico che serve i file generati in fase di build.

Per usare un foglio di stile Sass nel progetto si installa `npm install -D sass`, si crea il file `.scss` e lo si importa nel `main.ts`: Vite lo rileva, lo compila con `sass` e lo aggiunge al bundle dopo il CSS originale.

### La fase di build

Durante la build Vite svolge parecchio lavoro automaticamente:

- Analizza le risorse statiche effettivamente usate nell'app.
- Compila i file Sass (risp. TypeScript) in CSS (risp. JavaScript).
- Raggruppa in un singolo bundle i file CSS e i file JavaScript.
- Minifica i bundle.
- Applica il **filename fingerprinting** ai bundle finali.
- Sposta tutte le risorse in `/dist` e aggiorna l'`index.html` con i riferimenti corretti.

Definition

**Filename fingerprinting**

**Tecnica** che nomina i bundle finali con un hash del loro contenuto (es. `index-DiAK_ebR.js`): quando il contenuto cambia, cambia anche il nome del file, il che evita i problemi di **cache staleness**, il browser non continua a usare la vecchia versione in cache come farebbe con un nome fisso come `index.js`.

Capitolo 22

# Lezione 22 Single Page Applications

## App multi-page

Nelle web app tradizionali le pagine sono generate dinamicamente sul server e inviate al browser per la renderizzazione: una volta arrivate, sono intrinsecamente statiche.

Quando l'utente cambia pagina (es. clicca su un link) accade quanto segue:

- Una **nuova richiesta** viene inviata al server.
- La **nuova pagina** è restituita in risposta e visualizzata dal browser.

Questo è l'approccio **multi-page**: ogni navigazione provoca un **full-reload** della pagina.

Definition

**Single Page Application (SPA)**

**Applicazione web** che reagisce all'input dell'utente aggiornando dinamicamente il contenuto della pagina corrente, invece di caricare una nuova pagina dal server.

Le caratteristiche chiave di una SPA:

- L'intera applicazione è caricata nella **richiesta iniziale**; la pagina non viene mai ricaricata completamente.
- La complessità del rendering delle viste passa dal server al codice client-side (da *thin client* a *thick client*).
- Sono applicazioni ad alta densità di JavaScript.
- La navigazione a una vista diversa **non** provoca il full-reload: i dati necessari vengono recuperati da un backend (es. REST) e il JavaScript lato client crea dinamicamente la nuova pagina.

### Perché le SPA

Le SPA offrono alcuni vantaggi sulle app multi-page tradizionali:

- **Esperienza utente più dinamica**, simile alle app desktop.
- Il caricamento iniziale è più lento (serve scaricare tutta l'app), ma i successivi sono molto più veloci.
- **Carico del server ridotto**: si recuperano solo i dati necessari e solo quando servono, senza trasferire continuamente interi documenti HTML, con risparmi anche di banda.

## Client-side routing

Definition

**Client-side routing**

**Pratica** con cui, in una SPA, la navigazione e l'aggiornamento delle viste sono gestiti interamente lato client, senza ricaricare la pagina dal server.

Le diverse "pagine" di una SPA sono dette **viste**. Il client-side routing comporta:

- Aggiornare l'URL del browser.
- Caricare e renderizzare il contenuto (vista) appropriato nella pagina.
- Interagire con la **Browser History API**, così che i pulsanti indietro e avanti funzionino come in qualunque app multi-page.

I componenti chiave del client-side routing sono tre:

- **Router**: modulo che gestisce la mappatura tra URL e viste.
- **View rendering**: meccanismo per renderizzare le viste diverse.
- **History management**: meccanismo che usa la History API per manipolare la cronologia di navigazione del browser.

### Anatomia di una SPA minimale

C'è un **singolo documento HTML**, senza alcun contenuto: solo un `<div id="app">` vuoto e lo script principale caricato come modulo. Il grosso del lavoro avviene nel client-side JavaScript.

Il file principale costruisce tre elementi nel contenitore: una navbar e un footer, che non cambiano al variare della vista, e un contenitore `main` per il contenuto, che cambia.

Le rotte associano le viste ai path URL; ciascuna rotta è indicizzata per path e ha due proprietà: `title` (il titolo della pagina quando quella vista è caricata) e `render` (una funzione che ritorna la stringa HTML della vista):

```JavaScript
const routes = {
  "/":       { title: "Home", render: home },
  "/about":  { title: "About", render: about },
};
```

La funzione `router()` implementa la logica di routing:

```JavaScript
function router() {
  let view = routes[location.pathname];
  if (view) {
    document.title = view.title;
    content.innerHTML = view.render();
  } else {
    history.replaceState("", "", "/");
    router();
  }
}
```

Se una rotta combacia col path corrente, aggiorna il titolo e sostituisce il contenuto col risultato della `render` della vista; altrimenti sostituisce la voce di cronologia corrente con il path `/` e richiama se stessa.

### Gestione della navigazione

In una SPA possono esserci link che devono comportarsi normalmente (es. verso siti esterni) e link che devono solo aggiornare la vista; questi ultimi si marcano con un **attributo custom**, es. `data-link`, e si gestiscono con un listener globale:

```JavaScript
window.addEventListener("click", e => {
  if (e.target.matches("[data-link]")) {
    e.preventDefault();               // no full-reload
    history.pushState("", "", e.target.href);
    router();
  }
});
```

Il click su un link interno viene intercettato: si previene il comportamento di default, si aggiunge il nuovo stato sulla pila della cronologia con `pushState`, e si invoca `router()` per aggiornare la vista secondo l'URL corrente.

Infine, `router()` va eseguita anche in due occasioni:

- **`popstate`**: quando l'utente preme il pulsante indietro del browser.
- **`DOMContentLoaded`**: quando la pagina principale viene (ri)caricata completamente.

## Componenti

Man mano che la complessità del frontend cresce, gestire intere viste come monoliti diventa rapidamente infeasibile: la codebase diventa difficile da mantenere ed è complicato riusare pezzi di funzionalità in viste diverse.

La soluzione è **decomporre la UI in componenti modulari**; ogni componente incapsula una funzionalità specifica,

è il solo responsabile del proprio HTML e del proprio stato interno, e le viste che lo usano non si preoccupano di questi dettagli.

Definition

**Componente**

**Unità modulare della UI** che racchiude una specifica funzionalità, riusabile in più parti dell'applicazione.

Le caratteristiche chiave di un componente sono:

- **Modularità**: è autonomo e incapsula un pezzo specifico di funzionalità.
- **Encapsulation**: racchiude la propria business logic, il proprio stato, layout e stili.
- **Riuso**: è progettato per essere riusato in parti diverse dell'app.

I componenti hanno un ruolo centrale in tutti i framework frontend. Con i **custom elements** se ne può creare uno da zero estendendo `HTMLElement` e registrandolo:

JavaScript

```
class Counter extends HTMLElement {
  constructor() {
    super();
    this.innerHTML = "<button>Click</button>";
    this.querySelector("button").onclick = () => { /* ... */ };
  }
}
customElements.define("click-counter", Counter);
```

Una vista usa il componente semplicemente includendo il suo tag, es. `<click-counter></click-counter>`, dopo averne importato il codice.

## Recupero dei dati: AJAX e XMLHttpRequest

Prima dell'introduzione della Fetch API in ES6, le richieste di rete si effettuavano con **AJAX**, acronimo di *Asynchronous JavaScript And XML*, implementato tramite l'interfaccia **XMLHttpRequest**.

Info

**XMLHttpRequest oggi**

La Fetch API lo sta sostituendo, ma `XMLHttpRequest` è ancora comune e si trova in molti pacchetti esistenti.

Il workflow tipico era:

- Creare un oggetto `XMLHttpRequest`.
- Impostare una **callback** da eseguire quando la richiesta è completata.
- Inizializzare la richiesta con `open` (metodo HTTP, URL, flag asincrono).
- Inviare la richiesta con `send`.

JavaScript

```
let xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) { /* uso della risposta */ }
};
xhttp.open("GET", "/url/to/fetch", true);
xhttp.send();
```

## Le sfide delle SPA e i framework

Implementare una SPA da zero è istruttivo, ma fa emergere alcune difficoltà:

- **State management**: tenere stato e UI sempre aggiornati, e riflettere correttamente le viste dopo ogni cambiamento di stato.
- **DOM manipulation**: l'API `document` richiede molto codice boilerplate.
- **Client-side routing**: un router fatto a mano è basilare; servirebbero rotte parametriche, redirect, …
- **Efficient rendering**: si ridisegna tutta la lista a ogni cambiamento, senza ottimizzazioni.

I **framework frontend** come Angular, React, Vue.js, Ember e Svelte forniscono soluzioni pronte per queste sfide e, in genere, anche supporto di tooling (generazione di boilerplate, automazione di build e deploy) e un modo "standardizzato" di costruire i frontend, che facilita il passaggio di uno sviluppatore tra progetti che usano lo stesso framework.

Capitolo 23

# Lezione 23 Frontend Frameworks Angular

## Frontend framework

Definition

**Frontend framework**

**Insieme predefinito** di componenti software, strumenti e best practice che fa da fondamento per sviluppare Single Page Application.

Le SPA portano con sé una complessità nota: client-side routing, view rendering, componenti e riuso del codice. I framework frontend affrontano esattamente queste sfide.

## Angular

Definition

**Angular**

**Framework frontend** tra i più usati al mondo, sviluppato da Google, particolarmente diffuso in Italia e per le applicazioni enterprise; la versione 2.0 è del 2016.

Le sue caratteristiche distintive sono:

- **Focus sul tooling e sulla produttività** dello sviluppatore.
- **Batteries included**: è un framework opinionato e completo (include meccanismi di dependency injection, routing, …), che riduce la *decision fatigue* e garantisce consistenza tra progetti.
- **Supporto alla migrazione**: tooling dedicato per migrare a versioni più recenti.
- **Best practice da subito**: impone TypeScript.

### CLI

L'interfaccia a riga di comando è il punto d'ingresso tipico del lavoro con Angular:

- `npm install -g @angular/cli` installa la CLI, che espone il comando `ng`.
- `ng new nome-app` crea un nuovo progetto.
- `ng serve` avvia il dev server (di default su `http://localhost:4200/`).
- `ng generate component nome` genera il boilerplate di un componente.

Per l'ambiente di sviluppo si usa Visual Studio Code, con l'estensione raccomandata (ma facoltativa) **Angular Language Service**.

## Componenti

I componenti sono i **mattoni** delle app Angular: ciascuno è responsabile di funzione e aspetto di un elemento di pagina, codice (stato, business logic, eventi), layout HTML e stile CSS. Un componente può contenerne altri: un'app Angular è un **albero di componenti**.

Anatomicamente, un componente è una **classe TypeScript** annotata col decorator `@Component`, cui si passa un oggetto di metadati:

- **`selector`**: dice ad Angular dove renderizzare il componente; per ogni elemento HTML che combacia col selettore viene creata un'istanza del componente (es. il componente con `selector: 'app-root'` renderizza dentro `<app-root>`).
- **`template`** / **`templateUrl`**: il layout HTML, inline o in file separato.
- **`styles`** / **`styleUrl`**: gli stili CSS (o Sass), inline o in file separato.
- **`standalone: true`** e **`imports`**: dichiarano il componente come standalone e importano ciò che usa.

```typescript
@Component({
  selector: 'app-root',
  standalone: true,
  template: "<h1>Hello {{name.toUpperCase()}}!</h1>",
  styles: "h1 { color: darkblue; }",
})
export class AppComponent { name = 'Web Technologies'; }
```

Le espressioni in `{{ }}` nel template vengono valutate e il risultato, convertito in stringa, è renderizzato nella pagina.

### Standalone components

Definition

**Standalone component**

**Componente** con `standalone: true` nei metadati, che importa direttamente gli altri componenti, direttive e pipe usati nei suoi template.

Le versioni di Angular precedenti alla 14 usavano un meccanismo diverso, gli **NgModule**; nel corso si usano solo standalone component.

### Stato ed eventi

- Lo **stato** del componente sono le proprietà della classe, referenziabili nel template con l'interpolazione; se una proprietà cambia, la vista si aggiorna.
- Gli **eventi** si collegano a handler con la sintesi delle parentesi, es. `(click)="increment()"`, dove `increment()` è un metodo della classe del componente.

## Control flow nei template

I template Angular supportano gli scenari comuni con la nuova sintassi `@if` / `@for` (prima di Angular 17 si usavano le direttive `ngIf` e `ngFor`):

```HTML
<button (click)="increment()">Click me</button>
<ul>
  @for (ts of clickTimestamps; track ts) {
    <li>{{ts}}</li>
  } @empty {
    <li>Button has never been clicked!</li>
  }
</ul>
```

- **`@if` / `@else`**: renderizza una porzione di template in modo condizionale, es. `@if (counter === 1) {time} @else {times}`.
- **`@for`**: itera su una collezione rendendo la stessa porzione di template per ogni elemento; richiede `track` per identificare gli elementi; il blocco `@empty` è renderizzato quando la collezione è vuota.

## Comunicazione tra componenti

I componenti devono a volte condividere dati: il genitore configura il figlio o gli passa dati da visualizzare, il figlio comunica al genitore che un evento è avvenuto. I due pattern si implementano con `@Input()` e `@Output()`.

### Da genitore a figlio: `@Input()`

Nel componente figlio, le proprietà che il genitore può passare si dichiarano col decorator `@Input()`, con un valore di default usato quando il genitore non ne passa uno:

```typescript
export class CounterComponent {
  @Input() buttonText = "Click me";
  counter: number = 0;
}
```

Il genitore passa il valore impostando una proprietà sull'elemento HTML: `<app-counter buttonText="CLICK HERE" />`.

Info

**@Input() e input()**

In molti framework lo stesso meccanismo si chiama *Props* (es. React, Vue). Da Angular 19 l'input/output si gestisce anche con le funzioni `input()` e `output()`, in un approccio basato su signal.

### Property binding

Definition

**Property binding**

**Associazione dinamica** tra un attributo di un elemento HTML o di un componente nel template e una proprietà del componente corrente: il nome dell'attributo va tra parentesi quadre `[...]` e il valore è la proprietà collegata, es. `<app-counter [buttonText]="btnMsg" />`.

Il binding e l'interpolazione non sono sempre equivalenti:

- L'**interpolazione** valuta l'espressione e converte il risultato in stringa.
- Il **property binding** passa il valore così com'è: per passare un oggetto o un array come proprietà di un elemento serve il binding, non l'interpolazione.

### Da figlio a genitore: `@Output()`

Definition

**@Output()**

**Canale di comunicazione** da figlio a genitore: una proprietà del figlio decorata con `@Output()` e di tipo `EventEmitter`; il figlio emette un evento invocando `emit()` sulla proprietà.

```typescript
export class CounterComponent {
  @Output() counterClickedTenTimes = new EventEmitter<number>();
  increment() {
    this.counter++;
    if (this.counter % 10 === 0)
      this.counterClickedTenTimes.emit(this.counter);
  }
}
```

Il genitore reagisce agli eventi del canale definendo un handler, come per un evento ordinario: `<app-counter (counterClickedTenTimes)="addClicks($event)" />`.

## Routing

Una SPA deve gestire il client-side routing: il **Router** di Angular abilita la navigazione tra viste interpretando l'URL come istruzione per cambiare vista. I progetti creati con `ng new` hanno già il routing configurato.

### Setup

Il setup del Router tocca tre file:

- **`main.ts`**: carica il componente radice con `bootstrapApplication(AppComponent, appConfig)`.
- **`app.config.ts`**: esporta l'`appConfig` con `provideRouter(routes)`, che prepara i provider necessari al modulo Router.

- **`app.routes.ts`**: definisce l'oggetto `routes` di tipo `Routes` (un array di `Route`); ciascuna `Route` ha, tra le altre, le proprietà `path` (URL associato), `title` (titolo della pagina HTML) e `component` (componente associato al path).

```TypeScript
export const routes: Routes = [
  { path: "", component: HomePageComponent, title: "Home Page" },
  { path: "counter", component: CounterPageComponent, title: "Counter" },
];
```

Col setup in atto, il componente **`RouterOutlet`** analizza l'URL corrente e renderizza il componente della rotta combaciante al posto del tag `<router-outlet></router-outlet>` nel template radice.

### Navigazione senza full-reload

Usare link tradizionali con `href` fa eseguire al browser una nuova richiesta HTTP e ricarica tutta l'app: sconfigge lo scopo stesso di una SPA.

**Definition**

**RouterLink**

**Direttiva** da applicare agli elementi che innescano la navigazione, es. `<a routerLink="/counter">Counter</a>`, per usare il Router e cambiare vista senza ricaricare la pagina: è così che una SPA gestisce il routing.

Se l'utente digita manualmente l'URL, avviene un page load e il Router renderizza comunque il componente giusto.

### Link attivi

**Definition**

**RouterLinkActive**

**Direttiva** che applica una lista di classi CSS ai link corrispondenti alla rotta su cui l'app si trova correntemente, per ricordare all'utente in quale pagina è.

Si usa come `routerLinkActive="active"`, abbinata a un po' di CSS (es. `nav .active { font-weight: bold; }`); con `routerLinkActiveOptions` si può richiedere la combaciatura esatta del path.

Capitolo 24

# Lezione 24 Angular (parte II)

## Form

I form sono in molti casi il modo preferito di raccogliere input dell'utente. Angular li gestisce in due modi:

- **Template-driven**: le associazioni tra i dati del componente e il form sono definite implicitamente nel template, tramite direttive.
- **Reactive**: le associazioni sono definite esplicitamente nel componente.

### Template-driven

La direttiva `[(ngModel)]="prop"` lega il valore di un campo del form a una proprietà del componente. La notazione `[()]`, detta **"banana in a box"**, rappresenta un **two-way binding**: property binding ed event binding insieme.

Il two-way binding funziona in entrambe le direzioni: digitando nell'input si aggiorna la proprietà del componente (e con essa il template), e cambiando la proprietà si aggiorna il campo dell'input.

```HTML
<input id="pwd" name="pwd" type="password" [(ngModel)]="pass" />
```

### Reactive

Con l'approccio reactive il form è un oggetto creato nel componente: un **`FormGroup`** con dentro un **`FormControl`** per ogni campo, col valore iniziale come argomento.

```TypeScript
loginForm = new FormGroup({
  user: new FormControl(''),
  pass: new FormControl('')
});
```

Nel template si collega il gruppo con `[formGroup]="loginForm"` e i singoli campi con `formControlName`, e i valori si leggono da `loginForm.value.user`.

### Submission

L'invio del form si gestisce con l'event binding `(ngSubmit)="onSubmit()"` sull'elemento `<form>`, dove `onSubmit()` è un metodo del componente che accede ai valori del form.

## Services e Dependency Injection

Definition

**Service**

**Classe** che fornisce funzionalità usabili in più parti dell'applicazione, senza template né stili: solo logica invocabile quando serve.

I service sono ideali per compiti richiesti in punti diversi dell'applicazione, come recuperare dati da una REST API, fare logging o validare dati. Le classi dei componenti restano responsabili solo dell'esperienza utente: template, stili e la logica che media tra vista e business logic.

Definition

**Dependency Injection (DI)**

**Pattern** usato da Angular per far ottenere al codice le istanze delle proprie dipendenze (es. i service), separando le preoccupazioni di costruzione e di uso delle dipendenze e migliorando il disaccoppiamento.

Nel sistema DI di Angular esistono due ruoli: il **dependency consumer** (il codice scritto) e il **dependency provider** (in genere gestito da Angular).

Un service si dichiara iniettabile col decorator `@Injectable`, dove `providedIn: 'root'` lo rende disponibile ovunque nell'app (root injector):

TypeScript

```
@Injectable({ providedIn: 'root' })
export class CalculatorService {
  sum(x: number, y: number): number { return x + y; }
}
```

La dipendenza si ottiene in due modi:

- Con la funzione `inject()` in una proprietà: `private calc = inject(CalculatorService)`.
- Dichiarandola nel costruttore della classe: `constructor(private calc: CalculatorService) {}`.

# Proteggere le rotte

Un compito comune del routing è garantire che solo utenti autorizzati accedano a certe rotte.

Definition

**Route Guard**

**Funzione** invocata da Angular per decidere se l'utente può compiere una certa operazione su una rotta; i tipi includono `canActivate`, `canMatch`, `canLoad` e `canDeactivate`.

Le guardie si specificano come proprietà degli oggetti `Route`, es. `canActivate: [authorizationGuard]`. Se ne possono impostare più di una: Angular le esegue tutte in parallelo e l'azione è permessa solo se tutte restituiscono `true`.

TypeScript

```
export const authorizationGuard: CanActivateFn = (route, state) => {
  const router = inject(Router);
  const authService = inject(AuthService);
  if (authService.isUserLoggedIn()) return true;
  router.navigateByUrl("/login");
  return false;
};
```

Una guardia può restituire tre cose:

- **`true`**: la guardia è soddisfatta e la navigazione può procedere.
- **`false`**: la guardia non è soddisfatta e l'azione non può essere compiuta.
- **Un `UrlTree`**: il Router deve redirigere a quell'URL.

Warning

**Redirigere con UrlTree**

Con più guardie eseguite in parallelo, restituire sempre un `UrlTree` per la redirezione e non usare direttamente i metodi `Router.navigate()`: due guardie che redirigono possono creare race condition.

# HttpClient

Angular fornisce un'API per le richieste HTTP nel service **`HttpClient`**, da configurare con la dependency injection in `app.config.ts` tramite la funzione dedicata `provideHttpClient()`:

```typescript
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(withFetch(), withInterceptors([authInterceptor])),
    provideRouter(routes)
  ]
};
```

`HttpClient` si ottiene come dipendenza di service e componenti (con `inject()` o via costruttore) e offre metodi corrispondenti ai verbi HTTP; ogni metodo, es. `this.http.get<T>(url)`, restituisce un **`Observable`** dalla libreria **RxJS**.

## Observables

Angular usa gli Observable per gestire molte operazioni asincrone. A grandi linee la differenza con le Promise è questa:

- Una **Promise** può risolvere (o rigettare) una sola volta: emette asincronamente un singolo valore (o un errore).
- Un **Observable** può emettere asincronamente **più valori nel tempo**.

Per ricevere gli eventi si **fa subscribe** all'Observable, passando un oggetto con fino a tre callback:

```typescript
observable.subscribe({
  next(value)  { /* un valore è stato emesso */ },
  error(err)   { /* un errore è stato emesso */ },
  complete()   { /* l'Observable ha finito di emettere */ }
});
```

L'argomento del costruttore `new Observable(subscriber => { ... })` è la funzione da eseguire al momento della sottoscrizione; al suo interno si chiamano `subscriber.next(value)`, `subscriber.error(err)` o `subscriber.complete()`.

**Warning**

**Errore come completamento**

Gli errori sono considerati completamenti: dopo un `error()` l'Observable non emette più nulla e la callback `complete()` **non** viene invocata.

## Interceptor

**Definition**

**Interceptor**

**Forma di middleware** di HttpClient per le richieste in uscita: una funzione che riceve la `HttpRequest` corrente e una funzione `next` che rappresenta il passo successivo della catena di interceptor.

```typescript
function log(req: HttpRequest<unknown>, next: HttpHandlerFn) {
  console.log(req.url);
  return next(req);
}
```

Casi d'uso tipici: aggiungere header di autenticazione alle richieste (es. token JWT), mettere in cache le risposte per un periodo, loggare le richieste in uscita. Gli interceptor da usare si dichiarano configurando `provideHttpClient(withInterceptors([...]))`.

## Change detection

Quando una proprietà di un componente (il suo stato) cambia, Angular aggiorna automaticamente il suo template: aggiornare le viste solo quando necessario e in modo efficiente (solo le parti da aggiornare) è la chiave delle prestazioni

di una SPA.

Il funzionamento in sintesi:

- Quando qualcosa avviene nell'app (eventi DOM, operazioni asincrone, …) scatta il meccanismo di **change detection**.
- Angular percorre tutta la **gerarchia dei componenti**.
- Per ciascun componente controlla se lo stato è cambiato e se il cambio ha effetto sulla vista.
- In tal caso aggiorna la porzione di DOM corrispondente al template del componente.

# Signals

Definition

**Signal**

**Variabile che notifica** gli interessati quando il suo valore cambia; traccia in modo granulare come e dove lo stato è usato nell'app, permettendo ad Angular di evitare change detection non necessarie e aggiornare solo le parti di DOM cambiate.

I signal sono una via alla **programmazione reattiva**, forma di programmazione dichiarativa basata sull'elaborazione di eventi asincroni. Si dividono in **scrivibili** e **read-only**.

- Uno signal scrivibile si crea con `signal(valoreIniziale)`; la lettura avviene invocandolo come funzione, es. `{{count()}}` nel template.
- **`.set(v)`** imposta un nuovo valore; **`.update(fn)`** calcola il nuovo valore a partire dal precedente.

```TypeScript
count = signal(0);
handleClick() { this.count.update(value => value + 1); }
handleReset() { this.count.set(0); }
```

### Computed ed effects

- **`computed(fn)`**: crea un signal **read-only** che deriva il proprio valore da altri signal; Angular lo mantiene aggiornato in modo reattivo; chiamare `.set()` o `.update()` su di esso è errore.
- **`effect(fn)`**: crea una funzione eseguita ogni volta che i signal letti al suo interno cambiano; gli effect vengono eseguiti **almeno una volta**, e durante l'esecuzione registrano i signal letti, così da rieseguirsi al loro cambiamento.

```TypeScript
constructor() {
  effect(() => { console.log(`Count: ${this.count()}`); });
}
```

Capitolo 25

# Lezione 25 Angular To-do List SPA

Questa lezione applica i concetti di Angular a un'app To-do List completa: frontend Angular 19 e backend REST (l'API Express delle lezioni precedenti). Gli schemi architetturali nuovi o degni di nota sono riportati di seguito.

## Rotte e protezione

Le rotte dell'app dichiarano, oltre a `path` e `component`, anche `title` (titolo della pagina per la vista) e, per le rotte riservate, la guardia `canActivate`:

```TypeScript
{ path: "todos", component: TodoPageComponent,
  title: "To-do List | To-do List Angular App",
  canActivate: [authGuard] },
{ path: "todos/:id", component: TodoDetailComponent,
  canActivate: [authGuard] }
```

La rotta `todos/:id` mostra il **path param** `:id`, che il componente dettaglio usa per identificare la risorsa.

La guardia segue lo schema raccomandato: verifica l'autenticazione e, in caso negativo, **non** naviga direttamente ma restituisce un `UrlTree`, che il Router usa per redirigere:

```TypeScript
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);
  if (authService.isUserAuthenticated()) {
    return true;
  } else {
    return router.parseUrl("/login"); // restituisce un UrlTree
  }
};
```

## Login con reactive form

Il componente di login usa un `FormGroup` con `FormControl` e **validators** sincroni: `Validators.required`, `Validators.minLength(4)`, `Validators.maxLength(16)`.

Il metodo di submit segue il pattern consolidato:

- Se il form è **invalido**, si mostra un messaggio d'errore e non si procede.
- Altrimenti si inviano le credenziali al `RestBackendService`, sottoscrivendo l'`Observable` con le tre callback `next`, `error` e `complete`:
  - `next`: si salva il token restituito dal backend.
  - `error`: credenziali non valide, messaggio d'errore.
  - `complete`: si naviga verso la pagina dei to-do.

Nel template gli errori dei controlli si mostrano solo quando serve, col nuovo control flow `@if`, verificando la proprietà `submitted` e gli `errors` del controllo:

```HTML
@if (submitted && loginForm.controls.pass.errors) {
  @if (loginForm.controls.pass.errors['required']) {
    <p class="form-error">Password is required.</p>
```

```
  }
}
```

## `AuthService` con signal

Il servizio di autenticazione incapsula lo stato in un **signal** di tipo `WritableSignal<AuthState>`, dove lo stato contiene `user`, `token` e `isAuthenticated`. Il token e lo user sono inizializzati da `localStorage`, verificando alla partenza se il token non è scaduto.

Dal signal si espongono tre **computed**: `user`, `token` e `isAuthenticated`.

TypeScript

```
user = computed(() => this.authState().user);
token = computed(() => this.authState().token);
isAuthenticated = computed(() => this.authState().isAuthenticated);
```

La sincronizzazione con `localStorage` si affida a un **effect** nel costruttore: ogni volta che `token` o `user` cambiano nello stato, l'effect li riscrive in `localStorage`, o li rimuove se valgono `null`.

## Servizio REST e intercettore

Il servizio verso il backend incapsula `HttpClient` con le opzioni HTTP condivise (header `Content-Type: application/json`) e un metodo per endpoint:

TypeScript

```
login(loginRequest: AuthRequest) {
  return this.http.post<string>(`${this.url}/auth`, loginRequest, this.httpOptions);
}
```

Definition

**Auth interceptor**

**Interceptor** di HttpClient che clona ogni richiesta in uscita aggiungendo l'header `Authorization: Bearer <token>` quando il token è presente, così che ogni chiamata autenticata non debba occuparsene manualmente.

TypeScript

```
function authInterceptor(request: HttpRequest<unknown>, next: HttpHandlerFn) {
  const authService = inject(AuthService);
  const token = authService.getToken();
  if (token) {
    request = request.clone({ setHeaders: { Authorization: 'Bearer ' + token } });
  }
  return next(request);
}
```

## Pagina dei to-do e componente figlio

La pagina dei to-do dichiara un array `todos: TodoItem[]`, un form reattivo per il nuovo elemento, e delega a `RestBackendService` e ai toast le notifiche; in `ngOnInit()` chiama `fetchTodos()`.

Il singolo elemento della lista è un **componente separato**, incaricato di visualizzare un to-do; il collegamento col genitore segue i pattern `@Input()` / `@Output()`:

```TypeScript
export class TodoItemComponent {
  @Input({ required: true }) todoItem: TodoItem;
  @Output() delete: EventEmitter<number | undefined> = new EventEmitter();
}
```

- `@Input({ required: true })`: il genitore **deve** passare la proprietà, non è più opzionale.
- `@Output() delete`: il figlio emette l'evento (es. l'id dell'elemento) e il genitore lo gestisce con `(delete)="handleDelete($event)"`.

Il template del genitore itera la lista col control flow `@for`, con `track item.id` e blocco `@empty` per la lista vuota:

```HTML
<ul>
  @for (item of todos; track item.id) {
    <app-todo-item [todoItem]="item" (delete)="handleDelete($event)" />
  } @empty {
    <li>No to-do items to show.</li>
  }
</ul>
```

Capitolo 26

# Lezione 26 Web App Security

## Sicurezza delle web app

Definition

**Web Application Security**

**Insieme di misure e pratiche** implementate per proteggere le web application dalle minacce di sicurezza, con l'obiettivo di garantire la **confidentiality**, l'**integrity** e la **availability** dell'applicazione e dei dati che gestisce.

La sicurezza va gestita a due livelli:

- **Network-level**: riguarda la comunicazione tra web app e Internet, cifratura del traffico (HTTPS), firewall che filtrano richieste HTTP malevole, monitoraggio e logging per rilevare pattern sospetti.
- **Application-level**: riguarda i meccanismi interni della web app, autenticazione e autorizzazione, validazione degli input per l'integrità dei dati, gestione delle sessioni, riservatezza dei dati.

Il livello network protegge da attacchi al livello di rete (es. sniffing, man-in-the-middle, DDoS filtrati da un firewall); gli attacchi applicativi invece possono nascondersi in richieste HTTP perfettamente valide ed exploitare vulnerabilità nel modo in cui l'app è implementata. Nel corso ci si concentra sul livello applicativo, facendo sì che le app sviluppate non contengano vulnerabilità, e sui meccanismi di sicurezza dei browser (es. CORS).

### La catena dello stack moderno

Una web app moderna è molto più del codice che si scrive: è un'intera pila, dove ogni anello può introdurre vulnerabilità, dal proprio codice al codice di terze parti, al framework, al runtime (es. Node, PHP), al sistema operativo, all'hypervisor, fino all'hardware.

Info

**L'anello più debole**

Una web app è sicura solo quanto il suo anello più debole: una RCE nel template engine EJS o in PHP, un open redirect in Express, una vulnerabilità nel kernel Linux o nell'hypervisor possono compromettere tutta l'app.

Le pratiche di igiene di sicurezza che ne derivano:

- Prestare attenzione agli **advisory di sicurezza**.
- Usare solo librerie e framework **affidabili e mantenuti**.
- Mantenere tutto **aggiornato**.
- **Disabilitare** ogni servizio non strettamente necessario.
- Applicare il **principle of least privilege**: ogni modulo deve accedere solo alle informazioni strettamente necessarie al suo scopo.

Info

**OWASP**

L'**Open Worldwide Application Security Project** è un'organizzazione non profit che raccoglie dati sulle vulnerabilità e pubblica report periodici sulle classi di vulnerabilità più diffuse (**OWASP Top 10**); una quota significativa delle web app ne contiene almeno una.

Le vulnerabilità trattate nel seguito sono frequenti, molto dannose e facilmente prevenibili: XSS, CSRF, SQL injection, attacchi alle sessioni, validazione degli input insufficiente.

## Cross-Site Scripting (XSS)

Definition

**Cross-Site Scripting (XSS)**

**Attacco** che inietta codice client-side malevolo in siti altrimenti fidati, exploitando falle dell'app che mostra input non fidato (es. di altri utenti) senza validazione o escaping adeguati.

Il codice iniettato è eseguito dal browser come se fosse codice fidato: è indistinguibile dal JavaScript legittimo della pagina.

Le conseguenze: l'utente può essere attaccato semplicemente cliccando un link malevolo, e il codice iniettato può fare tutto ciò che il JavaScript fidato può fare:

- Accedere e manipolare i cookie (es. gli id di sessione).
- Accedere a `localStorage` e `sessionStorage` (es. token o dati privati).
- Manipolare la pagina (alterare link e form, aggiungere contenuto).
- Generare eventi e compiere azioni come l'utente (es. submit di form).

Warning

**I filtri non bastano**

Filtrare l'input (es. rimuovere `<script>` o i caratteri `<` e `>`) è aggirabile: esistono dedicate **tecniche di filter evasion** (es. `<iframe src="javascript:...">`, tab e caratteri codificati dentro `javascript:`, attributi `onerror` con entità numeriche). Sulla falsariga, l'OWASP pubblica un cheat sheet di evasione dei filtri.

Per esfiltrare un token dalla vitttima, il codice iniettato può redirigere la finestra (o un iframe) verso un URL malevolo, aggiungere un'immagine o caricare un foglio di stile il cui URL contiene il dato rubato (alcuni browser bloccano l'immagine, ma non tutte le vie).

Definition

**Escaping / sanitizzazione**

**Contromisura** fondamentale contro XSS: convertire i caratteri pericolosi (`<`, `>`, `"`, …) in **HTML entity** innocue (es. `<script>`) al momento dell'inserimento dell'input nelle pagine; esistono soluzioni dedicate nei framework e librerie apposite (es. `sanitize` per Node.js). Non affidarsi ai soli filtri.

### XSS in Angular

Angular **sanitizza automaticamente** ogni dato non fidato interpolato o collegato nei template; per default tutti i dati sono non fidati.

Warning

**DomSanitizer bypassa la protezione**

Marcare dati come fidati con `DomSanitizer` (es. `bypassSecurityTrustHtml(value)`) disattiva le contromisure predefinite e va fatto con cautela: nel codice, l'errore XSS era causato proprio da un `bypassSecurityTrustHtml` su un query param non fidato.

## Cross-Site Request Forgery (CSRF)

Definition

**Cross-Site Request Forgery (CSRF o XSRF)**

**Attacco** che spinge un utente finale a eseguire azioni non volute su una web app su cui è attualmente autenticato.

Le azioni mirate sono operazioni che cambiano lo stato (es. trasferimenti di fondi, cambio email dell'account); se la vittima ha privilegi amministrativi, l'attacco può compromettere l'intera applicazione.




Il meccanismo sfruta il fatto che il browser della vittmma **allega automaticamente il cookie di autenticazione** alle richieste verso il sito legittimo, anche se partono da una pagina di un sito malevolo (es. un form `POST` nascosto che punta a `https://bank.com/transfer`, sottomesso automaticamente via JavaScript).

Definition

**Synchronizer token pattern**

**Contromisura** contro il CSRF: la web app genera un **CSRF token** specifico della sessione e lo salva nella sessione; il token è incluso in un campo nascosto dei form sensibili; alla sottomissione l'app verifica che il token ricevuto combaci con quello generato per la sessione corrente. L'attaccante non può indovinarlo.

Il token-based approach funziona purché:

- Non esistano vulnerabilità XSS (che permetterebbero di leggere il token).
- I token CSRF non siano predicibili.

La protezione CSRF è inclusa in molti strumenti: middleware per Express (es. `tiny-csrf`), contromisure in Angular, protezione dei `POST` di Spring attiva di default. Se il framework la offre, va letta la documentazione per usarla correttamente; altrimenti va implementata per le operazioni sensibili.

## SQL injection

Definition

**SQL injection**

**Attacco** che inietta codice malevolo nelle query al database tramite input utente non sanitizzato; nonostante il nome, anche le query sui database NoSQL possono essere ugualmente vulnerabili.

Le conseguenze possibili: leak di dati privati, modifica (create, update, delete) dei dati, violazioni del controllo d'accesso e, in alcuni casi, esecuzione arbitraria di codice sul server del database.

La vulnerabilità nasce quando i parametri della richiesta vengono inseriti **direttamente nella stringa SQL**: l'attaccante può alterare la struttura della query stessa. Con l'email `luigi@webtech-sh.op' --` la query di login diventa:

```sql
SELECT * FROM Users
WHERE email = 'luigi@webtech-sh.op' --' AND password = '...' AND deletedAt IS NULL
```

In SQL il `--` inizia un commento su riga singola: il controllo sulla password viene commentato via e l'autenticazione è aggirata.

Definition

**Prepared statements**

**Contromisura** contro le SQL injection: query con **parametri** (`$1`, `$2`, …) in cui i valori sono passati separatamente (es. con `bind`) e non vengono mai interpretati come parte della struttura della query; in alternativa si usano stored procedure.

```JavaScript
models.sequelize.query(
  'SELECT * FROM Users WHERE email = $1 AND password = $2',
  { bind: [ req.body.email, security.hash(req.body.password) ] }
);
```

## Session hijacking

Definition

**Session hijacking**

**Classe di attacchi** volta a compromettere il token di sessione, tipicamente rubando un token valido o predidendone uno.

Le app vulnerabili a XSS sono una via tipica per il furto del token di sessione, che permette all'attaccante di impersonare la vittmma.

## Validazione degli input

Una corretta **input validation** è gran parte dell'integrità dei dati in una web app.

- La validazione **client-side** è una buona pratica per l'usabilità: gli errori sono presentati all'utente il prima possibile.
- Ma un attaccante può **aggirarla del tutto**: può accedere al codice client e manipolarlo, o generare richieste HTTP direttamente.

Warning

**Validare sempre anche lato server**

La validazione client-side serve all'esperienza utente, non alla sicurezza: la validazione deve essere sempre eseguita anche, e soprattutto, server-side.

## Il modello di sicurezza dei browser

I browser moderni applicano un modello di sicurezza severo, di cui lo sviluppatore web deve conoscere i meccanismi per lavorarci. Il principale è la **Same-origin Policy**.

Definition

**Same-origin Policy (SOP)**

**Politica** per cui, di default, un documento o uno script può interagire solo con risorse della **stessa origine**, cioè con lo stesso protocollo, la stessa porta e lo stesso host.

Rispetto al documento `http://store.webtech.com/cat/index.html`:

| URL | Stessa origine? | Motivo |
|---|---|---|
| `http://store.webtech.com/pages/about.html` | Sì | Differisce solo il path |
| `https://store.webtech.com/page.html` | No | Protocollo diverso |
| `http://store.webtech.com:81/dir/page.html` | No | Porta diversa (`http` usa di default la 80) |
| `http://course.webtech.com/cat/index.html` | No | Host diverso |

La policy si applica ai dati recuperati via JavaScript con `fetch()` o `XMLHttpRequest`, con lievi differenze tra browser.

### CORS

A volte serve accedere a contenuti di origini diverse dalla propria.

Definition

**Cross-Origin Resource Sharing (CORS)**

**Meccanismo** che permette a un server di fare "opt-out" della Same-origin Policy, dichiarando, tramite header dedicati nelle risposte, che il proprio contenuto è accessibile anche da origini diverse dalla sua.

Il meccanismo è **header-based**:

- Il browser allega alle richieste `fetch` in uscita un header **`Origin`** con l'origine della pagina.
- Il server include nella risposta un header **`Access-Control-Allow-Origin`** che specifica quali origini possono usare i dati: `*` significa "qualsiasi origine", oppure un'origine precisa, es. `http://localhost:3001`.
- Senza header, il default è che le richieste cross-origin sono **proibite**.
- Il browser verifica che `Access-Control-Allow-Origin` combaci con `Origin`: in caso positivo il codice client accede ai dati, altrimenti la richiesta è bloccata.

Capitolo 27

# Lezione 27 Web Testing

## Verifica del software

Definition

**Verifica del software**

**Insieme di pratiche** volte ad assicurare che un software soddisfi una specifica, per catturare difetti o bug prima del rilascio del prodotto.

Si divide in due famiglie:

- **Dynamic verification**: coinvolge l'esecuzione del programma; la principale è il **software testing**.
- **Static verification**: non esegue il programma; include code review, static analysis (es. taint analysis) e automated verification (model checking).

Definition

**Software test**

**Sequenza di azioni** progettata per valutare un particolare aspetto o funzionalità di un software; il software sotto test è detto **SUT**.

Un test in genere consiste di tre fasi:

- **Arrange**: garantire che le precondizioni richieste siano soddisfatte.
- **Act**: eseguire una o più azioni.
- **Assert**: controllare che il SUT si sia comportato come inteso.

### Livelli di testing

Il testing si pratica a livelli diversi, che formano la **testing pyramid** (i test più numerosi e rapidi alla base, quelli più lenti e costosi in cima):

- **Unit testing**: testa una singola unità (es. una classe o un metodo).
- **Integration testing**: verifica che unità diverse lavorino insieme come inteso.
- **System testing**: prende di mira il sistema intero contro la sua specifica.
- **End-to-End (E2E) testing**: prende di mira il SUT dal punto di vista degli utenti finali.

Nel testing di web app emergono sfide peculiari:

- A livello **unit**: si deve fare i conti con **codice asincrono** e **dipendenze da servizi esterni**.
- A livello **E2E**: si deve fare i conti con **fragilità** e **flakiness**.

## Unit testing con Angular

Angular include tutto il necessario per testare le app con **Jasmine**; il codice di test vive in **file spec** (specification), generati come stub dalla CLI insieme ai component/service, tipicamente nella stessa directory del codice testato con la convenzione dei nomi `<nome>.spec.ts`.

La struttura tipica di uno spec è:

TypeScript

```
describe('CalculatorService', () => {
  beforeEach(() => {
    TestBed.configureTestingModule({});
    service = TestBed.inject(CalculatorService);
  });
  it('should sum two positive numbers correctly', () => {
    expect(service.sum(42, 58)).toBe(100);
  });
```

```
});
```

- `describe` raggruppa i test; `beforeEach` esegue codice prima di ciascun test (qui: configurare il `TestBed` e iniettare il servizio).
- `it` definisce un singolo test; `expect(...)` con un matcher esprime l'asserzione.

Il comando `ng test` compila l'app e lancia il **Karma Test Runner**, che raccoglie i file spec, li esegue e produce un report di esecuzione.

### Test double e Spy

Definition

**Test Double**

**Sostituto** di un oggetto di produzione usato nei test al posto dell'originale, per testare un'unità in isolamento, senza che l'esito dipenda da unità esterne.

Il problema: le unità testate dipendono da altre (es. un componente dipende da un service, che dipende da `HttpClient`); un fallimento potrebbe dipendere da una qualunque delle dipendenze, rendendo difficile la fault localization, e alcune condizioni reali (es. temperatura sotto zero) sono impraticabili da attendere in un servizio reale.

La **Dependency Injection** è un supporto prezioso: nei test si configura l'injector perché risolva le dipendenze con i Test Double al posto degli originali.

Scriverli da zero resta laborioso, quindi i framework di testing aiutano: **Jasmine** supporta gli **Spy**, test double che possono *mockare* qualsiasi funzione:

JavaScript

```
restBackendSpy = jasmine.createSpyObj('RestBackendService', ['getTodos']);
restBackendSpy.getTodos.and.returnValue(of(todos));
```

Lo spy mocka il metodo `getTodos()` e lo configura per restituire sempre l'Observable di una lista data: il test non dipende né dal service reale né dalla REST API esterna.

Nei test dei componenti si assegnano gli spy alle dipendenze del componente, si invocano i metodi, si forza l'aggiornamento della vista con `fixture.detectChanges()` e si eseguono le asserzioni sia sullo stato (es. lunghezza dell'array) sia sul DOM via `DebugElement` (es. `element.query(By.css('ul'))` e il contenuto testuale della lista).

### Code coverage

Il comando `ng test --no-watch --code-coverage` calcola anche le **metriche di code coverage**, un possibile indicatore di quanto "bene" è stata testata l'app: percentuali di **Statements**, **Branches**, **Functions** e **Lines** coperte. Nella directory `/coverage` viene generato un report HTML ispezionabile file per file, riga per riga.

## End-to-End testing

Definition

**End-to-End (E2E) test**

**Test** che prende di mira il sistema intero dal punto di vista dei suoi utenti finali: ogni test replica un flusso d'uso realistico, interagendo con le interfacce esterne del sistema come farebbero gli utenti.

L'interfaccia esterna dipende dal tipo di web app:

- **REST API**: gli endpoint REST sono l'interfaccia esterna e gli "utenti" sono altri programmi; i test E2E inviano richieste HTTP e controllano che le risposte siano corrette secondo la specifica (es. scenario di login: `POST /auth` con credenziali → aspettarsi `200 OK` con `{token}` JWT valido; scenario di inserimento: autenticarsi, salvare il token, `POST /todos` con header `Bearer <TOKEN>` → aspettarsi `200 OK` col corpo atteso).
- **Web app tradizionale o SPA**: le pagine web sono l'interfaccia esterna e gli utenti sono umani con un browser; i test E2E, detti **E2E web test**, interagiscono con le pagine e controllano che cambino correttamente (es. visitare la pagina di login, riempire i campi, cliccare Login, aspettarsi la redirezione alla homepage e il saluto nella navbar).

Gli step chiave di un E2E web test sono due, ripetuti per ogni passo dello scenario:

1. **Selezionare** l'elemento con cui interagire (bottone, campo di testo, …).
2. **Interagire** con esso (click, fill, verifica di proprietà, …).

### Manuale vs automatizzato

L'E2E testing può essere manuale o automatizzato:

- **Manuale**: tester umani seguono uno script dettagliato di passi, replicando gli scenari utente e riportando i problemi.
- **Automatizzato**: si sviluppa software di test (test suite) che simula gli scenari utente, e che può essere rieseguito più volte.

| | Manuale | Automatizzato |
|---|---|---|
| Competenze | Nessuna conoscenza di programmazione | Programmazione e testing |
| Costi | Lento, tedioso, error-prone | Costo iniziale alto, ma rieseguibile con poco sforzo |
| Scala | Non scala: ripetere a ogni release, su tutti i browser e dispositivi | Serve manutenzione del codice di test al variare dell'app |
| Rischi | , | I test possono essere flaky e/o fragili |

### Locator strategies

Definition

**Locator**

**Strategia** con cui si seleziona l'elemento di una pagina con cui interagire; è una parte cruciale degli E2E web test.

Le tre strategie principali sono:

- **Coordinate assolute**: si identifica l'elemento dalle coordinate sullo schermo, es. `click(200, 300)`; è la strategia più fragile.
- **Visual-based locators**: si identifica l'elemento dall'aspetto, con algoritmi di image-matching (es. SikuliX); robusti ai cambi di layout, fragili ai cambi d'aspetto degli elementi.
- **Layout-based locators**: si identifica l'elemento da proprietà di layout, es. selettori CSS: `click(".item button.buy")`; robusti ai cambi puramente visivi, più fragili ai cambi di layout; sono i più usati nella pratica.

Info

**Il giusto compromesso**

Un buon locator non deve essere né troppo generico (può selezionare l'elemento sbagliato) né troppo specifico (anche un cambiamento minimo lo rompe): anche un piccolo cambiamento nell'app o nell'ambiente di test può rompere i locator, rendendo impossibile interagire con gli elementi giusti e richiedendo una costosa manutenzione per ripararli.

### Flakiness

Definition

**Flakiness**

**Tendenza** dei test E2E a comportarsi in modo inconsistente: lo stesso test può passare o fallire intermittentemente a condizioni identiche (nessun cambiamento nell'app né nel test).

La causa è tipicamente il **non-determinismo** dei tempi di caricamento e di fetch dei dati: una risorsa esterna può impiegare più tempo a restituire i dati da visualizzare, e il test può tentare di accedere a elementi non ancora renderizzati ("element not found"). Si mitiga con appropriate **waiting strategies**.

### Isolamento dei test E2E

I test E2E dovrebbero girare **in isolamento**, con tre benefici:

- **No failure carry-over**: un test che fallisce non influenza i successivi.
- **L'ordine di esecuzione non conta**: i test si possono eseguire in parallelo.
- Ogni test è eseguibile **indipendentemente**, il che facilita il debug.

L'isolamento si ottiene in due modi: **start from scratch**, eseguendo ogni test in un ambiente nuovo e pulito, oppure **clean-up**, ripristinando l'ambiente tra un test e l'altro.

# Playwright

Definition

**Playwright**

**Framework open source** di automazione e testing del browser, sviluppato da Microsoft (prima release: 31 gennaio 2020), cross-browser e cross-platform, event-driven e asincrono, scritto in TypeScript, con API in TypeScript, JavaScript, C#, Java e Python.

Gli altri strumenti principali del settore sono **Selenium** (dal 2009, cross-browser), **Cypress** (dal 2014, cross-browser) e **Puppeteer** (di Google, dal 2017, orientato a Chrome).

Si installa con `npm init playwright@latest`, scegliendo linguaggio, cartella dei test e browser da installare. I test si definiscono con la funzione `test()`, che riceve un titolo e una funzione async col codice del test:

TypeScript

```ts
import { test, expect } from '@playwright/test';
test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  await expect(page).toHaveTitle(/Playwright/);
});
```

Di default i test girano su **chromium** (Blink), **firefox** (Gecko) e **webkit** (il motore di Safari), configurabili in `playwright.config.ts` insieme alle piattaforme; Playwright sa anche emulare browser di tablet e mobile.

### Isolamento e fixture

Playwright ottiene l'isolamento **partendo da zero**: ogni test gira in un ambiente pulito, cioè un nuovo **`BrowserContext`**, una sorta di profilo in incognito, con i propri `localStorage`, `sessionStorage` e cookie, non influenzato dai test precedenti.

L'ambiente di ogni test è definito da **Test Fixtures**: tipicamente Playwright crea un nuovo `BrowserContext` per test e vi fornisce una `Page` di default, un'astrazione di una tab in un browser nuovo. L'argomento `{ page }` nella firma del test chiede a Playwright di configurare la fixture `page`.

### Locating, actions, asserting

I test E2E automatizzati ruotano su tre momenti chiave.

Per **locare gli elementi** Playwright offre funzioni dedicate, oltre ai locator CSS e XPATH con `page.locator(...)`:

- `getByText()`: per contenuto testuale; `getByLabel()`: form control tramite la label associata.
- `getByPlaceholder()`: input tramite il placeholder; `getByAltText()`: elemento (es. immagine) tramite il testo alt.
- `getByTitle()`: tramite l'attributo title; `getByTestId()`: tramite l'attributo `data-testid`.

Per **interagire** con l'elemento locato basta invocare il metodo adeguato: `fill()`, `clear()`, `press()`, `check()`, `selectOption()`, `click()`, `dblclick()`.

Per **asserire** si chiama `expect()` su un valore o un locator seguito dal matcher appropriato: `toBeVisible()`, `toContainText()`, `toHaveClass()`, `toBeDisabled()`, `toBeEmpty()`, `toBeInViewport()`, `toBeDefined()`, …

### Capture and Replay

Definition

**Capture and Replay (C&R)**

**Approccio** in cui il codice di test E2E è generato automaticamente "registrando" (catturando) le interazioni dell'utente con la web app, permettendo di creare test rieseguibili semplicemente usando l'app.

Playwright supporta il C&R col **Test Generator**, accessibile anche dall'estensione VS Code: durante la registrazione il tester interagisce col browser come un utente finale e Playwright cattura le interazioni generando il codice per replicarle, con euristiche integrate per scegliere i locator migliori.

Warning

**Codice generato**

Il codice C&R può essere subottimale rispetto a fragilità e flakiness: va trattato come punto di partenza, non come prodotto finale.